GPUNTOBIT



Il libro di *Triad*

Imparare il linguaggio e il codice del progetto triad2.0, dai primi programmi al seed in codice macchina

I Fondamenti · II Triad-Core · III Triad-Full · IV Dentro il progetto · V Sul campo

```
fattoriale(n) = n < 2 ? 1 : n * fattoriale(n - 1)
```

Prima edizione · Standard del linguaggio 0.2.0 · 2026

github.com/GPuntobit/triad2.0

Indice

1. - Indice -
2. Prefazione
3. Sommario
4. Manuale di Triad 2.0
5. Tutorial di Triad 2.0
6. Quarta di copertina

Prefazione

Prefazione

Triad è un linguaggio di programmazione autonomo: sintassi a blocchi e tipi dichiarati, multiple dispatch, programmazione logica integrata, codice come dato. La sua definizione è una specifica normativa indipendente dall'implementazione, accompagnata da una specifica eseguibile e da una suite di conformità con golden verificabili. Questo libro ne è il percorso di studio.

Il libro nasce da un'esigenza semplice: imparare Triad non significa solo leggere la specifica, ma costruire un modello mentale del linguaggio, dei suoi costrutti e del suo runtime. La specifica è normativa e precisa, ma è scritta per essere implementata, non per essere letta come un romanzo. Questo volume la traduce in un percorso: spiegazioni, esempi, esercizi, marcatori di profilo. Le spiegazioni, gli esempi e gli esercizi costruiscono passo dopo passo il modello mentale del linguaggio. La specifica resta la fonte normativa di riferimento, non un prerequisito di lettura.

Il lettore ideale sa già programmare in un altro linguaggio: sa cosa sono una variabile, una funzione, un ciclo, una ricorsione. Questo libro non insegna la programmazione in generale, insegna Triad. Le influenze del linguaggio (Julia, Prolog, Lisp) sono documentate nell'Appendice A della specifica: sono note informative, non definizioni. Triad non è un dialetto di nessuno dei tre.

Il volume riunisce due testi complementari e indipendenti. Il *Manuale di Triad 2.0* spiega il linguaggio, propone esercizi e chiude con grammatica,

schede rapide, tabella degli errori, glossario e appendici. Il *Tutorial di Triad 2.0* è diviso in cinque parti: primi passi, Triad-Core, Triad-Full, codice del progetto e riferimento. Si può leggere il Manuale per intero senza il Tutorial, e il Tutorial per intero senza il Manuale: ciascuno spiega ciò che gli serve. L'indice che segue raccoglie i capitoli di entrambi i testi, così da ritrovarli in un unico posto.

Se vuoi scrivere programmi eseguibili oggi, comincia dal Tutorial, parti I e II; usa il Manuale per approfondire i concetti e fare gli esercizi. Per studiare lo standard completo, prosegui con la parte III del Tutorial e i capitoli corrispondenti del Manuale. La parte IV è pensata per chi vuole esplorare o modificare il repository.

Le appendici del Manuale orientano tra le influenze del linguaggio e le fonti del progetto. I testi normativi integrali non sono riprodotti nel volume: chi vuole il dettaglio formale li trova nei file `SPEC.md` e `SPEC-CORE.md` del repository, che restano le fonti definitive in caso di divergenza. Il volume contiene comunque le regole necessarie a capire e usare il linguaggio: il rinvio alla specifica è un approfondimento, non una dipendenza.

Due profili di conformità attraversano tutto il libro. Triad-Core è il sottoinsieme di bootstrap: valori, funzioni, chiusure, controllo di flusso, liste, stringhe, I/O. Triad-Full è lo standard completo. Ogni programma Core è anche un programma Full valido e deve comportarsi allo stesso modo. I marcatori **[Core]** e **[Full]** indicano in quale profilo esiste ciascun costrutto; i blocchi di codice sono marcati `triad` per il codice Core e `triad-full` per il codice definito dallo standard Full, non ancora eseguibile dal runtime corrente. Gli esempi Core si possono provare con l'interprete

JavaScript di riferimento (`node seed/dev/triad_core.cjs`); i frammenti indicati come specifici del seed fanno eccezione.

Il seed S0 è un interprete Triad-Core scritto direttamente in codice macchina x86-64, senza un linguaggio ospite; oggi il seed eseguibile è quello per Windows. Non va confuso con l'interprete JavaScript usato per eseguire gli esempi Core e verificare la conformance. Il piano di bootstrap prevede S1 (un interprete Triad scritto in Triad-Core), S2 (compilatore nativo), S3 (self-hosting) e S4 (parità cross-platform). Dove il runtime non copre un costrutto, il testo lo dice esplicitamente.

Buona lettura, e buon codice.

— GPuntobit

Sommario

Sommario

Elenco completo di tutti i capitoli e le sezioni.

Manuale di Triad 2.0

- Manuale di Triad 2.0
- Come leggere questo manuale
- A chi è indirizzato
- Due profili, una sola lingua
- Convenzioni di notazione
- Come eseguire gli esempi
- Parte I — Il linguaggio
- 1. Lessico
- 1.1 Identificatori
- 1.2 Numeri
- 1.3 Stringhe
- 1.4 Simboli
- 1.5 Commenti
- 1.6 Separatori e fine-riga
- 1.7 La regola dei due punti e del punto interrogativo
- 1.8 Parole chiave
- 1.9 Identificatori avanzati e convenzioni
- 1.10 Escape e stringhe avanzate
- 1.11 Esercizi sul lessico
- 2. Valori e tipi
- 2.1 I tipi
- 2.2 Gerarchia dei tipi
- 2.3 Struct
- 2.4 Conversioni

- 2.5 Asserzione di tipo
- 2.6 Il vincolo del dichiarato
- 2.7 Le condizioni accettano solo `Bool`
- 2.8 Come stampa un valore
- 3. Variabili e assegnamento
- 4. Operatori e precedenza
- 4.1 Aritmetica intera
- 4.2 Logica e confronti
- 5. Controllo di flusso
- 5.1 `if` / `elseif` / `else` **[Core]**
- 5.2 `while`, `break`, `continue` **[Core]**
- 5.3 `let` e `begin` **[Core]**
- 5.4 `for` e i range **[Full]**
- 5.5 Le comprehension **[Full]**
- 5.6 `try` / `catch` **[Full]**
- 6. Funzioni
- 6.1 Tre forme di definizione **[Core]**
- 6.2 `return`, ricorsione, arità
- 6.3 Funzioni di ordine superiore
- 6.4 L'annotazione di effetto `!io`
- 7. Tipi e multiple dispatch
- 7.1 Il tipo di ritorno
- 7.2 Funzioni locali
- 8. Lambda, chiusure e scope
- 8.1 Lambda **[Core]**
- 8.2 Chiusure
- 8.3 Le regole di scope

- 9. Liste
- 10. Stringhe, simboli e conversioni
- 11. Errori come dati
- 11.1 La forma `Err`
- 11.2 Produrre e propagare
- 11.3 Le categorie di errore
- 11.4 In Core: errori come liste taggate
- 12. Contratti
- 13. Budget
- 14. Tracce
- Glossario degli eventi di traccia (porte di Byrd)
- 15. Effetti dichiarati
- 16. Logica
- 16.1 Fatti, regole, termini
- 16.2 I goal
- 16.3 `solve` è un valore
- 16.4 Un predicato usato come funzione
- 16.5 Esempi
- 17. Codice come dato
- 17.1 Quote, unquote, eval
- 18. Macro
- 18.1 Codice che genera codice
- 19. Builtin
- 20. Riga di comando e REPL
- 20.1 Il REPL
- 21. Primitivi di bootstrap
- 22. Stile e pattern idiomatici

- 22.1 Funzioni di ordine superiore scritte in Core
- 22.2 Record come liste taggate
- 22.3 Ambiente come lista di associazioni
- 22.4 Stato mutabile condiviso
- 22.5 Costruire stringhe
- 22.6 Regole di stile
- Parte II — Esercizi
- 23. Esercizi guidati (Triad-Core)
- Esercizio 1 — Potenza e somme
- Esercizio 2 — Numeri primi
- Esercizio 3 — Inverti una lista
- Esercizio 4 — Ordinamento per inserzione
- Esercizio 5 — Contatore di parole
- Esercizio 6 — Un mini-analizzatore lessicale
- Esercizio 7 — Valutare un'espressione
- Esercizio 8 — Un ambiente a dizionario
- 24. Esercizi su Triad-Full
- Esercizio 9 — Dispatch su struct
- Esercizio 10 — Una base di conoscenza
- Esercizio 11 — Contratti in pratica
- Esercizio 12 — Una macro che misura
- Parte III — Riferimento
- 25. Grammatica
- 25.1 Triad-Core (EBNF normativa)
- 25.2 Triad-Full (EBNF normativa)
- 26. Schede rapide
- 26.1 Valori

- 26.2 Controllo di flusso
- 26.3 Funzioni
- 27. Tabella degli errori
- 27.1 In Core
- 27.2 In Triad-Full
- 28. Glossario
- 29. Cosa gira oggi
- Appendice A — Influenze dichiarate
- Appendice B — Dove guardare

Tutorial di Triad 2.0

- Triad 2.0 — Tutorial: imparare il linguaggio e il codice del progetto
- Che cosa impari e come è organizzato
- Come usare questo tutorial
- Parte I — Primi passi
- 1. Cos'è Triad
- 1.1 Due profili: Core e Full
- 1.2 La catena di bootstrap
- 2. Installare ed eseguire
- 2.1 Requisiti
- 2.2 Eseguire un file
- 2.3 Materializzare il seed
- 2.4 Launcher `bin/triad` e `bin/triad.cmd`
- 2.5 Codici di uscita
- 2.6 La CLI di sviluppo in breve
- 3. Il primo programma
- 3.1 Argomenti da riga di comando
- 3.2 Il ciclo di lavoro consigliato
- Parte II — Triad-Core passo per passo (eseguibile oggi)
- 4. Lessico
- 5. Valori, tipi e stampa
- 5.1 Regole di stampa di `print`/`println`
- 5.2 Uguaglianza
- 6. Variabili e assegnamento
- 7. Operatori ed espressioni

- 7.1 Precedenza (dalla più bassa alla più alta)
- 7.2 Aritmetica intera
- 7.3 Logica e condizioni
- 8. Controllo di flusso
- 8.1 `if` / `elseif` / `else`
- 8.2 `while`, `break`, `continue`
- 8.3 `let` e `begin`
- 9. Funzioni
- 9.1 Tre modi di definire una funzione
- 9.2 `return`, ricorsione, arità
- 9.3 Annotazione di effetto `!io`
- 10. Lambda, chiusure e scope
- 10.1 Lambda
- 10.2 Chiusure
- 10.3 Regole di scope
- 11. Liste
- 12. Stringhe, simboli e conversioni
- 13. Primitivi di bootstrap
- 14. Errori di esecuzione
- 14.1 Errori come valori, in Core
- 15. Interprete JS e seed Windows: differenze
- 16. Pattern idiomatici Triad-Core
- 16.1 Funzioni di ordine superiore scritte in Core
- 16.2 Record come liste taggate
- 16.3 Ambiente come lista di associazioni
- 16.4 Stato mutabile condiviso
- 16.5 Costruire stringhe

- 16.6 Stile
- 17. Esercizi guidati
- Esercizio 1 — Potenza e somme
- Esercizio 2 — Numeri primi
- Esercizio 3 — Inverti una lista
- Esercizio 4 — Ordinamento per inserzione
- Esercizio 5 — Contatore di parole
- Esercizio 6 — Un mini-tokenizer
- Esercizio 7 — Valutare un'espressione
- Parte III — Triad-Full (lo standard completo)
- 18. Lessico e tipi Full (SPEC §1, §4)
- 19. Funzioni tipate e multiple dispatch (SPEC §5)
- 20. Struct (SPEC §4)
- 21. Controllo di flusso Full: `for`, range, comprehension, `try` (SPEC §2)
- 22. Logica: fatti, regole, `solve` (SPEC §6)
- 23. Destrutturazione (SPEC §2)
- 24. Errori come dati (SPEC §8)
- 25. Contratti (SPEC §9)
- 26. Budget (SPEC §10)
- 27. Tracce (SPEC §11)
- 28. Effetti dichiarati (SPEC §12)
- 29. Codice come dato, albero uniforme e macro (SPEC §3, §7)
- 29.1 L'albero uniforme
- 29.2 Quote, unquote, eval
- 29.3 Macro
- 29.4 Codice che genera codice

- 30. Builtin Full (SPEC §13)
- 31. Riga di comando e REPL (SPEC §14)
- 32. Esempi Full con output golden
- 32.5 Contratti e budget insieme
- Parte IV — Il codice del progetto
- 33. Mappa del repository
- 34. L'interprete di riferimento JS (`seed/dev/triad_core.cjs`)
- 34.1 Architettura
- 34.2 Esempio: aggiungere un builtin
- 34.3 Esempio: aggiungere sintassi
- 35. Il seed S0 Windows e il suo generatore
- 35.1 Cos'è
- 35.2 Convenzioni interne
- 35.3 Il generatore `seed/dev/gen_seed.js`
- 35.4 Flusso per modificare il seed
- 36. Il seed Linux (E0)
- 37. Self-hosting: S1 in `selfhost/`
- 37.1 Come si scrive codice S1 oggi
- 37.2 Dove va S1
- 38. Conformance: leggere, scrivere e verificare casi
- 38.1 Formato
- 38.2 Il runner `conformance/run.mjs`
- 38.3 Aggiungere un caso Core
- 39. La CLI di sviluppo `triad-cli`
- 39.1 Uso
- 39.2 Struttura del codice
- 39.3 Aggiungere un comando

- 39.4 Eseguibile standalone
- 40. Benchmark AI (`ai/`)
- 41. Editor, launcher e sito
- 42. Integrazione continua
- 43. Flusso di contributo
- Parte V — Riferimento
- 44. Grammatica Triad-Core (EBNF)
- 45. Schede rapide Core e Full
- 46. Debug e problemi comuni
- 47. Glossario
- 48. Stato, limiti noti e fix storici
- 49. FAQ

Manuale di Triad 2.0

Manuale di Triad 2.0

> *Il manuale completo del linguaggio: lessico, valori, controllo di flusso, funzioni, dispatch, logica, codice come dato, errori, contratti, budget, tracce, effetti, esercizi e riferimento.*

Triad è un linguaggio di programmazione autonomo: sintassi a blocchi e tipi dichiarati, multiple dispatch, programmazione logica integrata e codice come dato, in una sola sintassi. La sua definizione è una specifica normativa indipendente da qualsiasi implementazione (`SPEC.md`, standard `0.2.0`), accompagnata da una specifica eseguibile (`SPEC-CORE.md`) e da una suite di conformità con golden verificabili (`conformance/`).

Questo manuale insegna il linguaggio per come è definito dallo standard. Non è un riassunto della specifica: è un percorso di studio, con spiegazioni, esempi ed esercizi. Puoi leggerlo dall'inizio alla fine senza altro materiale: ogni concetto che serve per capire un capitolo è spiegato prima o in quel capitolo, e i rimandi esterni sono approfondimenti facoltativi. Non è una raccolta di pagine da consultare a caso: è un libro, e l'ordine dei capitoli costruisce il modello mentale del linguaggio un pezzo alla volta.

Come leggere questo manuale

A chi è indirizzato

Sai programmare, in qualunque linguaggio: sai cosa sono una variabile, una funzione, un ciclo, una ricorsione. Questo manuale non spiega la programmazione in generale, spiega **Triad**.

Il manuale non presume che tu abbia letto nient'altro: ogni concetto è spiegato dove serve. I documenti normativi del progetto — `SPEC.md`, `SPEC-CORE.md` — e la suite di conformità in `conformance/` sono i testi e le prove a cui puoi risalire per approfondire o per controllare un caso limite, ma non sono prerequisiti: sono il riferimento di chi vuole verificare, non il manuale di chi vuole imparare. Per questo il testo distingue sempre ciò che è regola del linguaggio da ciò che è comportamento di una particolare implementazione.

Due profili, una sola lingua

Lo standard definisce due profili di conformità, ed è utile saperli distinguere prima di cominciare:

profilo	che cosa contiene	dove è definito
Triad-Core	valori, funzioni, chiusure, controllo di flusso, liste, stringhe, I/O	`SPEC-CORE.md`
Triad-Full	tutto: tipi dichiarati, dispatch, logica, macro, contratti, budget, tracce, effetti	`SPEC.md`

Triad-Core è un **sottoinsieme** di Triad-Full: ogni programma Core è anche un programma Full valido e deve comportarsi allo stesso modo. Il manuale presenta i due profili **sullo stesso piano**, per costruzione logica: ogni capitolo introduce un costrutto e dice in quale profilo esiste. Non perché Full sia "più avanti", ma perché sono due facce della stessa specifica.

Il profilo Core non è una versione semplificata per comodità didattica: è il sottoinsieme scelto perché un interprete di bootstrap, scritto in poche migliaia di righe, possa eseguirlo, e perché lo stesso linguaggio possa descrivere se stesso. Full aggiunge i costrutti che richiedono un motore più ricco — dispatch multiplo, unificazione, espansione di macro — ma non cambia il significato di quelli che Core ha già. Imparare Core, quindi, non è tempo speso per un linguaggio diverso.

Dove la differenza conta, trovi due marcatori:

- **[Core]** — il costrutto esiste nel sottoinsieme di bootstrap;
- **[Full]** — il costrutto esiste solo nello standard completo.

Un marcatore di profilo descrive ciò che lo standard definisce, non ciò che ogni programma installato sa già eseguire. Per questo il manuale distingue tre livelli: la regola normativa (che cosa deve significare il costrutto), la disponibilità nel profilo Core o Full e lo stato delle implementazioni presenti nel repository. Un esempio Full può essere corretto secondo lo standard e, allo stesso tempo, non essere ancora eseguibile dal runtime corrente. Quando un esempio riporta **Output (golden Full)**, il risultato è un valore atteso conservato dalla suite, non una verifica appena eseguita da un interprete Full completo.

Per studiare un costrutto senza confondere questi livelli, chiediti nell'ordine: che forma accetta la grammatica? quali valori legge e produce? quali effetti o errori può generare? in quale profilo è definito? quale implementazione e quale caso di conformance lo verificano? Questa sequenza permette di passare dal codice alla semantica e poi alle prove, invece di trattare l'output di un singolo runtime come definizione del linguaggio.

Convenzioni di notazione

notazione	significa
`monospace`	codice Triad o un valore letterale
Output:	l'output esatto prodotto dal programma
[Core] / [Full]	il profilo in cui il costrutto esiste
`SPEC §n`	la sezione normativa corrispondente

I blocchi di codice sono marcati con il loro livello di eseguibilità:

fence	significa
```` ```triad ````	programma **eseguibile oggi** con l'interprete di riferimento
```` ```triad-full ````	programma **specifica dello standard**: definito e normativo, ma non ancora eseguibile dal runtime corrente

Come eseguire gli esempi

Gli esempi marcati `triad` si eseguono con l'interprete di riferimento JavaScript:

```
node seed/dev/triad_core.cjs programma.triad
```

Il programma riceve gli argomenti in `ARGS` (lista di stringhe, senza il nome del programma) e lo shebang `#!` in prima riga viene ignorato dal parser.

```
#!/usr/bin/env triad
# saluto.triad
println("ciao, " * "Triad")
println(6 * 7)
```

```
node seed/dev/triad_core.cjs saluto.triad uno due
```

Output:

```
ciao, Triad
42
```

L'esempio mostra tre cose che valgono in tutto il libro. Lo shebang è solo una comodità dei sistemi operativi: il parser lo salta, quindi il programma comincia davvero dalla seconda riga. `println` non inserisce separatori tra gli argomenti, quindi la virgola e lo spazio che vedi nell'output sono caratteri scritti nel programma, non una formattazione automatica. Infine `6 * 7` viene calcolato prima della stampa: gli argomenti di una chiamata sono

espressioni, non testo. Gli argomenti passati da riga di comando (`uno`, `due`) non diventano variabili con quei nomi: finiscono tutti nella lista `ARGS`, come spiegato nel capitolo 20.

Parte I — Il linguaggio

La prima parte costruisce il linguaggio strato per strato: prima i mattoni (lessico e valori), poi le espressioni e il controllo di flusso, quindi le funzioni e i modelli di calcolo che Triad aggiunge — dispatch, chiusure, logica, codice come dato — e infine gli strumenti con cui un programma comunica con l'esterno (errori, contratti, budget, tracce, effetti). Ogni capitolo si appoggia ai precedenti; gli esempi sono brevi apposta, perché il punto non è ammirare un programma ma capire una regola.

1. Lessico

1.1 Identificatori

Un identificatore comincia con una lettera o `_`, seguito da lettere, cifre o `_`. Un `!` finale è ammesso se non è seguito da `=`. **[Core]**

```
x = 1
somma_2 = 2
_ = 3
nome_valido = 4
```

`push!`, `sort!` e simili sono nomi legittimi: la convenzione del `!` finale è per le funzioni che mutano il loro argomento (cap. 9).

Il `!` non cambia il significato del nome e non è un operatore: è un segnale per chi legge. Il parser lo accetta solo come ultimo carattere, e solo se il carattere seguente non è `=`; altrimenti comincerebbe un confronto. Questa è la prima delle convenzioni che Triad eredita da Julia: il nome dichiara un contratto, ma è il programmatore a doverlo rispettare.

In Triad-Full gli identificatori ammettono anche lettere non ASCII. **[Full]**

```
età = 42
```

1.2 Numeri

Gli interi si scrivono con cifre decimali; `_` funziona come separatore. Il segno è un operatore unario, non parte del letterale. **[Core]**

La distinzione è più importante di quanto sembri: `-5` è l'applicazione dell'operatore `-` al letterale `5`, non un numero negativo scritto in un modo speciale. Per questo non esiste una "sintassi negativa" da ricordare: le parentesi e i livelli di precedenza del capitolo 4 spiegano già che cosa succede in un'espressione come `-x * y`.

```
println(42)
println(1_000_000)
println(-5)
```

Output:

```
42
1000000
-5
```

`Float` esiste solo in Triad-Full. **[Full]**

```
println(3.14)
println(1e-9)
```

1.3 Stringhe

Una stringa è una sequenza di byte UTF-8 **immutabile**, tra apici doppi. Gli escape sono `\n`, `\t`, `\"`, `\\`. **[Core]**

```
println("ciao\n")
println("tab:\t|")
println("virgolette: \"")
println("backslash: \\")
```

Output:

```
ciao


tab:    |
virgolette: "
backslash: \
```

In Core **non esiste l'interpolazione**: `"x = $x"` stampa letteralmente `x = $x`. In Full esiste, e si scrive `$x` per un'espressione semplice e `$(espr)` per una complessa. **[Full]**

```
x = 42
println("x = $x")
println("x + 1 = $(x + 1)")
```

1.4 Simboli

Un simbolo è `:` seguito da un nome o da un operatore: `:alice`, `:+`. Due simboli sono uguali se hanno lo stesso nome. In Triad-Full il simbolo è anche l'atomo della logica (cap. 16). **[Core]**

Un simbolo serve a dare un nome stabile a una cosa — una categoria, una chiave, un atomo — senza la flessibilità e il costo di una stringa. La differenza rispetto a una stringa non è solo di comodità: `:alice` è un'identità, `"alice"` è una sequenza di byte; due simboli con lo stesso nome sono lo stesso valore ovunque compaiano, mentre due stringhe vanno confrontate carattere per carattere. È questa proprietà che rende i simboli adatti alle chiavi dei record (cap. 2), ai tag delle liste e agli atomi dei fatti (cap. 16).

```
println(:alice)
println(:+)
println(:ok == Symbol("ok"))
```

Output:

```
alice
+
true
```

1.5 Commenti

`#` commenta fino a fine riga. `#= … =#` commenta su più righe; lo standard descrive questa forma ma **non** stabilisce che i commenti a blocco si annidino, e l'interprete di riferimento chiude al primo `=#`: non contare sull'annidamento. **[Core]**

```
#= commento
   su più righe =#
x = 1_000   # commento di riga
println(x)
```

Output:

```
1000
```

1.6 Separatori e fine-riga

Un'istruzione termina con un a capo o con `;`. Gli a capo **chiudono** un'istruzione, tranne dentro `( )`, `[ ]` e `:( )`, e tranne subito dopo un operatore binario o una virgola di `rule`.

```
println(1); println(2)
```

Una conseguenza pratica: **in Triad-Core ogni a capo chiude l'istruzione**. La grammatica di Core non ha una regola di continuazione di riga: un'espressione spezzata su più righe non è un programma valido. Scrivi **una istruzione per riga** e usa `;` per separare due istruzioni.

```
totale = 1 + 2 + 3
println(totale);
println("seconda istruzione")
```

Output:

```
6
seconda istruzione
```

La regola ha una conseguenza sulla formattazione: quando vuoi spezzare un'espressione lunga, le parentesi non bastano in Core. L'interprete di riferimento chiude l'istruzione al primo a capo anche dentro una chiamata, quindi `println(1 +` e `2)` su righe diverse non sono un programma valido oggi. Per il codice destinato al runtime attuale vale una regola sola: una

istruzione per riga, e `;` solo quando due istruzioni devono stare sulla stessa riga.

> ***In Triad-Full*** *la regola è più permissiva:* `SPEC §1` *stabilisce che un a capo* ***non*** *chiude l'istruzione dentro* `( )`*,* `[ ]` *e* `:( )`*, né subito dopo un operatore binario. L'interprete di riferimento attuale non implementa ancora questa eccezione, quindi anche in Full la forma sicura resta una istruzione per riga.*

1.7 La regola dei due punti e del punto interrogativo

`:` ha tre significati e `?` ne ha due. La specifica li chiama esplicitamente «il problema classico di `:`» — è il primo punto in cui Triad si discosta dalla sintassi che conosci.

scrittura	significato
`1:n`, `xs[2:end]`	`:` attaccato a un valore e senza spazi → **range**
`f(:a)`, `x = :a`	`:` non preceduto da un valore e seguito da un nome → **simbolo**
`:( … )`	→ **codice quotato**
`c ? a : b`	`:` circondato da spazi → **separatore del ternario**

scrittura	significato
`?x`	`?` attaccato → **variabile logica**
`?`	`?` con spazio dopo → **ternario**

Da qui la regola pratica: quando `:` è ambiguo, **metti o togli gli spazi**. `xs[2:end]` è un range; `c ? a : b` è un ternario perché i due spazi lo circondano.

Le due tabelle sembrano un elenco di casi speciali, ma discendono da un'unica idea: un carattere aderente al token precedente appartiene a quel token, uno separato appartiene alla sintassi. Per questo in Triad la spaziatura

non è questione di stile: è informazione che il parser legge. Se scrivi `xs[2:end]` oppure `a ? b:c`, cambiando gli spazi non cambi l'aspetto del programma: cambi che cosa il parser riconosce, e il più delle volte ottieni un errore di sintassi. Il consiglio è di scrivere sempre gli spazi che rendono esplicito il significato.

1.8 Parole chiave

Lo standard definisce **29 parole chiave**:

```
function struct macro end if elseif else for in while let begin return
break continue fact rule solve quote try catch with requires ensures
true false nothing isa is
```

E **4 parole contestuali**, valide solo dopo `solve`:

```
limit depth steps trace
```

`true`, `false` e `nothing` sono valori, non parole strutturali.

Le parole chiave sono riservate perché il parser deve riconoscerle prima di sapere che cosa stanno facendo: se `if` potesse essere anche un nome di variabile, la lettura di una riga dipenderebbe dal contesto in modo imprevedibile. `true`, `false` e `nothing` fanno eccezione solo in apparenza: sono valori che il linguaggio predefinisce, non forme sintattiche. Le quattro parole contestuali mostrano lo stesso principio al contrario: `limit`, `depth`, `steps` e `trace` sono nomi normalissimi finché non seguono `solve`, e solo lì hanno un significato speciale. Questa distinzione riduce il numero di parole riservate e ti permette di usare nomi naturali nel resto del programma.

1.9 Identificatori avanzati e convenzioni

In Triad-Core gli identificatori sono limitati all'ASCII: lettere, cifre e `_`. In Triad-Full si ammettono anche lettere non ASCII (accentate, greche, cirilliche): `età`, `π`, `Δx` sono nomi validi. **[Full]**

```
età = 42
π = 3.14159
Δx = 0.001
println(età + π)
```

La convenzione del `!` finale (`push!`, `sort!`) indica funzioni che **mutano** il loro argomento. È l'unico carattere speciale ammesso dentro un identificatore: la grammatica accetta lettere, cifre, `_` e il `!` finale, e nulla di più. In particolare il `?` **non** fa parte del nome, perché in Triad introduce una variabile logica o un ternario: `valido?` non è un identificatore valido. Per un predicato usa un nome normale e lascia che sia il tipo `Bool` del risultato a dire che cos'è — `vuoto`, `is_valido`. **[Core]**

```
function push!(xs, v)
    # muta xs e lo restituisce
    xs
end

function vuoto(xs)
    length(xs) == 0
end
```

Il corpo di `push!` qui è volutamente vuoto: l'esempio parla dei nomi, non dell'implementazione. La mutazione vera delle liste, con `push!` e con l'assegnamento a un elemento, è nel capitolo 9.

Gli identificatori che coincidono con una parola chiave non sono ammessi: `function`, `end`, `if` non possono essere usati come nomi di variabile o funzione. **[Core]**

1.10 Escape e stringhe avanzate

Lo standard fissa quattro escape nelle stringhe: `\n`, `\t`, `\"`, `\\`; in Triad-Full si aggiunge `\$`, perché il `$` introduce l'interpolazione e va neutralizzato quando deve restare un carattere normale. Le forme estese `\r`, `\0`, `\u{…}` e `\x{…}` che compaiono nel frammento seguente non sono elencate in `SPEC §1`: trattale come estensioni di una singola implementazione, non come garanzia del linguaggio. Un programma portabile usa le escape documentate. **[Full]**

```
println("a\r\nb")          # ritorno a capo CRLF
println("unicode: \u{1F600}")  # emoji
println("hex: \x{41}")     # 'A'
```

Una stringa è una sequenza di byte UTF-8 e le stringhe sono **immutabili**: ogni operazione di concatenazione crea una nuova stringa, come mostra l'esempio qui sotto. La lunghezza si misura in byte e non in caratteri, un dettaglio che conta appena esci dall'alfabeto latino (cap. 10). **[Core]**

```
a = "ciao"
b = a * " mondo"
println(a)        # "ciao" — inalterato
println(b)        # "ciao mondo"
```

Output:

```
ciao
ciao mondo
```

1.11 Esercizi sul lessico

1. Scrivi un programma che stampi il tuo nome e la tua età su due righe, usando `\n` e concatenazione di stringhe.
2. Definisci un identificatore con `_` iniziale e uno con `!` finale; stampali.
3. Prova a scrivere `x = 1_000_000` e verifica che l'output sia `1000000`.
4. (Full) Scrivi una stringa con interpolazione `$x` e `$(x + 1)`.
5. (Full) Definisci il simbolo `:ok` e confrontalo con `Symbol("ok")`.

---

2. Valori e tipi

2.1 I tipi

Un valore Triad appartiene sempre a un tipo, e il tipo si può sempre osservare: la tabella elenca i tipi dello standard, con il nome che `typeof` restituisce e il profilo in cui esistono.

tipo	valori	`typeof(v)`
`Int`	interi con segno a 64 bit, aritmetica con wrap	`Int`
`Float`	numeri in virgola mobile **[Full]**	`Float`
`Bool`	`true`, `false`	`Bool`
`String`	sequenza di byte UTF-8 immutabile	`String`
`Symbol`	`:nome`	`Symbol`
`List`	lista eterogenea mutabile, indici da 1	`List`
`Function`	funzione nominata, lambda, builtin	`Function`
`Nothing`	`nothing`	`Nothing`
`Expr`	codice quotato **[Full]**	`Expr`
`Term`, `LVar`	termini e variabili logiche **[Full]**	—
`Solution`	`{who = :dave}` **[Full]**	—
`Err`	errore come dato **[Full]**	—
`Type`	i tipi stessi sono valori **[Full]**	—

tipo	valori	`typeof(v)`
struct utente	`Point(3.0, 4.0)` **[Full]**	`Point`

Il set dei tipi Core è **chiuso e piccolo**: `Int`, `Bool`, `String`, `Symbol`, `List`, `Function`, `Nothing`. In Core non esistono `Float`, `Term`, `Solution`, `Err` come valori del linguaggio: un errore è una lista taggata `[:Err, kind, msg]` (cap. 11) e una struct è `[:Point, x, y]`. Il set chiuso non è una limitazione arbitraria: un interprete di bootstrap deve poter rappresentare ogni valore con poche strutture, e i concetti Full che mancano — errore, struct, soluzione — si ricostruiscono per convenzione sopra liste e simboli.

`typeof` restituisce il **nome del tipo come simbolo**. Il dettaglio del profilo conta: in Core il risultato è un `Symbol`, quindi `typeof(1) == :Int` è il confronto naturale; in Triad-Full i tipi sono valori di tipo `Type`, per cui `typeof(3) == Int` confronta direttamente con il tipo. La stampa è la stessa (`Int`), ma l'operazione che scrivi è diversa — ed è per questo che la tabella qui sopra ha una colonna `typeof(v)` con il nome e non con il tipo.

È utile separare tre idee che nella sintassi possono sembrare la stessa cosa. Il **valore** è il dato concreto, per esempio `5` o `"ciao"`; il **tipo dinamico** descrive la categoria del valore che il programma sta manipolando; un **vincolo dichiarato** restringe invece quali valori può assumere un nome o un parametro. In Core il tipo si osserva con `typeof`, ma non si dichiara una variabile come `x::Int`. In Full i tipi entrano anche nelle firme, nelle struct e nel dispatch. Una conversione come `Float(3)` costruisce un valore compatibile; un'asserzione come `x::Int` controlla un valore già ottenuto.

Sono operazioni diverse: convertire può cambiare il valore, asserire non è una richiesta di conversione.

Un'altra distinzione importante è tra tipo **nominale** e forma strutturale. In Full una `struct Point` dichiara un'identità di tipo, quindi un valore `Point` non diventa un altro tipo solo perché possiede campi simili. In Core, dove non ci sono struct, la stessa informazione è convenzionale: una lista come `[:Point, 3, 4]` porta il tag `:Point` e i campi in posizioni concordate dal programma. Il tag rende possibile riconoscere il record, ma non aggiunge da solo controlli di campo, immutabilità o dispatch.

```
xs = [1, :a, "s"]
println(typeof(1), " ", typeof("s"), " ", typeof(:a), " ", typeof(xs))
println(typeof(nothing), " ", typeof(true), " ", typeof(x -> x))
```

Output:

```
Int String Symbol List
Nothing Bool Function
```

2.2 Gerarchia dei tipi

`Int <: Number`, `Float <: Number`, e ogni tipo è sottotipo di `Any`. `Any` è il supertipo usato nelle firme quando un parametro accetta qualunque cosa. **[Full]**

La gerarchia esiste per il dispatch (cap. 7): un metodo dichiarato su `Number` accetta anche gli `Int`, perché `Int` è un sottotipo di `Number`. In Core la gerarchia non si può osservare, perché non c'è dispatch multiplo, ma resta il motivo per cui i tipi hanno un nome e non sono solo etichette di comodo: la relazione fra i nomi è ciò che permette di scrivere un metodo una volta e di applicarlo a una famiglia di valori.

```
function accetta_qualsiasi(x::Any)
    "qualunque cosa: " * string(x)
end
```

2.3 Struct

Una `struct` è un record immutabile con campi tipizzati e costruttore posizionale. Il costruttore controlla i campi; un `Int` passato a un campo `Float` viene convertito, un valore non convertibile produce `Err(:type)`. **[Full]**

```
struct Point
    x::Float
    y::Float
end

p = Point(3, 4)
println(p.x, " ", p.y)
println(typeof(p) == Point)
println(p isa Point)
```

Output (golden Full):

```
3.0 4.0
true
true
```

In Core lo stesso concetto si rappresenta con una **lista taggata**: `[:Point, x, y]`, con il simbolo del tipo in prima posizione. È così che gli interpreti di `selfhost/` rappresentano le struct Full. La differenza tra il record Full e quello Core non è solo di scrittura: la struct ha un'identità di tipo e campi immutabili, il record Core è una lista, quindi può essere mutata e nessuno impedisce di costruirne una con un tag sbagliato. Il tag rende il dato

riconoscibile, non lo rende sicuro: i controlli che una struct fa da sola, in Core li deve fare il programma.

```
punto(x, y) = [:Point, x, y]
px(p) = p[2]
py(p) = p[3]
println(punto(1, 2), " ", px(punto(1, 2)))
```

Output:

```
[:Point, 1, 2] 1
```

2.4 Conversioni

`Int(x)`, `Float(x)`, `String(x)`, `Symbol(x)`. Il nome del tipo usato come funzione converte; il nome di una struct usato come funzione **costruisce**. **[Full]**

La conversione è un'operazione esplicita: Triad non trasforma i tipi da solo, tranne nei punti in cui la specifica lo prescrive — un `Int` passato a un campo `Float` di una struct, o il valore di ritorno controllato con `::R`. Altrove la conversione la chiedi tu, e questo rende leggibile il punto in cui il tipo cambia. `string(x)`, che vedi nella tabella del capitolo 10, è un caso a parte: è definita per ogni valore e serve a costruire testo, per questo nei programmi Core è il modo normale di convertire un numero in stringa.

```
println(Int("42") + 1)
println(Float(3))
println(Symbol("ciao"))
println(String(42))
```

`Int(s)` e `Symbol(s)` esistono anche in Core, ma solo per stringhe e simboli. **[Core]**

```
println(Int("41") + 1)
println(Symbol("bob"))
```

Output:

```
42
bob
```

2.5 Asserzione di tipo

`e::T` restituisce `e` se è di tipo `T`, altrimenti produce `Err(:type)`. **[Full]**

```
 x = 3
println(x::Int)
println(iserr(x::String))
```

2.6 Il vincolo del dichiarato

`x::T = e` — e un parametro `x::T` — **fissa** il tipo di quel nome nell'ambiente in cui nasce. Gli assegnamenti successivi sono controllati e convertiti contro `T`. Una nuova dichiarazione `x::U = …` sostituisce il vincolo; un nome non dichiarato con tipo resta libero; una rilegatura in un ambiente interno (`let x = 2.5`) non eredita il vincolo esterno. **[Full]**

```
x::Int = 5
x = 6             # bene
x = 5.5           # Err(:type)

y::Float = 1.0
y = 2             # 2.0: conversione Int -> Float

s = "a"
s = 3             # bene: s non è dichiarato con un tipo
```

Il vincolo vale per l'ambiente in cui il nome nasce, non per sempre: è un'annotazione locale, non una dichiarazione globale. Questo permette di introdurre i tipi in un programma Core già scritto senza riscriverne la struttura, e spiega perché `let x = 2.5` dentro una funzione non è un errore anche quando fuori `x` era vincolato a `Int`. Il controllo avviene a ogni assegnamento, quindi il costo è un controllo in più per ogni scrittura; in cambio, un valore del tipo sbagliato non entra mai nel nome, e l'errore arriva dove il tipo è stato promesso, non molto più tardi.

2.7 Le condizioni accettano solo `Bool`

Non esiste il concetto di «verità implicita»: un valore non diventa vero o falso per convenzione. `if 1` è un errore, non `true`. Questo vale per `if`, `while`, `&&`, `||`, `!`, il ternario e i filtri delle comprehension. **[Core]**

```
println(1 < 2 && 2 < 3)
x = 3
println(x > 2 ? "grande" : "piccolo")
```

Output:

```
true
grande
```

La scelta è deliberata. In un linguaggio con tipi dinamici e con gli errori come valori, l'unica cosa che una condizione deve fare è produrre `true` o `false`; ammettere valori "veri" per convenzione (`0`, `""`, `[]`) renderebbe ambiguo il significato di un programma e sposterebbe l'errore lontano dalla sua causa. Con la regola stretta, `if xs` è un errore immediato e dice che lì serve un confronto — per esempio `if length(xs) > 0`. Lo stesso vale per `&&` e `||`: entrambi gli operandi che vengono valutati devono essere `Bool`, anche se il cortocircuito ne risparmia uno.

2.8 Come stampa un valore

`print` e `println` concatenano gli argomenti **senza separatori**; `println` aggiunge un a capo.

valore	come viene stampato
`Int`	decimale
`Bool`	`true` / `false`
`String`	così com'è
`Nothing`	`nothing`
`Symbol`	**senza** `:` se passato direttamente, **con** `:` dentro una lista
`List`	`[a, b]` ; le stringhe dentro non hanno virgolette
`Function`	`#lambda(x)`

```
xs = [1, :a, "s", true, nothing, [2, 3]]
println(xs)
println(:alice)
print("senza ", "a capo")
println()
```

Output:

```
[1, :a, s, true, nothing, [2, 3]]
alice
```

```
senza a capo
```

La differenza tra un simbolo stampato da solo e dentro una lista non è un capriccio: fuori da una lista il simbolo è l'unico contenuto dell'output, dentro una lista serve un modo per distinguerlo da una stringa con lo stesso testo. Per questo `println(:a)` stampa `a` mentre le liste usano `:a`, e per questo la tabella va letta come «come appare un valore quando è stampato», non come «come è fatto il valore». Le stringhe dentro una lista non hanno virgolette per la stessa ragione: la stampa è pensata per essere leggibile, non per essere riletta dal parser.

L'uguaglianza confronta numeri, stringhe e simboli **per valore**, le liste **elemento per elemento** (anche annidate), le funzioni **per identità**. **[Core]**

```
println([1, [2, [3]]] == [1, [2, [3]]])
println("ab" == "a" * "b", " ", :a == :a, " ", 1 == 2)
```

Output:

```
true
true true false
```

---

3. Variabili e assegnamento

`nome = espr` lega o aggiorna un nome. L'assegnamento **è un'espressione** e vale il valore assegnato. `+=`, `-=`, `*=` aggiornano una variabile esistente e valutano il lato sinistro una volta sola. Usare un nome mai assegnato è un errore di esecuzione. **[Core]**

Il segno `=` non si legge «scrivi in memoria»: lega un nome a un valore. Quando il nome non esiste lo crea, quando esiste ne sostituisce il legame. Triad non distingue i due casi con parole chiave diverse e non ha una parola per dichiarare una variabile: `let` introduce uno scope, non dichiara un tipo. Questa semplicità ha un prezzo che conviene conoscere subito: un nome scritto male non viene segnalato mentre scrivi, ma solo quando quella riga viene eseguita.

```
x = 42
y = x + 8
x = x + 1
z = (x + y) * 2
println(x, " ", y, " ", z)
n = 10
n += 5
n -= 3
n *= 2
println(n)
```

Output:

```
 43 50 186
24
```

Non esistono dichiarazioni: le variabili sono dinamiche. Le regole di **visibilità** si vedono nel capitolo 8.

Perché mai un assegnamento dovrebbe avere un valore? Perché in Triad tutto ciò che può comparire in un programma è un'espressione, e l'assegnamento non fa eccezione: così `x = 42` può stare dentro un `if`, un `let` o un argomento di funzione senza una sintassi speciale. Il valore è quello assegnato, non un `Bool` di successo come in altri linguaggi; per questo non scriverai `if x = 5` aspettandoti un controllo, perché `5` non è un `Bool` e la condizione fallisce con un errore di tipo.

Anche `+=`, `-=` e `*=` sono espressioni e valgono il valore aggiornato. Non sono forme dichiarative e non introducono un nome nuovo: aggiornano un legame che deve già esistere. In cambio valutano il lato sinistro una volta sola, un dettaglio che con un semplice nome non si nota ma che conta quando il bersaglio è un elemento di lista (`xs[i] += 1`, cap. 9): così l'indice non viene calcolato due volte.

Leggi l'esempio distinguendo il **nome** dal valore: dopo `x = 42`, `x` è un legame dell'ambiente corrente; `y = x + 8` legge quel legame e memorizza il risultato `50` in un legame diverso; `x = x + 1` calcola prima `43` e poi rimpiazza il valore associato a `x`. Di conseguenza `y` resta `50`: non è un alias automatico dell'espressione che lo ha prodotto. L'assegnamento ritorna il nuovo valore, perciò può comparire dentro un'espressione, ma questo non

significa che ogni assegnamento sia globale: funzione, lambda e `let` definiscono regole di scope distinte.

Infine, riassegnare un nome non equivale a mutare un oggetto. Con `x = 43` cambia il legame di `x`; con `xs[1] = 43` cambia un elemento della lista a cui `xs` fa riferimento. Se un secondo nome indica la stessa lista, vedrà la mutazione; una riassegnazione locale di `xs`, invece, non cambia automaticamente il legame dell'altro nome. Questa differenza diventa essenziale quando una funzione condivide una lista con il chiamante.

---

4. Operatori e precedenza

Il parser è un **Pratt** con dodici livelli. «Pratt» significa che ogni operatore ha un numero di precedenza e il parser, leggendo da sinistra, sa quando fermarsi e restituire il controllo al livello più basso: è un modo compatto di descrivere l'albero senza scrivere una regola per ogni combinazione di operatori. Dalla precedenza più bassa alla più alta:

livello	operatori	associatività
1	`=` `+=` `-=` `*=`	destra
2	`x -> e`, `(a, b) -> e`, `c ? a : b`	destra
3	`||`	sinistra, cortocircuito
4	`&&`	sinistra, cortocircuito
5	`==` `!=` `<` `<=` `>` `>=` `in` `isa` `is`	sinistra
6	`a:b`	range, senza spazi
7	`+` `-`	sinistra
8	`*` `/` `%`	sinistra
9	`-x` `!x` `+x` `$x`	prefissi
10	`^`	**destra**
11	`e::T`	asserzione di tipo

livello	operatori	associatività
12	`f(args)` `xs[i]` `a.campo`	postfissi, senza spazio prima di `(` e `[`

I livelli 6, 8 (la divisione), 9 (`$x`), 11 (`::`) e alcuni operatori di confronto esistono solo in Triad-Full. In Triad-Core i livelli effettivamente usati sono **dieci**, perché mancano il range, la divisione, `$x`, `::` e gli operatori `in`, `isa`, `is`; tolti quelli, la tabella si compatta ma il rapporto fra gli operatori che restano è lo stesso. Il numero dodici, che il glossario riporta, è quindi il conteggio del profilo Full: non è in contraddizione con i dieci livelli di Core, è il conto di un profilo più grande. **[Core]**

La precedenza decide **come il parser raggruppa** gli operatori, non quale calcolo avviene per primo in ogni possibile programma. Per esempio, `2 + 3 * 4` viene letto come `2 + (3 * 4)`: la moltiplicazione appartiene a un livello più forte della somma. Le parentesi non cambiano la regola del linguaggio; rendono esplicita una scelta del programmatore e aiutano chi legge. L'associatività risolve invece la parità di precedenza: `2 ^ 3 ^ 2` è `2 ^ (3 ^ 2)`, mentre le sottrazioni a catena si raggruppano a sinistra. Quando la precedenza è poco familiare, scrivere le parentesi è una forma di documentazione eseguibile.

Il raggruppamento non va confuso con la valutazione. `&&` e `||` sono operatori a cortocircuito: il secondo operando viene valutato solo se il primo non determina già il risultato. Ciò consente di mettere a destra un controllo che dipende da un valore valido a sinistra, per esempio prima verificare che un indice sia entro i limiti e poi leggere la lista. In Core ogni

condizione deve comunque produrre un `Bool`; un intero o una lista non vengono convertiti implicitamente in vero/falso.

4.1 Aritmetica intera

L'aritmetica di Core è interamente intera e queste quattro regole vanno lette come un modello unico, non come curiosità isolate: il tipo `Int` è un blocco di 64 bit, le operazioni lo trattano come tale, e la divisione è lasciata a Full perché richiede un altro tipo di numero.

- `+ - * ^` su `Int` con **wrap a 64 bit** (complemento a due);
- `%` è il **resto con il segno del dividendo**, come `rem` di Julia: `-7 % 3 == -1`;
- `^` con esponente negativo è un errore; `%` per zero è un errore;
- `/` **non esiste in Core**: la divisione è di Triad-Full.

```
println(2 + 3 * 4, " ", (2 + 3) * 4, " ", 2 ^ 3 ^ 2)
println(7 % 3, " ", -7 % 3, " ", 10 % 4)
println(9223372036854775807 + 1)
```

Output:

```
14 20 512
1 -1 2
-9223372036854775808
```

Nota `2 ^ 3 ^ 2 == 512`: l'esponente è associativo a destra, come in matematica.

Il wrap non è un difetto da aggirare: `Int` è un intero a 64 bit e l'aritmetica è quella del complemento a due, quindi superare il massimo riporta al

minimo. Se il tuo calcolo può crescere oltre i 64 bit, in Full puoi passare a `Float` (con i suoi limiti di precisione); in Core devi progettare il calcolo per restare nei limiti, per esempio controllando i valori prima di moltiplicare.

`%` segue il segno del dividendo perché è il resto della divisione intera troncata verso zero, non il modulo matematico, che sarebbe sempre non negativo: `-7 % 3` è `-1` e non `2`. La distinzione conta negli algoritmi che usano il resto per alternare indici o per normalizzare valori negativi.

Infine `/`: in Core non esiste perché la divisione esatta di due interi non ha un tipo in cui vivere senza i numeri in virgola mobile. In Full `/` produce un `Float` e `div` fa la divisione intera; se lavori in Core, la divisione intera la scrivi a mano, come nell'esempio del capitolo 11.4.

4.2 Logica e confronti

`&&` e `||` cortocircuitano. I confronti **non** si concatenano: `1 < 2 < 3` è un errore (si confronta `Bool < Int`); scrivi `1 < 2 && 2 < 3`. **[Core]**

```
println(!false, " ", true && false, " ", false || true)
println(1 < 2 && 2 < 3)
```

Output:

```
true false true
true
```

---

5. Controllo di flusso

In Triad **tutto è un'espressione**: `if`, `while`, `let`, `begin` hanno un valore, quello dell'ultima istruzione eseguita (`nothing` se non ce n'è).

Le eccezioni sono `return`, `break` e `continue`: non producono un valore, interrompono il flusso. L'assegnamento invece è un'espressione (cap. 3), e questa uniformità è ciò che rende possibile scrivere `v = if condizione … end` senza una sintassi speciale: un blocco è un'espressione come lo è una somma.

Questa regola unifica due usi che in altri linguaggi spesso hanno sintassi separate: scegliere un ramo e calcolare un risultato. In `v = if condizione ... end`, l'espressione completa produce il valore dell'ultima espressione del ramo scelto; il codice dei rami non scelti non viene eseguito. Le istruzioni che producono output o mutano una lista possono stare nello stesso blocco di un'espressione che restituisce un valore: il risultato del blocco è l'ultima espressione, non una combinazione automatica di tutti i risultati intermedi.

Le condizioni sono intenzionalmente restrittive: `if`, `while`, ternario, `&&` e `||` chiedono un valore `Bool`. Non si usa una regola di «verità implicita» che interpreti `0`, stringhe vuote o liste vuote come falsi. Questo rende visibili gli errori di tipo vicino alla decisione che li ha provocati, invece di trasformarli in rami inattesi.

5.1 `if` / `elseif` / `else` [Core]

```
function classifica(x)
    if x < 0
        "negativo"
    elseif x == 0
        "zero"
    else
        "positivo"
    end
end
println(classifica(-3), " ", classifica(0), " ", classifica(7))
v = if 2 > 1 :si else :no end
println(v)
```

Output:

```
negativo zero positivo
si
```

Nell'esempio `classifica` restituisce l'ultima espressione del ramo scelto, quindi non serve nessun `return`. `v = if 2 > 1 :si else :no end` mostra la stessa cosa senza funzione: il blocco è un valore e `v` riceve `:si`. Il ramo che non vince non viene eseguito affatto, non viene valutato e poi scartato: se contenesse una stampa o una mutazione, non la vedresti.

Il ternario `c ? a : b` è la forma breve di `if` e ha la stessa precedenza del `->`. **[Core]**

5.2 `while`, `break`, `continue` [Core]

`break` e `continue` agiscono sul ciclo **più interno**.

```
i = 0
somma_dispari = 0
while i < 10
    i += 1
    if i % 2 == 0
        continue
    end
    somma_dispari += i
end
println(somma_dispari)
while true
    break
end
println(:fine)
```

Output:

```
25
fine
```

Il primo ciclo somma solo i dispari: `continue` salta il resto del corpo, quindi l'aggiunta non avviene per i pari, e il contatore `i` è già stato incrementato prima del salto. Se l'incremento fosse stato in fondo al corpo, `continue` lo avrebbe saltato e il ciclo sarebbe stato infinito: è la trappola classica di

`continue` nei cicli con contatore manuale, e il motivo per cui conviene incrementare subito, prima di ogni possibile uscita anticipata.

`break` e `continue` agiscono sul ciclo **più interno**; non esiste un'etichetta per uscire da due cicli in un colpo, quindi in quel caso si usa una variabile di stato.

In Core **non esiste** `for`: si itera con un contatore e `while`. In Full `for` esiste (cap. 5.6).

5.3 `let` e `begin` [Core]

`let` introduce un nuovo scope con legami locali. `begin … end` raggruppa istruzioni in una sola espressione, ma non è «senza conseguenze» sullo scope: l'interprete di riferimento aggiorna un nome già legato fuori dal blocco, mentre un nome **nuovo** assegnato dentro `begin` non risulta visibile fuori. La specifica non elenca `begin` fra i costrutti che introducono ambienti e nessun caso di conformità fissa il punto: per i nomi nuovi è prudente trattare il blocco come un confine.

```
x = 1
let x = 10
    let x = 20
        println(x)
    end
    println(x)
end
println(x)
r = begin
    a = 2
    a * 21
end
println(r)
```

Output:

```
20
10
```

```
1
42
```

L'esempio mostra tre legami distinti per `x`: quello esterno, quello introdotto dal primo `let` e quello introdotto dal secondo. Ogni `let` crea un ambiente nuovo, e un nome viene cercato prima nell'ambiente più interno; per questo le tre stampe danno `20`, `10`, `1` e nessuna delle tre rilegature tocca le altre. `begin`, invece, è solo un raggruppamento: l'esempio assegna `a = 2` e usa `a * 21` nello stesso blocco, quindi non dipende da che cosa resta visibile dopo. La regola da tenere presente è che il blocco non serve a far uscire nomi nuovi: se un valore deve essere usato fuori, assegnalo a un nome che esiste già prima del `begin` o restituiscilo come valore dell'espressione. In pratica `let` serve quando vuoi isolare un nome o riusare un nome già preso senza alterarne il legame esterno; `begin` serve a raggruppare istruzioni in un'espressione unica.

5.4 `for` e i range [Full]

`for` itera su una lista, un range o un risultato di `solve`. `a:b` produce il range da `a` a `b` inclusi.

```
s = 0
for i in 1:10
    s += i
end
println(s)

xs = [1, 2, 3]
println(xs[end])
println(xs[2:end])
```

Output (golden Full):

```
55
3
[2, 3]
```

`end` è un nome speciale che vale la posizione dell'ultimo elemento e si può usare dentro le parentesi quadre di un indice: `xs[end]` è l'ultimo elemento, `xs[2:end]` è la coda dal secondo elemento in poi. Il range `a:b` è inclusivo su entrambi gli estremi, quindi `1:10` contiene dieci valori; è una forma di Triad-Full, e in Core si ottiene con un ciclo. La destrutturazione nel `for` è trattata nel capitolo 24.

5.5 Le comprehension [Full]

La comprehension `[e for x in xs]` costruisce una lista; con `if` aggiunge un filtro.

```
println([x^2 for x in 1:4])
println([x^2 for x in 1:4 if x > 2])
```

Output (golden Full):

```
[1, 4, 9, 16]
[9, 16]
```

La comprehension è zucchero per un ciclo che accumula in una lista: per ogni `x` di `1:4` valuta `x^2` e aggiunge il risultato, senza un contatore esplicito. Il filtro `if x > 2` agisce sull'elemento, non sull'indice: `9` e `16` restano perché `3` e `4` superano la soglia. In Core `[e for x in xs]` non esiste e lo stesso lavoro lo fanno le funzioni scritte a mano del capitolo 22.1; in Full spesso la comprehension è la forma più leggibile, purché l'espressione resti breve.

5.6 `try` / `catch` [Full]

In Full l'errore è un **valore**, quindi `try` ha due forme: con `catch` il ramo gestito produce il valore dell'espressione; senza `catch`, l'errore **diventa** il valore dell'espressione.

```
e = try div(1, 0) end
println(iserr(e), " ", e.kind)

r = try div(1, 0) catch err; 0 end
println(r)
```

Output (golden Full):

```
true domain
0
```

Le due forme non sono intercambiabili. `try e end` non gestisce l'errore: lo rende il valore dell'espressione, così puoi controllarlo con `iserr` e decidere dopo. `try e catch err; h end` invece recupera: lega `err` all'errore e valuta `h`, il cui valore diventa il risultato. Nel primo caso `e` è un `Err`, nel secondo `r` è `0`. Scegliere fra i due è una decisione di progetto: la prima forma tiene l'errore nel flusso normale, la seconda lo confina in un punto preciso. È il motivo per cui il capitolo 11 insiste sul fatto che «errore come dato» non significa «il programma continua da solo»: senza un `try` che lo osservi, l'errore arriva al livello superiore e il programma termina.

---

6. Funzioni

6.1 Tre forme di definizione [Core]

Una funzione nominata si definisce in due modi — forma breve e forma a blocco — e la terza forma è una funzione anonima legata a un nome. La differenza è solo sintattica: la funzione che ottieni è la stessa. La forma breve si usa quando il corpo è una sola espressione; la forma `function … end` quando il corpo ha più istruzioni o un `return` che esce prima della fine; la lambda assegnata a un nome è utile quando il valore funzione nasce dentro un'espressione.

```
 # forma breve: il corpo è un'espressione
quadrato(x) = x * x

# forma blocco: il valore è quello dell'ultima espressione
function calcola(n)
    m = n + 1
    m * 2
end

# funzione anonima assegnata a un nome
moltiplica = function (a, b)
    a * b
end

println(quadrato(4), " ", calcola(3), " ", moltiplica(6, 7))
```

Output:

```
16 8 42
```

`quadrato` e `calcola` sono funzioni nominate, `moltiplica` è un valore di tipo funzione legato a un nome: dal punto di vista della chiamata non c'è differenza. Scegli la forma che rende più leggibile il corpo, non quella che sembra più moderna; la forma breve con una sola espressione rende evidente che non ci sono effetti nascosti tra le righe.

6.2 `return`, ricorsione, arità

`return e` esce subito dalla funzione; `return` da solo restituisce `nothing`. Una funzione nominata è visibile nel proprio corpo, quindi la ricorsione diretta funziona. Il numero di argomenti deve coincidere con i parametri. **[Core]**

```
function taglia(n)
    if n > 100
        return :grande
    end
    :piccola
end
fattoriale(n) = n < 2 ? 1 : n * fattoriale(n - 1)
fib(n) = n < 2 ? n : fib(n - 1) + fib(n - 2)
mcd(a, b) = b == 0 ? a : mcd(b, a % b)
println(taglia(500), " ", taglia(5))
println(fattoriale(10), " ", fib(15), " ", mcd(48, 18))
```

Output:

```
grande piccola
3628800 610 6
```

Una chiamata può essere letta come una sequenza precisa: si valutano gli argomenti, si crea il contesto della funzione, si associano i valori ai parametri e si esegue il corpo. Il corpo termina in uno di due modi: raggiunge la fine e restituisce l'ultima espressione, oppure esegue `return`, che interrompe immediatamente il corpo e restituisce il valore indicato. `return` senza espressione restituisce `nothing`. Una ricorsione è una nuova

chiamata con un proprio insieme di parametri locali; il caso base impedisce di continuare a creare chiamate senza fine.

L'arità è parte della firma anche in Core: `somma(1, 2)` non è intercambiabile con `somma(1)`. Triad-Full aggiunge i tipi alle firme e consente più metodi con lo stesso nome; questo non cambia la nozione di chiamata, ma aggiunge una fase di selezione del metodo prima di eseguirne il corpo.

Un'ultima nota pratica: la ricorsione profonda consuma spazio di chiamata, quindi per profondità grandi conviene un `while` con uno stato esplicito (cap. 22.6). Il caso base non è una formalità: senza di esso la chiamata ricorsiva non si ferma e il programma non arriva a un risultato.

6.3 Funzioni di ordine superiore

Le funzioni sono valori di prima classe: si passano, si restituiscono, si compongono. In Core `map`, `filter` e `reduce` **non esistono**: si scrivono a mano (cap. 22.1). In Full sono builtin. **[Core]**

```
function applica_tutti(f, xs)
    out = []
    i = 1
    while i <= length(xs)
        push!(out, f(xs[i]))
        i += 1
    end
    out
end
println(applica_tutti(x -> x * x, [1, 2, 3]))
```

Output:

```
[1, 4, 9]
```

In Full, `reduce(+, xs)` e `map(sq, xs)` funzionano perché anche gli **operatori sono funzioni**.

Il punto di `applica_tutti` non è la funzione specifica: è che `f` è un parametro come gli altri, e `x -> x * x` è un valore costruito al momento della chiamata. Questa è la differenza fra scrivere dieci cicli quasi uguali e scrivere un ciclo una volta sola; in Core `map` e `filter` non esistono perché il linguaggio di bootstrap deve restare piccolo, non perché l'idea sia estranea a Triad. Le

funzioni del capitolo 22.1 sono la risposta pratica: scrivile una volta e riusale.

6.4 L'annotazione di effetto `!io`

Una funzione che stampa deve dichiarare `!io` nella firma. Core **accetta l'annotazione e la ignora**: scriverla rende il codice già pronto per Full. **[Core]**

```
function mostra() !io
    println("hello from block")
    20 + 22
end
println(mostra())
```

Output:

```
hello from block
42
```

L'annotazione non è decorativa. In Triad-Full una funzione dichiara le capacità che usa, e il runtime verifica che il chiamante le permetta (cap. 15); in Core il controllo non esiste, ma scrivere `!io` adesso significa non dover riscrivere la firma quando il programma passerà a Full. La parola `io` è un permesso, non un'etichetta descrittiva: dice che la funzione può produrre output, e chi la chiama deve essere autorizzato a farlo.

---

7. Tipi e multiple dispatch

Una funzione generica è un insieme di metodi. Ridefinire la stessa firma sostituisce il metodo precedente. Tra i metodi applicabili (stessa arità, ogni argomento sottotipo del parametro) vince il **più specifico**. **[Full]**

```
area(n::Number) = :number
area(n::Int) = :int

println(area(1))
println(area(1.5))
```

Output (golden Full):

```
int
number
```

L'esempio va letto come una ricerca: `area(1)` ha un `Int`, che è sottotipo sia di `Int` sia di `Number`; entrambi i metodi sono applicabili, ma `Int` è più specifico e vince. `area(1.5)` ha un `Float`, che è sottotipo di `Number` ma non di `Int`, quindi resta un solo metodo applicabile. Se avessi definito solo `area(n::Number)`, entrambe le chiamate avrebbero usato quello: il dispatch non sceglie il metodo «migliore», sceglie il più specifico fra quelli che accettano gli argomenti.

Un metodo A è più specifico di B se ogni suo parametro è sottotipo del corrispondente parametro di B e le firme differiscono.

- Se nessun metodo è applicabile: `Err(:method)`.

- Se più metodi sono massimali: `Err(:ambiguous)`.
- Se il nome è anche un builtin (per esempio `+` sui numeri), si usa il builtin.

Il dispatch si può modellare come una ricerca in due passaggi. Prima si eliminano i metodi con arità errata o con almeno un argomento che non soddisfa il tipo dichiarato; poi si confrontano quelli rimasti secondo la relazione di sottotipo, parametro per parametro. Un metodo è più specifico solo se non è più generale in nessuna posizione e lo è in almeno una. Non basta quindi scegliere il metodo aggiunto più di recente. Se restano due firme incomparabili, la chiamata è ambigua e produce `Err(:ambiguous)` invece di dipendere dall'ordine dei file o dall'ordine di registrazione. Questo comportamento consente di aggiungere metodi specializzati senza riscrivere la funzione generica, mantenendo però espliciti i casi che non hanno una risposta univoca.

Gli operatori si estendono come funzioni, ridefinendoli:

```
 struct Point
     x::Float
     y::Float
end


function +(a::Point, b::Point)
     Point(a.x + b.x, a.y + b.y)
end


p = Point(1, 2) + Point(3, 4)
println(p.x, " ", p.y)
```

Output (golden Full):

```
4.0 6.0
```

7.1 Il tipo di ritorno

`::R` è controllato all'uscita, con la conversione `Int -> Float`. Se il valore non è conforme: `Err(:type)`. **[Full]**

```
function metà(n::Number)::Float
    n / 2
end
println(metà(3))
```

Output (golden Full):

```
1.5
```

Il controllo avviene all'uscita perché solo allora il valore esiste; la conversione `Int -> Float` permette di scrivere `n / 2` senza preoccuparti del tipo letterale del risultato. Se il valore non è convertibile, ottieni un `Err(:type)` e non un'eccezione: anche una promessa di firma violata entra nel flusso degli errori come dato.

7.2 Funzioni locali

Una `function` definita dentro un'altra funzione è una **chiusura locale**, senza dispatch. **[Full]**

```
function esterno(x)
    function interno(y)
        y * 2
    end
    interno(x) + 1
end
println(esterno(5))
```

---

8. Lambda, chiusure e scope

8.1 Lambda [Core]

Una lambda è una funzione senza nome: `x -> x * 2` è un valore, non una definizione. La forma `(a, b) -> …` dichiara più parametri, e il corpo è un'espressione oppure un blocco. Le lambda si passano come argomenti e si restituiscono come risultati, ed è così che una funzione di ordine superiore (cap. 6.3) descrive un'operazione senza eseguirla subito.

```
doppio = x -> x * 2
somma = (a, b) -> a + b
applica = f -> f(3)
println(doppio(5), " ", somma(1, 2), " ", applica(x -> x * x))
```

Output:

```
10 3 9
```

Una lambda con corpo a più istruzioni si scrive con `begin … end` o con `function (…) … end`. Le lambda senza parametri (`() -> …`) **non** sono supportate: usa `function () … end`.

8.2 Chiusure

Una lambda cattura l'ambiente in cui è creata. Secondo `SPEC-CORE §5` le variabili catturate sono **condivise**: le mutazioni sono visibili. **[Core]**

```
function add(n)
    x -> x + n
end
f = add(10)
g = add(100)
println(f(5), " ", g(5))

function contatore()
    v = 0
    x -> begin
        v += x
        v
    end
end
acc = contatore()
acc(5)
println(acc(10))
```

Output:

```
15 105
15
```

> ***Attenzione.*** *Il seed Windows cattura* ***per valore*** *e copre solo l'ultimo legame del frame: un accumulatore come* `contatore` *può non funzionare sul seed. L'interprete JS e la specifica usano il modello condiviso.*

Una chiusura non conserva una fotografia del solo testo della lambda: mantiene accesso ai legami che il suo corpo può leggere. Nel primo esempio la funzione restituita da `add(10)` mantiene il valore `n = 10` anche dopo che la chiamata a `add` è terminata; `add(100)` crea una chiusura distinta. Nel contatore, invece, la variabile `v` è un legame condiviso: ogni chiamata aggiorna lo stesso stato e la successiva lo osserva. È il motivo per cui una chiusura può rappresentare un piccolo oggetto con stato privato. È anche il motivo per cui il comportamento specifico del seed Windows va verificato separatamente dall'interprete di riferimento.

La differenza fra cattura condivisa e cattura per valore non è teorica: con il modello condiviso due chiusure create dallo stesso frame vedono lo stesso stato, con il modello per valore ognuna ha la sua copia. Quando progetti un oggetto con stato, sappi quale modello stai usando, perché è quello che decide se due contatori sono indipendenti o no.

8.3 Le regole di scope

Le regole seguono un principio unico: lo scope è **lessicale**, cioè conta dove il codice è scritto, non da dove viene chiamato. Da quel principio discendono le cinque regole che seguono; leggerle in ordine aiuta a prevedere che cosa modifica davvero un assegnamento.

1. `function`, lambda, `let`, `while`, `for` e le comprehension introducono **ambienti**; un nome è visibile nell'ambiente in cui è legato e in quelli interni (scope lessicale), mai in quelli esterni.
2. **Dentro una funzione nominata** l'assegnamento lega nel frame locale: una funzione **non modifica i globali**.
3. **Dentro una lambda** l'assegnamento a un nome già legato fuori aggiorna quel legame.
4. **Al top level** i cicli possono aggiornare i globali.
5. `let x = …` crea un nuovo `x`: assegnare `x` dentro il `let` non tocca l'`x` esterno.

```
x = 1
function f()
    x = 2
    x
end
println(f(), " ", x)
c = 0
function incrementa()
    c += 1
end
```

```
incrementa()
println(c)
```

Output:

```
 2 1
0
```

Per far uscire un risultato da una funzione, restituiscilo — oppure usa una lista, che è mutabile.

L'esempio mostra il caso più importante: `f` restituisce `2` e il globale `x` resta `1`, perché l'assegnamento dentro una funzione nominata crea un legame locale. `incrementa` incrementa un legame locale `c` che nasce e muore nella chiamata, e il globale `c` resta `0`. Non è una svista del linguaggio: è ciò che rende il comportamento di una funzione indipendente dallo stato globale. Quando hai bisogno di stato condiviso, le strade sono due — una lista mutabile passata come argomento (cap. 9) o una chiusura (cap. 8.2) — e in entrambi i casi la condivisione è visibile nella firma o nella definizione.

---

9. Liste

Una lista è eterogenea, annidabile e **mutabile**. Gli indici partono da **1**. **[Core]**

Una lista si scrive fra parentesi quadre, `[1, 2, 3]`; l'ordine degli elementi è significativo e i valori possono essere di tipi diversi. La mutabilità è la proprietà che distingue la lista da un record: la lista cresce e cambia, e più nomi possono osservare la stessa lista. È la struttura d'appoggio di quasi tutto il codice Core, dai record alle code, dagli ambienti ai parser.

```
xs = [1, 2, 3]
println(xs[1], " ", length(xs))
push!(xs, 4)
xs[1] += 100
println(xs)
alias = xs
push!(alias, 5)
println(xs)
```

Output:

```
1 3
[101, 2, 3, 4]
[101, 2, 3, 4, 5]
```

`push!` **muta** la lista e la restituisce. Le liste sono riferimenti: due nomi possono indicare la stessa lista, come si vede sopra.

Il fatto che `push!` restituisca la lista, e non `nothing`, è una scelta comoda: permette di concatenare l'aggiunta con un'altra operazione, ma rende anche possibile scrivere `xs = push!(xs, 4)`, che è ridondante ma innocuo. L'assegnamento a un elemento (`xs[1] += 100`) funziona anche con gli operatori composti: il lato sinistro viene valutato una volta sola, quindi l'indice non viene calcolato due volte.

Una lista è un oggetto mutabile; una variabile contiene un riferimento a quell'oggetto. Se `ys = xs`, i due nomi permettono di raggiungere la stessa lista: `push!(ys, valore)` e `xs[i] = valore` sono osservabili anche leggendo `xs`. Se invece scrivi `ys = [1, 2]`, hai sostituito soltanto il legame `ys`; `xs` continua a riferirsi alla lista precedente. Questa distinzione spiega perché assegnare una lista non ne crea automaticamente una copia e perché gli argomenti lista possono essere usati per condividere stato tra funzioni.

La convenzione 1-based è coerente con esempi e specifica: per visitare `n` elementi, il contatore inizia da 1 e la condizione tipica è `i <= length(xs)`. In un ciclo, aggiornare il contatore dopo aver letto l'elemento evita di saltare il primo elemento; aumentarlo in ogni iterazione assicura che la condizione prima o poi diventi falsa. Sul seed Windows la funzione ausiliaria sperimentale `get(xs, i)` è diversa e 0-based: non va usata come se fosse la semantica standard delle liste.

La scelta di partire da 1 non è un dettaglio estetico: rende coerente l'accesso con le posizioni umane e con i range, e permette di usare `length(xs)` come ultimo indice valido. Il prezzo è che un indice calcolato pensando allo 0 va corretto; l'errore tipico è `xs[0]`, che non restituisce un valore ma produce un errore. Un indice fuori intervallo non è `nothing`: in Core il programma

termina, in Full arriva un `Err(:bounds)` da gestire. Per questo, quando l'indice è calcolato, è buona norma controllarlo prima con `1 <= i && i <= length(xs)`.

La somma degli elementi con un ciclo esplicito:

```
xs = [101, 2, 3, 4, 5]
tot = 0
i = 1
while i <= length(xs)
    tot += xs[i]
    i += 1
end
println(tot)
```

Output:

```
115
```

Il ciclo mostra la forma canonica di attraversamento in Core: contatore da 1, condizione `i <= length(xs)`, lettura di `xs[i]`. `length` viene ricalcolata a ogni iterazione, quindi se il corpo del ciclo cambia la lista il confine si sposta; mutare la lista mentre la si attraversa è possibile, ma va fatto con consapevolezza. Aggiungere elementi allunga la lista e il ciclo li visiterà; in Full, dove esistono `pop!` e `append!`, togliere elementi mentre l'indice cresce fa saltare quelli successivi.

In Full esistono anche `reverse`, `sort`, `sum`, `first`, `last`, `rest`, `pop!`, `append!`, `vcat`, `collect` e l'operatore `x in xs`. Nessuno di questi nomi esiste in Core: se ti servono, li scrivi a mano con `while` e `push!`, come nel capitolo 22.1. Non è

una perdita: gli algoritmi scritti a mano mostrano che cosa fanno e quanto costano, e in un bootstrap questa trasparenza vale più della brevità.

10. Stringhe, simboli e conversioni

Le stringhe in Triad sono valori immutabili e le operazioni sono poche, volutamente esplicite: si concatena con `*`, si misura la lunghezza in byte, si converte con funzioni dal nome chiaro. La tabella raccoglie le operazioni disponibili in Core; dove Full si comporta diversamente lo trovi indicato sotto, con il perché.

operazione	significato
`a * b`	concatenazione di stringhe
`length(s)`	numero di **byte** UTF-8 (`length("é") == 2`)
`string(x)`	rappresentazione testuale di qualsiasi valore (`string(:a) == "a"`, `string([1, 2]) == "[1, 2]"`)
`Int(s)`	da stringa a intero (`Int("42") == 42`)
`Symbol(s)`	da stringa a simbolo (`Symbol("ciao") == :ciao`)
`typeof(x)`	in Core, il nome del tipo come simbolo (cap. 2.1)

```
nome = "Triad"
saluto = "ciao " * nome
println(saluto, " (", length(saluto), " byte)")
println("risposta: " * string(40 + 2))
println(Int("41") + 1, " ", Symbol("bob"), " ", string(true))
```

Output:

```
ciao Triad (10 byte)
risposta: 42
42 bob true
```

La prima cosa da notare è che `length(saluto)` vale `10`: "ciao Triad" ha dieci caratteri e dieci byte, perché l'alfabeto latino occupa un byte per carattere. Con `"é"` il conto cambia (`length("é") == 2`), e con un'emoji cambia ancora: la lunghezza è una proprietà della codifica UTF-8, non del testo come lo vedi. È il motivo per cui in Triad non si indicizza una stringa come una lista: `s[1]` restituirebbe un byte, non un carattere, e sarebbe una trappola silenziosa. Per lavorare sui singoli byte servono i primitivi di bootstrap (cap. 21).

La concatenazione con `*` non modifica `saluto`: crea una nuova stringa, e la vecchia resta disponibile a chiunque la stia usando. Questa immutabilità rende le stringhe sicure da passare alle funzioni, ma significa anche che ogni `*` alloca una stringa nuova: in un ciclo lungo il costo cresce con la stringa, non con il numero di giri.

`string(x)` e `Symbol(s)` vanno tenute distinte: `string` trasforma un valore in testo, `Symbol` trasforma testo in un'identità. `string(42)` produce la stringa `"42"`; `Symbol("42")` produce un simbolo, che è un valore completamente diverso. `Int("42")` fa il cammino inverso: è una conversione, non una valutazione, quindi il testo deve essere un intero scritto in decimale. Che cosa succeda con un testo non numerico non è fissato dallo standard: l'interprete di riferimento restituisce `0`, ma è un comportamento da verificare prima di contarci.

In Full le stringhe hanno in più l'interpolazione (cap. 1.3) e `repr`, che produce una rappresentazione pensata per essere riletta; in Core `string` è

l'unica porta dal valore al testo, e non esiste l'interpolazione.

11. Errori come dati

Questa è una delle idee portanti di Triad: **l'errore è un valore**, non un'eccezione che interrompe il programma.

11.1 La forma `Err`

Un errore è un valore `Err` con tre campi:

- `kind::Symbol` — la categoria, su cui si può fare dispatch;
- `msg::String` — il messaggio;
- `data` — un valore qualsiasi, spesso un record `Info(...)` con i dettagli.

[Full]

I tre campi separano tre domande diverse: *che cosa* è successo (`kind`), *come raccontarlo* (`msg`) e *quali informazioni servono per reagire* (`data`). Tenerle separate è ciò che permette di scrivere codice che decide in base alla categoria senza analizzare il testo del messaggio, che è pensato per una persona e non per un programma.

```
e = err(:mio, "ops", 42)
println(e.kind, " ", e.msg)
println(iserr(e))
```

11.2 Produrre e propagare

La tabella riassume i costrutti che creano, propagano o osservano un errore: leggerla dall'alto in basso è anche l'ordine in cui un errore attraversa un programma, dal punto in cui nasce al punto in cui qualcuno decide che cosa farne.

costrutto	effetto
`err(:kind, "msg"[, data])`	costruisce un `Err` **senza** sollevarlo
`throw(e)`, `error("msg")`	solleva (`error` produce `Err(:error, …)`)
`try b catch e h end`	se `b` solleva, `e` è legato all'`Err` e si valuta `h`
`try b end`	**l'errore diventa il valore** dell'espressione
`iserr(x)`	controlla se `x` è un `Err`

Tutti i fallimenti del linguaggio — divisione intera per zero, `sqrt(-1.0)`, indici fuori range, nessun metodo applicabile — arrivano come `Err` e **mai** come fallimenti fatali. **[Full]**

```
e = try div(1, 0) end
println(e.kind)
println(iserr(e))

r = try div(10, 2) catch err; 0 end
println(r)
```

Output (golden Full):

```
 domain
true
5
```

È utile distinguere tre momenti: **produrre**, **propagare** e **gestire** un errore. `err(...)` costruisce un dato e non interrompe nulla; un'operazione che fallisce produce un `Err` secondo la specifica; `try` stabilisce il punto in cui quel valore viene osservato o recuperato. In Full, `try espressione end` restituisce l'errore come valore, mentre `try ... catch nome ... end` permette al ramo di recupero di esaminare la categoria, il messaggio o i dettagli. Se l'errore non viene gestito, arriva al livello superiore e diventa un codice di uscita non riuscito: "errore come dato" non significa che il programma continui automaticamente dopo qualunque operazione.

Il tipo del dato permette di scrivere politiche diverse per categorie diverse. Per esempio, un errore `:bounds` può essere presentato come input non valido, mentre `:method` può indicare che manca un'implementazione. Separare la categoria stabile (`kind`) dal testo leggibile (`msg`) evita di dover analizzare stringhe di diagnostica per decidere come reagire.

11.3 Le categorie di errore

La categoria è la parte stabile dell'errore: è un simbolo, il programma può confrontarlo e fare dispatch su di esso. La tabella elenca le categorie che lo standard definisce; i messaggi che vedi stampati sono invece dell'implementazione e possono cambiare.

kind	quando
`:syntax`	errore di parse; `data` contiene `line`, `col` e `incomplete`
`:undef`	nome, funzione, macro o predicato non definito
`:type`	tipo sbagliato (campi, ritorno, `::`, condizioni non `Bool`)
`:method`	nessun metodo applicabile; `data.types` contiene i tipi degli argomenti
`:ambiguous`	più metodi ugualmente specifici
`:field`, `:bounds`, `:domain`	campo inesistente, indice fuori range, errore matematico
`:instantiation`	variabile logica non legata dove serve un valore
`:contract`	`requires`/`ensures` violata (cap. 12)
`:budget`	passi esauriti (cap. 13)
`:effect`	effetto non dichiarato (cap. 15)

kind	quando
`:host`	qualsiasi altro errore dell'interprete

Un errore non gestito al top level interrompe il programma: il messaggio riporta la riga dell'istruzione (`riga 12: …`) e il codice di uscita è `1`.

11.4 In Core: errori come liste taggate

Core non ha `try`/`catch` né il tipo `Err`: un errore **termina il processo** con exit code `1` e un messaggio diagnostico su stderr. Ma il pattern "errore come dato" si può già scrivere, ed è esattamente così che S1 implementerà Full. La convenzione è `[:Err, kind, msg]`. **[Core]**

```
err(kind, msg) = [:Err, kind, msg]
is_err(v) = typeof(v) == :List && length(v) == 3 && v[1] == :Err

function div_intera(a, b)
    if b == 0
        return err(:domain, "divisione per zero")
    end
    q = 0
    while a >= b
        a -= b
        q += 1
    end
    q
end

r = div_intera(7, 0)
if is_err(r)
    println("errore ", r[2], ": ", r[3])
end
println(div_intera(17, 5))
```

Output:

```
errore domain: divisione per zero
3
```

Nota che `&&` cortocircuita: `v[1]` viene valutato solo se `v` è una lista di 3 elementi.

Il perché di una lista taggata è semplice: in Core non esiste un tipo `Err`, ma esistono le liste e i simboli, e una lista con il tag in prima posizione può rappresentare qualunque record. `is_err` è quindi una convenzione, non una garanzia di tipo: chiunque può costruire una lista `[:Err, ...]` con campi sbagliati, e chi la riceve deve controllare ciò che gli serve. È lo stesso patto che gli interpreti in `selfhost/` usano per le struct (cap. 2.3). La parte importante non è il contenitore, è l'idea: un fallimento è un valore che viaggia come gli altri, e chi lo riceve può scegliere se gestirlo, trasformarlo o restituirlo.

---

12. Contratti

Un contratto è una precondizione (`requires`) o una postcondizione (`ensures`) dichiarata nel corpo di una funzione. **[Full]**

Un contratto non è un commento: è una condizione eseguita, e una sua violazione è un errore come dato. Serve a rendere esplicito che cosa una funzione pretende da chi la chiama e che cosa promette a chi la riceve, senza affidare il patto alla buona volontà di chi legge. Non dimostra che la proprietà valga per ogni input possibile — la controlla per le esecuzioni che avvengono — ma trasforma una violazione silenziosa in un `Err` che riporta quale clausola è fallita.

```
function safe_div(a::Number, b::Number)::Float
    requires b != 0
    ensures abs(result * b - a) < 1e-9
    a / b
end
```

- `requires c` viene controllata **nel punto in cui compare**: per convenzione all'inizio del corpo.
- `ensures c` viene registrata e controllata all'uscita della funzione, con `result` legato al valore restituito, **dopo** il controllo del tipo di ritorno.
- Una violazione produce `Err(:contract, "requires violata in safe_div: b != 0", Info(fn, clause, cond[, result]))`. `cond` contiene il **codice** della condizione: è un dato, si può ispezionare.
- Le condizioni sono **pure**: un effetto dentro un contratto dà `Err(:effect)`.
- Al top level `requires c` funziona come un'asserzione.

- `ensures` ha senso solo dentro una `function`: usa `result`, che esiste soltanto quando la funzione sta per restituire un valore. Fuori da una funzione non c'è un risultato a cui riferirsi.

```
println(safe_div(6, 3))


e = try safe_div(1, 0) end
println(e.kind)
println(e.data.clause)
println(e.data.fn)
```

Output (golden Full):

```
2.0
contract
requires
safe_div
```

Nell'esempio `safe_div(6, 3)` produce `2.0`: il tipo di ritorno `Float` converte il risultato e lo espone come promessa mantenuta. `safe_div(1, 0)` non arriva mai a dividere: la precondizione viene controllata all'inizio, l'errore è già un valore e il programma continua. Il messaggio dell'errore porta con sé `clause = requires` e `fn = safe_div`, quindi chi chiama può distinguere una violazione d'ingresso da una promessa d'uscita non mantenuta senza leggere il testo.

La sequenza di controllo è significativa: il corpo valuta le clausole nell'ordine in cui compaiono; le precondizioni vengono verificate nel punto in cui sono dichiarate, mentre le postcondizioni vengono conservate fino al completamento della funzione. Il valore finale viene prima sottoposto al

vincolo di ritorno e poi esposto come `result` alle clausole `ensures`. Un errore di contratto include quale clausola è fallita e il codice che la rappresenta, così chi chiama può distinguere una violazione d'ingresso da una promessa d'uscita non mantenuta. I contratti controllano un'invariante osservabile, ma non provano matematicamente che valga per ogni input possibile: vengono valutati per le esecuzioni che avvengono.

---

13. Budget

Un **budget** limita quanti passi il programma può consumare. Un passo è una chiamata, un'iterazione di ciclo o un passo di risoluzione. Quando il budget si esaurisce, **il valore del `with` è `Err(:budget)`**: il programma non si blocca e non viene interrotto. **[Full]**

```
r = with budget(steps = 10_000)
    while true end
end
println(r.kind)
```

Output (golden Full):

```
budget
```

I budget si annidano e vale il più stretto. Un `with` converte in valore **solo** l'esaurimento del proprio budget, mentre quello di un budget esterno continua a propagarsi. `solve … steps N` è il budget specifico di una query.

Il budget misura il **lavoro**, non il tempo: due esecuzioni dello stesso programma con lo stesso budget consumano gli stessi passi, quindi il risultato è riproducibile. Un timeout basato sull'orologio non lo sarebbe e renderebbe i test instabili. Per questo il limite è parte del linguaggio e non dell'ambiente: un programma può decidere che cosa fare quando lo esaurisce, e quella decisione è anch'essa codice.

Il motore logico usa uno stack esplicito: la ricorsione infinita, anche sinistra, finisce in `Err(:budget)` e **non** in uno stack overflow.

Il limite è parte del risultato del calcolo perché il programma può scegliere come trattarlo: mostrare una diagnosi, ridurre il problema o restituire un risultato parziale solo se la propria logica lo consente. Non confondere `limit n`, che limita il numero di soluzioni restituite, `depth d`, che taglia i rami oltre una profondità, e `steps s`, che limita il lavoro complessivo. Una query che ha esplorato lo spazio previsto senza trovare soluzioni restituisce una lista vuota; una query fermata dall'esaurimento dei passi segnala invece `Err(:budget)`.

---

14. Tracce

La traccia di un calcolo è un **valore**, non output stampato. **[Full]**

```
fib(n::Int) = n < 2 ? n : fib(n - 1) + fib(n - 2)

t = with trace
    fib(3)
end
println(t.result)
println(length(t.trace))
```

Output (golden Full):

```
2
10
```

`fib(3)` effettua 5 chiamate (`fib(3)`, `fib(2)`, `fib(1)`, `fib(0)`, `fib(1)`): la traccia registra **due eventi per ogni chiamata**, `:call` e `:exit`, quindi 10 eventi. Per `fib(5)` sono 15 chiamate e 30 eventi — è il valore atteso dal caso di conformance `garanzie_core`.

`with trace … end` restituisce `Traced(result, trace)` con gli eventi `:call` e `:exit` di ogni funzione generica chiamata nel corpo. `solve … trace` restituisce `Traced(result, trace)` dove `result` è la lista delle soluzioni e gli eventi sono le porte del modello a scatole di Byrd (`call`, `exit`, `redo`, `fail`) più `depth`, quando un ramo viene tagliato da `depth`.

Gli eventi sono record: si filtrano e si contano come ogni altro valore.

Raccogliere una traccia non è gratis: ogni evento è un valore allocato, e su calcoli grandi la traccia può pesare più del risultato. Chiedila quando ti serve, non come impostazione predefinita.

Una traccia aggiunge una dimensione temporale ai dati di debug: due esecuzioni possono avere lo stesso risultato ma seguire percorsi diversi, per esempio se una query ha dovuto provare e scartare alternative. Invece di scrivere log dentro ogni funzione, il chiamante racchiude il calcolo in `with trace` oppure chiede `trace` a `solve`; il risultato e gli eventi vengono restituiti insieme. Questo mantiene separati il risultato applicativo e la diagnostica, e consente di analizzare la traccia con gli stessi strumenti usati per liste e record.

Glossario degli eventi di traccia (porte di Byrd)

Quando si usa `solve … trace`, ogni passo della ricerca produce un evento che registra **a che riferimento** (quale goal, quale clausola, quali variabili logiche) si sta lavorando. Le quattro porte del modello a scatole di Byrd sono:

porta	quando si verifica	significato
`:call`	si entra in un goal	inizio della valutazione della clausola candidata
`:exit`	unificazione **riuscita**	il goal è dimostrato: è il "success" della ricerca
`:redo`	backtracking	si torna indietro per cercare un'altra soluzione
`:fail`	nessuna clausola applicabile	il goal **fallisce**: nessuna soluzione per quel ramo

Note importanti:

- **"success" e "fail" non sono esiti generici**: sono le porte `:exit` e `:fail` della traccia, riferite al goal/clausola in corso. Il fallimento di un goal non è un'eccezione: si manifesta come **lista di soluzioni vuota** nel `result` di `Traced(result, trace)`.
- `depth` appare come evento aggiuntivo quando un ramo viene tagliato dal limite di profondità (`solve … depth N`), segnalando che la ricerca è

stata interrotta prima di un `:exit`/`:fail`.

- Gli eventi sono **record ordinari**: si filtrano (`[e for e in t.trace if e.port == :fail]`), si contano (`length`), si ispezionano (`e.args`, `e.depth`) come ogni altro valore Triad.
- Il riferimento di ogni evento è il **goal corrente e le variabili logiche** (`?x`, `?y`, …) coinvolte in quel passo: leggendo la traccia si ricostruisce *quale* alternativa è stata provata e *perché* è fallita.

```
# Conta i fallimenti di una ricerca
t = solve path(:a, :c) trace
fails = [e for e in t.trace if e.port == :fail]
println(length(fails))
```

```
fib(n::Int) = n < 2 ? n : fib(n - 1) + fib(n - 2)
t = with trace
    fib(3)
end
println([e.args for e in t.trace if e.port == :call])
```

Output (golden Full):

```
[[3], [2], [1], [0], [1]]
```

---

15. Effetti dichiarati

Le funzioni sono **pure per impostazione predefinita**. Gli effetti si dichiarano nella firma, dopo i parametri. **[Full]**

«Puro per impostazione predefinita» è una scelta di sicurezza: se dimentichi una dichiarazione, la funzione non può fare più del previsto, e l'errore arriva quando l'effetto viene tentato. In questo senso un effetto è una **capacità**, non un'annotazione descrittiva: `!io` dice che la funzione ha il permesso di stampare, e il controllo dinamico verifica il permesso al momento dell'esecuzione.

```
function saluta(nome::String) !io
    println("ciao, ", nome)
end

function impara(x) !state
    fact visto($x)
end
```

effetto	operazioni
`io`	`println`, `print`
`state`	`fact` / `rule` eseguiti dentro una funzione (modificano la knowledge base)

Il controllo è **dinamico**. Una funzione viene eseguita solo con gli effetti che dichiara, e solo se il chiamante li permette. Se una funzione pura chiama

`println`, o chiama una funzione `!io`, si ottiene `Err(:effect)`; `data` contiene `fn`, `caller` e `missing`.

La purezza per impostazione predefinita significa che una funzione può calcolare e restituire valori senza modificare lo stato esterno né produrre I/O; non significa che il runtime analizzi staticamente ogni possibile percorso prima dell'esecuzione. Il controllo dinamico osserva gli effetti mentre avvengono e li confronta con il contesto attivo. Le mutazioni locali e quelle di liste non sono classificate come effetti dichiarati in questa specifica: `!state` si riferisce alla modifica della knowledge base tramite `fact` e `rule`, non a qualunque assegnamento.

- Il top level permette tutti gli effetti.
- Le lambda ereditano gli effetti del contesto in cui vengono eseguite.
- Macro e contratti sono puri.
- Leggere la knowledge base (`solve`, chiamare un predicato) è puro.
- La mutazione locale di dati (`push!` su una lista, assegnamento a variabili locali) **non** è un effetto: modifica solo valori raggiungibili dall'ambiente locale.

```
function rumorosa()
    println("faccio rumore")
end

function tranquilla()
    rumorosa()
end
```

```
e = try tranquilla() end
println(e.kind)
```

Output (golden Full):

```
effect
```

`rumorosa` non dichiara `!io`, quindi il suo `println` è un effetto non permesso; `tranquilla` la chiama da un contesto puro e riceve lo stesso errore. `data` contiene `fn` (chi ha tentato l'effetto), `caller` (da dove) e `missing` (quale effetto mancava): sono le tre informazioni che servono per correggere la firma. Il controllo è dinamico, quindi un ramo che non viene eseguito non produce errori di effetto, e non c'è un'analisi statica che ti avvisa prima: la dichiarazione è un patto che il runtime verifica mentre il codice gira.

---

16. Logica

Triad integra la programmazione logica **nella sintassi a blocchi**: i fatti e le regole sono costrutti di primo livello, non un linguaggio incorporato. **[Full]**

Fatti e regole formano la **knowledge base**, che il programma costruisce mentre gira. Un fatto asserisce che una relazione vale; una regola dice che vale a condizione che valgano altre cose. `solve` pone una domanda alla base e restituisce le risposte come valori: non c'è un interprete separato da invocare, perché la logica sta dentro la stessa sintassi e lo stesso motore del resto del linguaggio.

16.1 Fatti, regole, termini

```
fact parent(:tom, :bob)
fact parent(:bob, :jim)

rule ancestor(?a, ?d) :- parent(?a, ?d)
rule ancestor(?a, ?d) :- parent(?a, ?x), ancestor(?x, ?d)
```

Dentro un termine:

- `:atomo` è un simbolo; `?x` è una variabile logica; numeri, stringhe e liste sono se stessi;
- `f(t1, t2)` è un termine composto. Anche `?x + 1` è un termine (`+(?x, 1)`) e **non viene valutato**;
- un **nome nudo** è un valore dell'ambiente corrente, letto al momento in cui viene eseguito `fact`, `rule` o `solve`: `chi = :bob; solve parent(chi, ?c)`;
- `$(espr)` inserisce esplicitamente un valore calcolato: `fact eta(:bob, $(2026 - 1975))`;
- `[a, b]` e `[?h | ?t]` sono liste e forme che unificano con le liste;
- il nome di una struct con argomenti, come `Point(?x, ?y)`, unifica con le istanze di quella struct.

Le variabili di una clausola vengono **rinominate a ogni uso** (standardizzazione a parte). L'unificazione fa l'**occurs-check**: `solve ?x = f(?x)` non ha soluzioni.

Per leggere una query occorre distinguere i **termini** dai normali valori calcolati. Un simbolo come `:alice` è un termine costante; `?who` è una

variabile logica che cerca un valore; una forma come `parent(:alice, ?who)` descrive un goal da dimostrare. Le variabili logiche non sono variabili locali Triad assegnate con `=`: il motore cerca associazioni che rendano compatibile il goal con fatti e regole. In particolare, `?x + 1` dentro un termine non si calcola da sé; l'operatore `is` esprime esplicitamente la richiesta di valutare un'espressione usando variabili già legate.

L'unificazione costruisce progressivamente un insieme di associazioni. Se si confrontano `parent(:alice, ?who)` e `parent(:alice, :bob)`, gli argomenti corrispondenti coincidono e il motore può legare `?who` a `:bob`; se un argomento è diverso, quel tentativo fallisce. Quando più clausole possono soddisfare un goal, il backtracking torna al punto di scelta e prova la clausola successiva. Ogni soluzione è una fotografia delle variabili richieste dalla query; una lista vuota significa "nessuna dimostrazione trovata", non un errore di esecuzione. I limiti `limit`, `depth` e `steps` controllano quante soluzioni o quanti passi di ricerca sono ammessi.

16.2 I goal

Un goal è la descrizione di ciò che vuoi che sia vero: il motore cerca legami per le variabili che lo rendono dimostrabile. La tabella raccoglie le forme che un goal può assumere; le prime due sono le più importanti, perché `p(t…)` è l'unico modo per interrogare la knowledge base e la congiunzione è l'unico modo per chiedere più cose insieme.

goal	significato
`p(t…)`	predicato della knowledge base, risolto con SLD e backtracking, clausole in ordine
`g1 && g2`, `(g1, g2)`	congiunzione; nelle `rule` anche con `,`
`g1 || g2`	disgiunzione
`!g`, `not(g)`	negazione per fallimento
`not(b)` *fuori* da un goal	è una funzione: nega un `Bool` (`not(true) == false`)
`a = b`	unificazione
`?y is espr`	valuta `espr` (le variabili devono essere legate) e unifica con `?y`
`a < b`, `==`, `!=`, `<=`, `>`, `>=`	confronto tra valori, valutati

goal	significato
`f(t…)` con `f` funzione Triad	chiama la funzione con gli argomenti valutati; deve restituire `Bool`
`true`, `false`	successo, fallimento

Un predicato senza clausole con quell'arità, che non sia nemmeno una funzione, dà `Err(:undef, "predicato sconosciuto: p/2")`. Una variabile non legata dentro un'espressione dà `Err(:instantiation)`.

16.3 `solve` è un valore

`solve` **restituisce una lista** di `Solution`. Non stampa nulla: è un valore come un altro.

```
fact parent(:alice, :bob)
fact parent(:alice, :carol)
fact parent(:bob, :dave)

rule grandparent(?x, ?z) :- parent(?x, ?y), parent(?y, ?z)

sols = solve grandparent(:alice, ?who) limit 5
println(sols)
println(sols[1].who)
println(length(sols))
```

Output (golden Full):

```
[{who = :dave}]
dave
1
```

Ogni soluzione lega le variabili della query **non anonime**, nell'ordine in cui compaiono. Le variabili rimaste libere restano `?x#n`. `vars(sol)` restituisce i nomi legati.

`Solution` è un record con i nomi della query: `sols[1].who` legge il legame, e `vars(sol)` elenca i nomi. Il risultato di `solve` è una lista normale, quindi puoi

filtrare, ordinare o contare le soluzioni con gli strumenti del resto del linguaggio: la logica non produce un formato speciale, produce valori.

Opzioni di `solve`:

- `limit n` — al massimo `n` soluzioni;
- `depth d` — i rami più profondi di `d` vengono tagliati (falliscono);
- `steps s` — al massimo `s` passi di risoluzione, altrimenti `Err(:budget)`; il valore predefinito è `1 000 000`;
- `trace` — vedi cap. 14.

Le opzioni non sono equivalenti: `limit` ferma la ricerca quando hai abbastanza soluzioni, `depth` taglia i rami troppo profondi e li considera falliti, `steps` limita il lavoro complessivo e segnala l'esaurimento con `Err(:budget)`. Distinguerle conta perché solo l'ultima è un errore: le altre due producono un risultato parziale ma legittimo.

16.4 Un predicato usato come funzione

Chiamare un predicato dove ci si aspetta un valore restituisce un `Bool`: dice se il goal è dimostrabile.

```
if grandparent(:alice, :dave)
    println("alice è nonna di dave")
end
```

16.5 Esempi

Percorso con costo:

```
fact edge(:a, :b, 2)
fact edge(:b, :c, 3)
rule path(?x, ?y, ?c) :- edge(?x, ?y, ?c)
rule path(?x, ?y, ?c) :- edge(?x, ?z, ?c1), path(?z, ?y, ?c2), ?c is ?c1 + ?c2
println((solve path(:a, :c, ?costo))[1].costo)
```

Output (golden Full):

```
5
```

Il costo si accumula per composizione: `path(:a, :c, ?costo)` dimostra il goal unendo il fatto `edge(:a, :b, 2)` alla regola ricorsiva; il secondo passo trova `edge(:b, :c, 3)` e `?c is ?c1 + ?c2` calcola `2 + 3`. `is` è necessario perché `?c1 + ?c2` dentro un termine resterebbe un termine non valutato; `is` è il punto in cui la logica chiede al valutatore di fare l'aritmetica.

Append in stile Prolog:

```
rule app([], ?l, ?l)
rule app([?h | ?t], ?l, [?h | ?r]) :- app(?t, ?l, ?r)
println((solve app([1, 2], [3], ?r))[1].r)
println(length(solve app(?a, ?b, [1, 2, 3])))
```

Output (golden Full):

```
[1, 2, 3]
4
```

`app([1, 2], [3], ?r)` unifica la lista concatenata e restituisce `[1, 2, 3]`. La seconda query non fissa il risultato ma chiede tutte le coppie di liste che, concatenate, danno `[1, 2, 3]`: sono quattro, perché il primo pezzo può essere la lista vuota, `[1]`, `[1, 2]` o `[1, 2, 3]`. Il numero non è un errore: è il motore che enumera le alternative, e `limit` serve proprio a fermarlo quando non ti interessano tutte.

Negazione per fallimento:

```
fact parent(:tom, :bob)
persone = [:tom, :bob]
foglie = filter(p -> !parent(p, ?_), persone)
println(foglie)
```

Output (golden Full):

```
[:bob]
```

`!parent(p, ?_)` è la negazione per fallimento: `p` passa il filtro solo se il goal `parent(p, ?_)` non è dimostrabile. `:tom` ha un figlio, quindi non passa; `:bob` non ne ha, quindi passa. La variabile anonima `?_` è quella che non compare nel risultato: ogni sua occorrenza è una variabile nuova, così non devi inventare un nome per una variabile che non ti interessa.

---

17. Codice come dato

Il parser produce **un solo tipo di nodo**, `Node(head, args)`. È lo stesso albero che il kernel esegue, che `:( … )` restituisce come valore e che `triad --tree` stampa come S-expression. **[Full]**

L'idea che sta dietro è radicale: il programma non è una sequenza di comandi che il valutatore interpreta a mano, è un **dato**. Il parser produce l'albero, il valutatore esegue l'albero, le macro ricevono pezzi di albero e ne restituiscono altri. Poiché esiste un solo tipo di nodo, chi scrive codice che manipola il programma non deve conoscere dieci rappresentazioni diverse: conosce `Node(head, args)` e basta.

La tabella mostra come le forme che hai già incontrato diventano nodi: leggila come un dizionario tra la sintassi che scrivi e il dato che il programma può ispezionare.

sintassi	nodo
`f(a, b)`, `a + b`	`(call f a b)`, `(call + a b)`
`a && b`, `a || b`	`(and a b)`, `(or a b)`
`x = e`	`(assign x e)`
`:alice`	`(sym alice)`
`a.x`, `xs[i]`	`(dot a x)`, `(index xs i)`
`[a, b]`, `[h | t]`	`(list a b)`, `(cons (list h) t)`

sintassi	nodo
`[e for x in xs if c]`	`(comprehension e x xs c)`
`x -> e`	`(lambda (tuple x) e)`
`x::T`	`(typed x T)`
`"a $x"`	`(string "a " x)`
`if c a else b end`, `c ? a : b`	`(if c (block a) (block b))`
`while`, `for`, `let`, `begin`	`(while c body)`, `(for x iter body)`, `(let (tuple …) body)`, `(block …)`
`function f(a::T)::R !io … end`	`(function f (tuple (typed a T)) R (tuple io) body)`
`struct P; x::Float; end`	`(struct P (tuple (typed x Float)))`
`macro m(a) … end`, `@m(e)`	`(macro m (tuple a) body)`, `(macrocall m e)`
`:(e)`, `quote … end`, `$e`	`(quasi e)`, `(quasi (block …))`, `(unquote e)`
`fact t`, `rule h :- g1, g2`	`(fact t)`, `(rule h g1 g2)`
`solve g limit 5 trace`	`(solve g (tuple (assign limit 5) (assign trace true)))`
`try b catch e h end`	`(try b e h)`
`with budget(steps = N), trace … end`	`(with (tuple (call budget (assign steps N)) trace) body)`

17.1 Quote, unquote, eval

```
c = :(a.x + f(b, 2))
println(c.head)
println(sexpr(c))

n = 7
quadrato = :($n * $n)
println(eval(quadrato))
```

Output (golden Full):

```
call
(call + (dot a x) (call f b 2))
49
```

Dentro un quote, `$x` e `$(e)` inseriscono il **valore** di un'espressione: se è codice viene innestato, se è un numero o una lista diventa una costante, se è un Symbol diventa un identificatore. `eval(c)` esegue codice **nello scope globale**. Gli altri builtin del gruppo "codice" sono `parse`, `unparse`, `sexpr`, `Expr` e `gensym`.

Quotare e valutare sono operazioni opposte ma non intercambiabili. Il parser normalmente trasforma il sorgente in un albero e il valutatore esegue quell'albero; `:(...)` ferma il processo al confine tra le due fasi e restituisce l'albero come valore `Expr`. `eval(expr)` lo consegna di nuovo al valutatore, nello scope globale. L'unquote `$x` dentro una quote inserisce il valore calcolato di `x` nel nuovo albero: se è un `Expr`, innesta codice; se è un

numero, crea un nodo costante. `sexpr` serve a ispezionare la struttura e non esegue il programma.

L'esempio mostra il viaggio completo: `:(a.x + f(b, 2))` restituisce un `Expr` senza eseguirlo, `c.head` vale `call` e `sexpr` lo scrive in forma lineare. `:($n * $n)` con `n = 7` non crea una funzione: crea un albero che contiene già la costante `7`, e `eval` lo esegue. La differenza fra `eval` e una chiamata normale è lo scope: `eval` parte dallo scope globale, quindi non vede le variabili locali di chi lo invoca. È il motivo per cui `eval` si usa per costruire codice completo, non per leggere valori del contesto.

---

18. Macro

Una macro riceve gli argomenti **come codice**, non valutati, e restituisce codice, che viene poi eseguito al posto della chiamata `@nome(...)` nello scope del chiamante. **[Full]**

La differenza rispetto a una funzione non è solo tecnica: una funzione riceve valori già calcolati, quindi non può decidere se valutare un argomento una volta, due volte o mai. Una macro riceve la forma sorgente, e questa è la sua unica vera ragione d'essere: si usa quando il comportamento dipende dal codice, non dal valore degli argomenti.

```
 macro unless(c, body)
    :(if !$c; $body; end)
end

macro swap(a, b)
    t = gensym(:tmp)
    quote
        $t = $a
        $a = $b
        $b = $t
    end
end

x = 1
y = 2
@swap(x, y)
println(x, " ", y)
```

Output (golden Full):

```
2 1
```

L'espansione è **pura** (§15) e **non è igienica**: per i nomi temporanei si usa `gensym()`.

Questa distinzione è importante per gli effetti. Una funzione riceve normalmente valori già calcolati; una macro riceve la forma sorgente e può quindi decidere se inserirla una volta, più volte o non inserirla. Le macro di Triad non sono igieniche: un nome temporaneo introdotto dalla macro può collidere con un nome del chiamante. `gensym` crea un simbolo fresco per i temporanei, ma non rende automaticamente sicuro ogni riferimento libero. Per questo conviene limitare le macro a trasformazioni piccole e ispezionare l'albero espanso prima di attribuire al risultato un comportamento inatteso.

18.1 Codice che genera codice

```
function poly(coeffs)
    acc = :(0)
    i = 0
    for c in coeffs
        acc = :($acc + $c * x ^ $i)
        i += 1
    end
    acc
end

p = poly([1, 2, 3])
x = 2
println(eval(p))
```

Output (golden Full):

```
17
```

`poly` costruisce un albero a ogni giro: `:(0)` è il punto di partenza, e ogni coefficiente innesta l'albero precedente con `$acc`, la costante `$c` e l'esponente `$i`. Il `for` qui lavora su valori normali; solo i quote e gli unquote costruiscono codice. Con `[1, 2, 3]` l'albero finale rappresenta `1 + 2 * x + 3 * x^2`, e con `x = 2` vale `1 + 4 + 12 = 17`. `eval(p)` cerca `x` nello scope globale: se `x` fosse stato locale a una funzione, l'albero non l'avrebbe visto.

---

19. Builtin

Un **builtin** è una funzione che il linguaggio fornisce senza che tu la definisca. Alcuni sono primitive vere e proprie (l'aritmetica), altri sono comodità costruite sopra il nucleo (le funzioni di ordine superiore in Full). La tabella elenca i builtin di Triad-Full, raggruppati per ambito; sotto trovi la parte che esiste anche in Core.

ambito	funzioni
aritmetica e confronto	`+ - * / ^ % == != < <= > >= !`, `abs sqrt floor ceil round exp log sin cos div mod min max`; `*` concatena le stringhe
liste	`length isempty first last rest reverse sort sum push! pop! append! vcat collect`, `a:b`, `x in xs`
ordine superiore	`map filter reduce any all count`
stringhe e output	`string repr println print`
tipi	`typeof`, `isa`, `Int Float String Symbol`
errori	`err iserr throw error`
codice	`eval parse unparse sexpr Expr gensym`
logica	`vars(sol)`

Due dettagli che la tabella nasconde. Primo, gli **operatori sono funzioni**: in Full `+` è un nome come gli altri, quindi `reduce(+, xs)` funziona e puoi aggiungere metodi a un operatore (cap. 7). Secondo, builtin e funzioni utente non sono mondi separati: in Full puoi aggiungere metodi a un nome che è anche un builtin, come `+` su `Point`, e il builtin resta la scelta per i valori che non hanno un metodo più specifico.

Il sottoinsieme di Core è più piccolo: `+ - * % ^`, i confronti, `&& || !`, `length`, `push!`, la costruzione `[a, b]` e l'indicizzazione `xs[i]`, `*` sulle stringhe, `Int`, `string`, `Symbol`, `print`, `println`, `typeof` (`SPEC-CORE §7`). La riduzione è per progetto: un interprete di bootstrap deve poter eseguire se stesso, e ogni builtin in più è codice che quel bootstrap deve fornire. Per questo in Core `map`, `filter`, `reduce`, `sum`, `sort`, `div` e `mod` **non esistono**: si scrivono a mano (cap. 22.1).

---

20. Riga di comando e REPL

Un programma Triad non vive solo dentro un file: lo standard definisce anche come si invoca l'interprete. L'interfaccia di `triad` è normativa (`SPEC §14`), cioè fa parte del linguaggio tanto quanto la sintassi; se due implementazioni leggono gli argomenti in modo diverso, i programmi non sono portabili.

```
triad                           # REPL
triad FILE [ARG...]             # esegue FILE; ARGS == [ARG...]
triad -l FILE                   # carica FILE e apre il REPL
triad -i FILE                   # esegue FILE e poi apre il REPL
triad -e CODE                   # esegue CODE
triad -E CODE                   # esegue CODE e stampa il valore
triad --tree FILE               # stampa l'albero uniforme come S-expression
triad -q                        # REPL senza banner
triad -h, --help
triad -v, --version
triad -                         # legge lo script da stdin
```

Le opzioni coprono tre esigenze diverse. `-e` e `-E` servono per provare un frammento da terminale, con la differenza che la seconda stampa anche il valore; `-l` e `-i` combinano un file e la sessione interattiva, secondo che il file vada caricato o anche eseguito; `--tree` mostra l'albero uniforme del capitolo 17, utile quando vuoi vedere come il parser ha raggruppato il codice. Il trattino finale legge lo script da stdin, così Triad può stare in una pipeline.

- L'estensione `.triad`/`.td` si può omettere; lo shebang `#!` è ignorato.

- Il programma vede gli argomenti in `ARGS`, lista di stringhe, senza il nome del programma.
- Codici di uscita: `0` successo, `1` errore non gestito (su stderr), `2` uso non valido.

Questa interfaccia è normativa, ma non tutte le implementazioni la coprono: il seed Windows oggi esegue un file e basta, senza REPL né opzioni (cap. 29). Per eseguire gli esempi di questo manuale basta la forma `triad FILE`, oppure l'`node seed/dev/triad_core.cjs` mostrato in apertura.

20.1 Il REPL

Il REPL è una sessione interattiva: scrivi un'espressione, vedi il valore, e le definizioni restano attive tra un comando e l'altro. È utile per esplorare, per provare un goal logico o per capire perché un'espressione dà un certo risultato, senza creare un file ogni volta.

Il REPL è in stile Prolog:

```
triad> 1 + 2
3
triad> ?- ancestor(:tom, ?d)
?d = :bob ;
?d = :liz.
triad> parent(:jim, ?x)
false.
```

Le risposte mostrano i due modi del REPL: `1 + 2` è un'espressione Triad normale e stampa il valore; `?- ancestor(...)` è una query e stampa le soluzioni una per una, con `;` per chiedere la successiva e `.` per fermarsi. `false.` non è un errore: significa che non ci sono altre soluzioni.

- I blocchi restano aperti finché non sono chiusi da `end`.
- `?- goal` (il `.` finale è facoltativo) esegue una query con le opzioni di `solve`: `limit`, `depth`, `steps`, `trace`.
- Un'istruzione che è un goal su un predicato della knowledge base è trattata come query.
- Dopo ogni soluzione: `;`, `n` o spazio per la successiva, `a` per tutte le rimanenti, invio o `.` per fermarsi. `false.` significa nessun'altra

soluzione.

- Comandi: `:load FILE…` (`:l`), `:reload` (`:r`), `:listing [nome]` (`:ls`), `:tree CODICE`, `:reset`, `:help` (`:h`), `:quit` (`:q`, `halt`, Ctrl-D).
- Ctrl-C interrompe il calcolo in corso; un errore non gestito è stampato e il REPL continua.

---

21. Primitivi di bootstrap

Cinque primitive servono a S1 per leggere e analizzare il sorgente Triad. **Non fanno parte di Triad-Full** (`SPEC-CORE §8`): usale solo nel codice di bootstrap.

Sono primitive e non funzioni perché stanno **sotto** il linguaggio: servono a un interprete scritto in Triad per leggere il proprio sorgente come sequenza di byte. Per questo non fanno parte di Triad-Full: un programma applicativo non dovrebbe dipendere da loro, e il codice che le usa non è portabile fuori dal bootstrap. La loro presenza nel profilo Core spiega perché una stringa è una sequenza di byte e non una sequenza di caratteri (cap. 10).

primitiva	semantica
`readfile(path)`	contenuto del file come `String`; errore se non leggibile
`strlen(s)`	numero di byte
`strbyte(s, i)`	byte (`Int` 0–255) alla posizione `i`, da 1
`strsub(s, a, b)`	sottostringa dai byte `a` a `b` inclusi, da 1
`exit(n)`	termina il processo con codice `n`

Le cinque primitive sono volutamente minime: nessuna conosce i caratteri, nessuna conosce la sintassi. `strbyte` restituisce un `Int` fra 0 e 255 e `strsub` lavora su intervalli di byte inclusivi, quindi un programma che analizza sorgente deve fare da sé il confronto con i codici dei caratteri che gli interessano.

```
s = "abc 123 xyz"
cifre = 0
i = 1
while i <= strlen(s)
    b = strbyte(s, i)
    if b >= 48 && b <= 57
        cifre += 1
    end
    i += 1
end
println(cifre)
fine = 1
while fine <= strlen(s) && strbyte(s, fine) != 32
    fine += 1
end
println(strsub(s, 1, fine - 1))
```

Output:

```
3
abc
```

Il primo ciclo conta le cifre confrontando il byte con `48` e `57`, che sono i codici ASCII di `0` e `9`: è il modo in cui un analizzatore lessicale distingue un numero da un nome senza avere funzioni sulle stringhe (cap. 23, esercizio 6). Il secondo ciclo cerca il primo spazio e si ferma, poi `strsub(s, 1, fine - 1)` estrae i byte dalla posizione 1 all'ultimo prima dello spazio: `abc`. Nota l'ordine della condizione nel `while`: prima si controlla di essere dentro la stringa, poi si legge il byte; invertirle produrrebbe un accesso fuori intervallo.

Un programma può leggere se stesso con `readfile(ARGS[1])`: è il primo passo di un interprete self-hosted.

---

22. Stile e pattern idiomatici

Questo capitolo non introduce costrutti nuovi: mostra come combinare quelli dei capitoli precedenti nei casi che ricorrono sempre — trasformare una lista, rappresentare un record, tenere uno stato condiviso, costruire una stringa. Sono pattern, non regole del linguaggio, e li riconoscerai nel codice del progetto.

22.1 Funzioni di ordine superiore scritte in Core

In Core non esistono `map` / `filter` / `reduce`: si scrivono una volta, con `while` e lambda, e poi si usano ovunque.

La differenza rispetto alle versioni Full non è solo nel nome: qui il ciclo è esplicito, quindi vedi l'indice, la condizione di fine e il punto in cui l'elemento viene aggiunto. Se un giorno una di queste funzioni dà un risultato inatteso, il difetto è visibile in tre righe.

```
function mappa(f, xs)
    out = []
    i = 1
    while i <= length(xs)
        push!(out, f(xs[i]))
        i += 1
    end
    out
end


function filtra(p, xs)
    out = []
    i = 1
    while i <= length(xs)
        if p(xs[i])
            push!(out, xs[i])
        end
        i += 1
    end
    out
end
```

```
function riduci(f, acc, xs)
    i = 1
    while i <= length(xs)
        acc = f(acc, xs[i])
        i += 1
    end
    acc
end

xs = [1, 2, 3, 4, 5, 6]
println(mappa(x -> x * x, xs))
println(filtra(x -> x % 2 == 0, xs))
println(riduci((a, b) -> a + b, 0, xs))
```

Output:

```
 [1, 4, 9, 16, 25, 36]
[2, 4, 6]
21
```

`mappa` e `filtra` restituiscono liste nuove, `riduci` restituisce un valore; in tutti e tre i casi la funzione ricevuta è un parametro normale, quindi il comportamento si può cambiare senza toccare il ciclo. Nota che `riduci` accumula in `acc` e non muta `xs`: la lista in ingresso resta la stessa.

22.2 Record come liste taggate

In Core non c'è `struct`: un record è una lista con un simbolo come tag in prima posizione. La convenzione è importante perché senza struct non esiste un tipo nominale: il significato del record sta interamente nel tag e nell'ordine dei campi, che il programma deve rispettare da solo.

```
punto(x, y) = [:Punto, x, y]
px(p) = p[2]
py(p) = p[3]
somma_punti(a, b) = punto(px(a) + px(b), py(a) + py(b))
p = somma_punti(punto(1, 2), punto(10, 20))
println(p, " ", p[1] == :Punto)
```

Output:

```
[:Punto, 11, 22] true
```

L'esempio è costruito per mostrare il patto: `punto` è il costruttore, `px`/`py` sono gli accessori e nessuna funzione tocca gli elementi a mano tranne loro. Se un giorno cambi l'ordine dei campi, cambi solo il costruttore e gli accessori; chi usa `px` e `py` non se ne accorge. È lo stesso principio di incapsulamento di una struct, ottenuto con convenzioni invece che con il tipo.

22.3 Ambiente come lista di associazioni

È la struttura usata dagli interpreti in `selfhost/`: una lista piatta `[nome1, valore1, nome2, valore2, …]`.

```
function env_get(env, nome)
    i = 1
    while i < length(env)
        if env[i] == nome
            return env[i + 1]
        end
        i += 2
    end
    nothing
end

function env_set(env, nome, valore)
    i = 1
    while i < length(env)
        if env[i] == nome
            env[i + 1] = valore
            return env
        end
        i += 2
    end
    push!(env, nome)
    push!(env, valore)
end

env = []
env_set(env, :x, 10)
```

```
env_set(env, :y, 20)
env_set(env, :x, 11)
println(env, " ", env_get(env, :x), " ", env_get(env, :z))
```

Output:

```
[:x, 11, :y, 20] 11 nothing
```

È una lista piatta per non pagare un livello di annidamento: le coppie sono affiancate e `env[i+1]` legge il valore del nome in posizione `i`. Il costo è che ogni ricerca è lineare e che le due funzioni devono avanzare di due in due; il vantaggio è che la struttura è una lista normale, quindi si stampa e si ispeziona senza conversioni. È lo stesso schema che un interprete usa per l'ambiente dei nomi.

22.4 Stato mutabile condiviso

Poiché una funzione nominata non può riassegnare un globale, lo stato condiviso si tiene in una lista mutabile o in una chiusura.

```
stato = [0]
function tick()
    stato[1] += 1
end
tick()
tick()
println(stato[1])
```

Output:

```
2
```

`tick` non riassegna `stato`: muta l'elemento della lista che `stato` indica. È la differenza del capitolo 3 fra cambiare un legame e mutare un oggetto, e qui è ciò che permette a una funzione nominata di aggiornare uno stato esterno senza violare la regola 8.3 sull'assegnamento locale. La lista fa da contenitore: il legame `stato` resta lo stesso, il contenuto cambia. Se lo stato deve avere più campi, si allunga la lista (o si usa un record taggato come in 22.2).

22.5 Costruire stringhe

Due funzioni che tornano sempre: ripetere una stringa e unire una lista con un separatore. In Core non c'è una funzione che unisca una lista con un separatore, quindi il separatore va inserito a mano, e la posizione è tutto: va messo davanti a ogni elemento tranne il primo. Ricorda che ogni `*` crea una stringa nuova (cap. 10).

```
function ripeti(s, n)
    out = ""
    while n > 0
        out = out * s
        n -= 1
    end
    out
end

function unisci(xs, sep)
    out = ""
    i = 1
    while i <= length(xs)
        if i > 1
            out = out * sep
        end
        out = out * string(xs[i])
        i += 1
    end
    out
end
```

```
println(ripeti("ab", 3))
println(unisci([1, 2, 3], ", "))
```

Output:

```
ababab
1, 2, 3
```

`unisci` mette il separatore prima di ogni elemento tranne il primo: per questo il controllo `i > 1` viene prima delle due concatenazioni. Se lo mettessi dopo l'elemento, l'ultima concatenazione lascerebbe il separatore in fondo alla stringa.

22.6 Regole di stile

- Funzioni piccole con nomi chiari; caso base esplicito in ogni ricorsione; per profondità grandi preferisci `while`.
- Cicli con contatori espliciti (`i = 1 … while i <= length(xs)`).
- Non usare `fact`, `rule`, `solve`, `struct` e le altre parole chiave di Full come nomi: il codice deve restare valido anche in Triad-Full.
- Scrivi `!io` sulle funzioni che stampano: Core lo ignora, Full lo richiede.
- Non spezzare un'espressione su più righe: l'interprete di riferimento chiude l'istruzione a ogni a capo anche dentro le parentesi (cap. 1.6). Una istruzione per riga.

Queste regole non sono gusti personali: ognuna evita un problema concreto — un nome che collide in Full, un effetto non dichiarato, un'ambiguità di lettura, una riga spezzata che il runtime rifiuta.

Parte II — Esercizi

Ogni esercizio ha una soluzione verificata. Prova prima, poi apri il riquadro «Soluzione».

Gli esercizi della prima serie sono programmi Triad-Core e si eseguono con l'interprete di riferimento; quelli della seconda usano costrutti Full, quindi hanno risultati attesi ma non sono eseguibili oggi. In entrambi i casi la soluzione non è l'unico programma giusto: è un programma che funziona, ed è utile confrontarla con la tua per capire quale delle due è più chiara.

23. Esercizi guidati (Triad-Core)

Esercizio 1 — Potenza e somme

Senza usare `^`, scrivi `potenza(b, e)` ricorsiva e `somma_fino(n)` (somma `1 + … + n`) con `while`. Stampa `potenza(3, 13)`, `potenza(2, 10)` e `somma_fino(100)`.

▶ Soluzione

Esercizio 2 — Numeri primi

Stampa la lista dei numeri primi minori di 30.

▶ Soluzione

Esercizio 3 — Inverti una lista

Scrivi `inverti(xs)` che restituisca una nuova lista con gli elementi in ordine inverso.

▶ Soluzione

Esercizio 4 — Ordinamento per inserzione

Ordina in loco `[5, 2, 9, 1, 7]`.

▶ Soluzione

Esercizio 5 — Contatore di parole

Conta le parole (separate da spazi) di `"il linguaggio Triad "` usando `strbyte`.

▶ Soluzione

Esercizio 6 — Un mini-analizzatore lessicale

Trasforma `"12+x*3"` in una lista di token `[tipo, testo]`, con tipi `:num`, `:id`, `:op`. È il primo passo di un interprete.

▶ Soluzione

Esercizio 7 — Valutare un'espressione

Usando i token dell'esercizio 6, scrivi un valutatore a discesa ricorsiva per `+` e `*` con precedenza, dove le variabili vengono da una lista di associazioni. Suggerimento: tieni la posizione corrente in una lista `pos = [1]`, così le funzioni possono aggiornarla.

▶ Soluzione

Esercizio 8 — Un ambiente a dizionario

Estendi l'esercizio 7 (il valutatore con ambiente) con la rimozione di un nome, e scrivi `ambiente_da_elenco` che costruisce un ambiente da coppie `[nome, valore]`. Verifica con `env_get` ed `env_set` del capitolo 22.3.

▶ Soluzione

24. Esercizi su Triad-Full

Questi esercizi usano i costrutti dello standard completo. Sono definiti dalla specifica e hanno risultati attesi in `conformance/cases/`, ma **non sono ancora eseguibili** dal runtime corrente.

Studiarli serve comunque: le soluzioni sono codice normativo, e vederle accanto agli esercizi Core mostra che i costrutti Full si innestano nella stessa sintassi, non in un linguaggio separato. L'output riportato è un valore atteso conservato dalla suite, non una verifica eseguita oggi.

Esercizio 9 — Dispatch su struct

Definisci `struct Vec2` con due campi `Float` e tre metodi di `area`: uno per `Vec2`, uno per `Number` e uno per `Any`. Verifica che il metodo più specifico vinca.

▶ Soluzione

Esercizio 10 — Una base di conoscenza

Scrivi fatti e regole per `parent` e `sorella`, e usa `solve` per trovare le sorelle di `:bob` e i nipoti di `:alice`.

▶ Soluzione

Esercizio 11 — Contratti in pratica

Scrivi `dividi(a, b)` con `requires b != 0` e `ensures abs(result * b - a) < 1e-9`, e usa `try` per distinguere il caso di contratto violata dal caso di successo.

▶ Soluzione

Esercizio 12 — Una macro che misura

Scrivi la macro `@misura(label, espr)` che valuta l'espressione e stampa etichetta e valore, restituendo il valore.

▶ Soluzione

Parte III — Riferimento

Questa parte è pensata per la consultazione, dopo la lettura della prima: raccoglie la grammatica, le schede rapide, la tabella degli errori, il glossario e lo stato reale delle implementazioni. Non è materiale nuovo: se un punto non torna, il rimando al capitolo della Parte I ti riporta alla spiegazione. Tieni presente che le sezioni di riferimento descrivono lo **standard**; lo stato di ciò che è eseguibile oggi è raccolto nell'ultimo capitolo.

25. Grammatica

La grammatica è la forma più compatta in cui una specifica dice che cosa è un programma valido. Si legge come una ricetta: `=` significa «è fatto di», `{ … }` significa «zero o più», `[ … ]` «opzionale». Le due grammatiche che seguono non descrivono due linguaggi: la prima è il sottoinsieme Core, la seconda è Full, e le produzioni che Core condivide compaiono in Full con la stessa forma.

25.1 Triad-Core (EBNF normativa)

```
program  = { sep } { stmt sep { sep } } ;
sep      = NEWLINE | ";" ;
stmt     = funcdef | "return" [ expr ] | "break" | "continue" | expr ;
funcdef  = "function" NAME "(" [ NAME { "," NAME } ] ")" block "end"
         | NAME "(" [ NAME { "," NAME } ] ")" "=" expr ;
expr     = assegnamento ;
assegnamento = NAME ( "=" | "+=" | "-=" | "*=" ) expr | ternario ;
ternario = or_expr [ "?" expr ":" expr ] ;
or_expr  = and_expr { "||" and_expr } ;
and_expr = cmp_expr { "&&" cmp_expr } ;
cmp_expr = sum_expr { ( "==" | "!=" | "<" | "<=" | ">" | ">=" ) sum_expr } ;
sum_expr = mul_expr { ( "+" | "-" ) mul_expr } ;
mul_expr = pow_expr { ( "*" | "%" ) pow_expr } ;
pow_expr = unary { "^" unary } ;                    (* destra *)
unary    = ( "-" | "!" ) unary | postfix ;
postfix  = primary { "(" args ")" | "[" expr "]" } ;
primary  = INT | STRING | SYMBOL | "true" | "false" | "nothing" | NAME
         | "(" expr ")"
         | "[" [ expr { "," expr } ] "]"
         | "(" NAME { "," NAME } ")" "->" expr
         | "if" expr block { "elseif" expr block } [ "else" block ] "end"
         | "while" expr block "end"
         | "let" [ NAME "=" expr { "," NAME "=" expr } ] block "end"
         | "begin" block "end"
         | "function" "(" [ NAME { "," NAME } ] ")" block "end" ;
block    = { stmt sep { sep } } ;                   (* fino a end/else/elseif *)
```

La grammatica Core è volutamente piccola: niente `for`, niente range, niente tipi nelle firme. Non descrive però ogni dettaglio che l'esecuzione accetta:

l'assegnamento a un elemento (`xs[i] = e`, `xs[i] += e`), che gli esempi del capitolo 9 usano per mutare le liste, non compare nella produzione `assegnamento`. È una lacuna del testo della specifica, non del linguaggio: l'interprete di riferimento lo supporta, e il codice che lo usa è portabile fra le implementazioni Core.

25.2 Triad-Full (EBNF normativa)

```
program    = { sep } { stmt sep { sep } } ;
sep        = NEWLINE | ";" ;
block      = { sep } { stmt sep { sep } } ;

stmt       = funcdef | structdef | macrodef | fact | rule
           | "requires" expr | "ensures" expr
           | "return" [ expr ] | "break" | "continue"
           | "(" NAME { "," NAME } ")" "=" expr
           | expr ;

funcdef    = "function" fname "(" [ params ] ")" [ "::" TYPE ] [ "!" EFFECT {
"," EFFECT } ]
             block "end"
           | NAME "(" [ params ] ")" "=" expr ;
fname      = NAME | "+" | "-" | "*" | "/" | "%" | "^" | "==" | "<" | "<=" | ">"
| ">=" ;
params     = param { "," param } ;
param      = NAME [ "::" TYPE ] | "(" NAME { "," NAME } ")" ;
structdef  = "struct" NAME sep { NAME [ "::" TYPE ] sep } "end" ;
macrodef   = "macro" NAME "(" [ NAME { "," NAME } ] ")" block "end" ;

fact       = "fact" term ;
rule       = "rule" term [ ":-" goal { "," goal } ] ;

expr       = (* Pratt: precedenze dalla più bassa alla più alta *)
  1  a = b   a += b   a -= b   a *= b                (destra)
  2  x -> e   (a, b) -> e   c ? a : b                (destra)
  3  ||
  4  &&
  5  ==  !=  <  <=  >  >=  in  isa  is
```

```
  6  a:b                                              (range, senza spazi)
  7  +  -
  8  *  /  %
  9  -x  !x  +x  $x                                   (unari)
 10  ^                                                (destra)
 11  e::T                                             (asserzione di tipo)
 12  f(args)  xs[i]  a.campo                          (postfissi, senza spazio
prima di ( e [)

primary    = NUMBER | STRING | SYMBOL | LVAR | NAME | "true" | "false" |
"nothing"
           | "(" expr ")" | "(" expr "," ... ")"       (* tupla: parametri di
lambda *)
           | "[" [ expr { "," expr } [ "|" expr ] ] "]"
           | "[" expr "for" NAME "in" expr [ "if" expr ] "]"
           | ":(" expr ")" | "quote" block "end"
           | "@" NAME "(" args ")"
           | "if" expr block { "elseif" expr block } [ "else" block ] "end"
           | "while" expr block "end"
           | "for" NAME "in" expr block "end"
           | "for" "(" NAME { "," NAME } ")" "in" expr block "end"
           | "let" [ NAME "=" expr { "," NAME "=" expr } ] block "end"
           | "begin" block "end"
           | "try" block [ "catch" [ NAME ] block ] "end"
           | "with" modifier { "," modifier } block "end"
           | "solve" goal { "limit" expr | "depth" expr | "steps" expr | "trace"
}
           | "function" "(" params ")" block "end" ;
modifier   = "budget" "(" "steps" "=" expr ")" | "trace" ;
```

La grammatica Full elenca i livelli di precedenza nella forma numerata che il parser usa: è la stessa tabella del capitolo 4, e la produzione `primary` dice

quali forme possono stare alla base di un'espressione. Le produzioni `stmt` e `block` dicono invece che cosa può contenere un blocco, e quindi in quali punti `fact`, `rule`, `requires` e `ensures` sono ammessi.

---

26. Schede rapide

Le schede rapide non sostituiscono i capitoli: servono a ricordare la forma esatta di un costrutto quando sai già che cosa vuoi scrivere. Sono divise per tipo di problema — un valore, un controllo di flusso, una funzione — e ogni riga rimanda, quando serve, al capitolo che spiega la scelta.

26.1 Valori

vuoi	scrivi
un intero	`42`, `1_000`
un float **[Full]**	`3.14`, `1e-9`
un booleano	`true`, `false`
il vuoto	`nothing`
una stringa	`"testo"`
un simbolo	`:alice`
una lista	`[1, 2, 3]`
una lambda	`x -> x + 1`
una funzione	`function (x) x end`
un record Core	`[:Punto, 1, 2]`
una struct **[Full]**	`Point(3, 4)`
codice come dato **[Full]**	`:(a + b)`, `quote … end`

26.2 Controllo di flusso

vuoi	scrivi
una condizione	`if c … elseif d … else … end`
una condizione breve	`c ? a : b`
un ciclo	`while c … end` **[Core]** · `for x in xs … end` **[Full]**
uscire dal ciclo	`break`
saltare al passo dopo	`continue`
uscire dalla funzione	`return e`
uno scope nuovo	`let x = 1 … end`
un blocco senza scope	`begin … end`
una lista costruita **[Full]**	`[x * x for x in xs]`
catturare un errore **[Full]**	`try b catch e h end`

26.3 Funzioni

vuoi	scrivi
una funzione	`f(x) = x + 1`
un corpo multi-istruzione	`function f(x) … end`
un tipo di ritorno **[Full]**	`function f(x)::Float … end`
un effetto **[Full]**	`function f(x) !io … end`
un predicato in un goal **[Full]**	`function evena(n) n % 2 == 0 end`
una precondizione **[Full]**	`requires cond`
una postcondizione **[Full]**	`ensures cond`

27. Tabella degli errori

La tabella raccoglie i messaggi che l'interprete di riferimento stampa oggi in Triad-Core, con la causa tipica. I messaggi sono dell'implementazione, non dello standard: la categoria dell'errore è normativa (cap. 11.3), il testo no, quindi non scrivere programmi che dipendono da queste stringhe.

27.1 In Core

Core non ha `try`/`catch` né valori `Err`: un errore **termina il processo** con exit code `1` e un messaggio diagnostico su stderr. Quello già stampato su stdout resta.

messaggio	causa
`parse error: unexpected token …` / `parse error: expected …`	sintassi non Core (`1.5`, `/`, `() -> …`, `end` mancante)
`error: undefined variable NOME`	nome mai assegnato, o funzione chiamata prima di essere definita
`error: not a function`	si chiama un valore che non è una funzione (`x = 5; x(1)`)
`error: arity mismatch`	numero di argomenti diverso dai parametri
`error: index out of range`	`xs[i]` con `i < 1` o `i > length(xs)`
`error: + type error` (e simili)	operandi di tipo sbagliato (`1 + "a"`)
`error: if cond` / `while cond` / `ternary cond` / `&& expects bool`	condizione non `Bool`
`error: div by zero` / `negative exponent`	`x % 0`, `2 ^ -1`

messaggio	causa
`error: unknown function NOME`	builtin riservato usato in modo non supportato
`cannot read file: …`	file inesistente o non leggibile
`usage: triad FILE [ARG...]` (exit 2)	nessun file passato

27.2 In Triad-Full

Gli stessi fallimenti arrivano come `Err` con un `kind` (cap. 11.3), invece di terminare il processo. Un errore non gestito al top level stampa `riga N: …` e esce con codice `1`.

---

28. Glossario

Il glossario raccoglie i termini che il manuale usa con un significato preciso e che ricorrono in capitoli diversi. Le definizioni sono brevi apposta: il rimando al capitolo serve quando la parola da sola non basta.

termine	significato
Albero uniforme	l'unico tipo di nodo prodotto dal parser, `Node(head, args)`; è anche ciò che il kernel esegue
Ambiente	l'insieme dei legami di nomi a valori visibile in un punto del programma
Backtracking	il motore logico torna sui legami e prova la clausola successiva
Budget	limite di passi eseguibili; esaurito produce `Err(:budget)`
Clausola	un `fact` o una `rule` della knowledge base
Chiusura	funzione che cattura l'ambiente in cui è stata creata
Dispatch	scelta del metodo in base ai **tipi** degli argomenti
Effetto	capacità dichiarata di una funzione: `io`, `state`

termine	significato
Err	l'errore come dato: `kind`, `msg`, `data`
Goal	una domanda alla knowledge base: predicato, congiunzione, disgiunzione, unificazione
Knowledge base	l'insieme dei fatti e delle regole
Macro	una funzione che riceve e restituisce codice
Occurs-check	la regola che impedisce a `?x` di unificarsi con un termine che contiene `?x`
Porta (di Byrd)	uno dei quattro eventi `:call`, `:exit`, `:redo`, `:fail` che descrivono un passo della ricerca logica (cap. 14)
Precedenza	l'ordine in cui gli operatori si raggruppano (12 livelli in Full, 10 in Core)
Profilo	un sottoinsieme normativo dello standard: Core o Full
Scope lessicale	un nome è visibile dove è stato legato e all'interno, mai all'esterno
Soluzione	una `Solution`: i legami prodotti da un goal risolto
Standardizzazione a parte	le variabili di una clausola vengono rinominate a ogni uso

termine	significato
Termine	un dato della logica: costante, variabile logica o forma composta, non valutata finché non serve
Tracciato	il valore `Traced(result, trace)` restituito da `with trace` o `solve … trace`
Unificazione	l'uguaglianza logica: due termini si rendono identici legando le variabili
Valori di prima classe	funzioni e operatori usabili come qualsiasi altro valore

---

29. Cosa gira oggi

Questa sezione non fa parte del linguaggio: è lo stato delle implementazioni, utile per orientarsi prima di scrivere codice. Serve a rispondere a una domanda che un manuale rischia di lasciare implicita: quello che hai appena letto è **definito** dallo standard, ma che cosa puoi **eseguire** oggi? Le due cose non coincidono, e confonderle porta a due errori opposti: credere che un esempio Full non funzioni perché è sbagliato, o credere che un programma Core funzioni su ogni runtime.

profilo	stato
Triad-Core	**eseguibile**: interprete di riferimento JavaScript `seed/dev/triad_core.cjs` (37/37 casi Core) e, in parte, il seed Windows `seed/windows-x86_64/seed.exe`
Triad-Full	**non eseguibile**: struct, dispatch, logica, macro, contratti, budget, tracce, effetti, destrutturazione, `for`, `try`, `Float` sono specifica; nella conformance Full passa solo `errore`

Il profilo Full non è «quasi pronto»: è definito per intero nello standard, e i suoi esempi hanno risultati attesi conservati dalla suite, ma nessuno dei runtime del repository lo esegue oggi. Quando nel libro vedi `**Output (golden Full):**`, quindi, leggi un valore atteso, non una verifica appena fatta. Sul versante Core la situazione è diversa: l'interprete JavaScript passa l'intera suite, mentre il seed Windows copre solo una parte dei casi — in particolare le liste restano parziali.

Anche restando in Core, «eseguibile» non significa «identico ovunque»: esistono due runtime, e non coprono lo stesso sottoinsieme. La tabella elenca le differenze che contano per chi scrive codice destinato a girare su entrambi; leggerla prima di scegliere un costrutto evita di scoprire il problema solo quando il programma arriva sul seed.

Differenze note fra i due runtime Core:

funzionalità	interprete JS	seed Windows
`return`, `break`, `continue`	sì	non elencati nel README del seed
lambda	più parametri	**un solo parametro**
chiusure	condivise	**per valore**, solo l'ultimo legame del frame
`string(x)`	tutti i tipi	solo `Int` e `String`
liste	eterogenee, `xs[i]`, `length`, `push!`	solo interi; costruzione, stampa, `push!`
`get(xs, i)`	non esiste	funzione ausiliaria **non standard**, indici **da 0**
`ARGS`, primitivi di bootstrap	sì	non elencati nel README del seed
messaggi di errore	su stderr	solo exit code

Le righe più importanti sono le prime tre. Un `return` o un `break` che l'interprete JS esegue può non essere supportato dal seed; una lambda con due parametri funziona su uno e non sull'altro; una chiusura che aggiorna uno stato condiviso si comporta diversamente. Sono differenze di implementazione, non di standard: la semantica corretta è quella dell'interprete JS e della specifica, e il seed è il runtime più limitato. Se un programma deve girare su entrambi, scegli il sottoinsieme comune e non contare su ciò che una sola colonna dichiara.

Regola pratica: **scrivi codice che rispetta `SPEC-CORE` e verificalo con l'interprete JS**; se deve girare anche sul seed, restringiti alle funzionalità elencate nella colonna di destra e verificalo su Windows.

Due limiti noti che il lettore deve conoscere prima di scrivere codice:

- **ogni a capo chiude l'istruzione**, anche dentro `( )`, `[ ]` e negli argomenti di una chiamata. L'eccezione prevista da `SPEC §1` per Triad-Full (a capo dentro le parentesi, o subito dopo un operatore binario) non è implementata nell'interprete di riferimento. Scrivi una istruzione per riga;
- **il seed Windows non implementa l'indicizzazione standard `xs[i]`**: lì c'è la funzione ausiliaria non standard `get(xs, i)`, con indici **da 0**.

A questi due limiti va aggiunto un dettaglio di piattaforma: il seed Linux è allo stadio E0 e stampa solo un messaggio di prova, quindi per eseguire programmi su Linux oggi si usa l'interprete JavaScript (`node seed/dev/triad_core.cjs`). Il seed Windows è il runtime in codice macchina più completo, ma non è l'unico modo di eseguire Triad.

Che cosa farne, in pratica: per studiare e per scrivere codice usa l'interprete JS; per verificare la portabilità verso il bootstrap prova sul seed; per gli esempi Full leggi e confronta con i golden, sapendo che nessun runtime li esegue ancora. Il manuale non promette ciò che il repository non sa fare, e questa sezione esiste per rendere esplicito il confine.

Appendice A — Influenze dichiarate

Triad nasce da idee di Julia, Prolog e Lisp, ma **non è un dialetto di nessuno dei tre**: la definizione del linguaggio è la specifica.

Conoscere le influenze non è curiosità: se vieni da uno di questi linguaggi, sapere che cosa Triad ha ripreso e che cosa ha cambiato ti aiuta a non importare abitudini sbagliate. La tabella non è un elenco di parentele, è un avviso: le somiglianze sono di idea, non di comportamento.

da	idea ripresa
Julia	blocchi `function … end`, `struct`, tipi con `::`, multiple dispatch, operatori-funzione, `if` / `for` / `while` / `let`
Prolog	`fact` / `rule` come costrutti, unificazione con occurs-check, backtracking, `solve`, variabili logiche `?x`
Lisp	codice come dato, albero uniforme, `:( … )` / `quote` con `$`, macro

Propri di Triad, non presi in prestito: errori come dati, contratti, budget, tracce ed effetti dichiarati come parte del linguaggio, e l'integrazione dei tre modelli in una sintassi a blocchi.

Appendice B — Dove guardare

Questa appendice elenca dove approfondire. Sono materiali **facoltativi**: il manuale è autosufficiente, e questi rimandi servono a chi vuole risalire alla fonte normativa, controllare un caso limite o vedere il codice del progetto. Il percorso di studio per capitoli e la visita al codice sono un complemento, non un prerequisito.

vuoi	leggi
la definizione normativa completa	`SPEC.md`
il sottoinsieme di bootstrap	`SPEC-CORE.md`
il percorso di studio per capitoli e il codice del progetto	`TUTORIAL.md`
gli esempi completi con output	`examples/`
il contratto eseguibile, con i golden	`conformance/`
lo stato del bootstrap	`STATUS.md` · `BOOTSTRAP.md`
il piano di lavoro	`plan.md` · `TASK.md`
gli strumenti per gli editor	`editors/`
la licenza	`LICENSE` — MIT

Manuale scritto per lo standard Triad `0.2.0`. Le sezioni normative sono in `SPEC.md` e `SPEC-CORE.md`; in caso di divergenza, prevale la specifica.

Tutorial di Triad 2.0



Triad 2.0 — Tutorial: imparare il linguaggio e il codice del progetto

Questo tutorial è il percorso guidato con cui impari Triad partendo da zero. Non dà per scontato che tu conosca il linguaggio: dà per scontato solo che tu sappia programmare in qualche altro linguaggio. Se sai già scrivere cicli, funzioni e liste, hai tutto il necessario; il resto — la sintassi di Triad, il comportamento dei valori, la distinzione tra il sottoinsieme eseguibile e lo standard completo — è spiegato qui, nell'ordine in cui conviene impararlo.

Il percorso porta a tre risultati. Primo, **scrivere programmi Triad-Core che girano davvero**, cioè programmi completi, eseguibili oggi con l'interprete di riferimento. Secondo, **conoscere Triad-Full**, il linguaggio completo definito dallo standard, comprese le parti che il runtime attuale non esegue ancora. Terzo, **saper leggere il codice del progetto**: l'interprete di riferimento, il seed in codice macchina, la suite di conformità e gli strumenti di sviluppo. Il tutorial non rimanda altrove per capire i concetti: la specifica (`SPEC.md`, `SPEC-CORE.md`) e i casi di `conformance/` sono approfondimenti facoltativi, non prerequisiti.

Che cosa impari e come è organizzato

Il tutorial è un libro in cinque parti, pensate per essere lette in ordine ma abbastanza autonome da poter essere consultate separatamente.

- **Parte I — Primi passi.** Che cos'è Triad, come si installa e si esegue, il primo programma spiegato riga per riga.
- **Parte II — Triad-Core passo per passo.** Il sottoinsieme eseguibile oggi, costrutto per costrutto: lessico, valori, variabili, espressioni, controllo di flusso, funzioni, chiusure, liste, stringhe, primitivi di bootstrap, errori e differenze tra i runtime. Gli esercizi guidati in fondo alla parte servono a fissare quello che hai letto.
- **Parte III — Triad-Full (lo standard completo).** Tipi dichiarati, multiple dispatch, struct, `for` e comprehension, logica con fatti e regole, macro, contratti, budget, tracce ed effetti. È il linguaggio definito dalla specifica: qui lo studi, ma il runtime attuale non lo esegue ancora.
- **Parte IV — Il codice del progetto.** Una visita guidata al repository: l'interprete di riferimento, il seed, S1, i casi di conformità, la CLI, il benchmark, il flusso di contributo. Serve a capire come il linguaggio è implementato, non solo come si usa.
- **Parte V — Riferimento.** Grammatica, schede rapide, messaggi di errore, glossario, stato del progetto e FAQ: capitoli da consultare, più che da leggere di seguito.

Che cosa significa «eseguibile oggi». Lo standard Triad definisce due profili di conformità. Un esempio è *eseguibile oggi* quando gira, senza modifiche, su un'implementazione che esiste già nel repository; è *definito*

dallo standard quando la specifica lo descrive ma nessun runtime attuale lo esegue. Il tutorial distingue sempre i due casi, e ogni blocco di codice dichiara il profilo a cui appartiene.

profilo	dove è definito	eseguibile oggi?
Triad-Core	`SPEC-CORE.md`	**sì**: interprete di riferimento JavaScript `seed/dev/triad_core.cjs` (37/37 casi Core) e, in parte, il seed Windows `seed/windows-x86_64/seed.exe` (33 casi Core verificati in CI)
Triad-Full	`SPEC.md`	**no**: struct, dispatch, logica, macro, contratti, budget, tracce, effetti, destrutturazione, `for`, `try`, `Float` sono specifica; nella conformance Full passa solo `errore`

Due precisazioni sul contesto, utili per non confondere il libro con il progetto. La versione dello standard è la `0.2.0`, registrata in `VERSION`. Triad non dipende da Julia: quella vecchia implementazione è stata rimossa, e gli output Triad-Full che leggi nel libro sono i **golden**, cioè i risultati attesi prodotti a suo tempo da quell'implementazione e congelati in `conformance/cases/`; non sono stati riprodotti dal runtime attuale. Al contrario, ogni esempio marcato **Output:** è un programma Triad-Core verificato con `node seed/dev/triad_core.cjs`; gli esempi Full sono marcati **Output (golden Full):** oppure non riportano output.

Come usare questo tutorial

Non c'è un solo modo di leggere il libro: dipende da quello che ti serve. La tabella qui sotto riassume i percorsi principali. Chi non conosce Triad, però, dovrebbe cominciare dalla Parte I: i capitoli successivi danno per acquisito il lessico del linguaggio.

vuoi…	leggi
scrivere programmi Triad che girano oggi	Parte I → Parte II (capitoli 4–17, con esercizi)
conoscere il linguaggio completo dello standard	Parte III (capitoli 18–32)
contribuire all'interprete JS, al seed, a S1, alla CLI o alla conformance	Parte IV (capitoli 33–43)
consultare grammatica, schede rapide, errori, glossario	Parte V (capitoli 44–49)

Alcune convenzioni rendono più facile seguire gli esempi. I comandi sono mostrati per Linux/macOS (`sh`) e, quando sono diversi, per Windows (`cmd`/PowerShell). I blocchi `triad` contengono codice Core, normalmente eseguibile con l'interprete JS; i frammenti specifici del seed Windows sono indicati nel testo. I blocchi `triad-full` mostrano codice definito dallo standard Full, non ancora eseguibile dal runtime corrente. Con **interprete JS** si intende `seed/dev/triad_core.cjs`, l'interprete di riferimento Triad-Core usato su Linux/macOS e dal runner di conformità; con **seed** si intende `seed/windows-`

`x86_64/seed.exe`, l'interprete Triad-Core in codice macchina (S0). Dove i due si comportano diversamente lo dice il capitolo 15. I sorgenti si scrivono in file `.triad` (estensione canonica) o `.td` (alias storico).

Parte I — Primi passi

1. Cos'è Triad

Triad è un linguaggio di programmazione autonomo che riunisce tre paradigmi — imperativo, logico e metaprogrammatorio — in un'unica sintassi a blocchi. L'idea di fondo è che questi paradigmi non vadano tenuti in linguaggi separati: un programma Triad può definire funzioni e cicli come un linguaggio imperativo, dichiarare fatti e regole come Prolog e trattare il codice come un valore costruibile e trasformabile come in Lisp, e le tre cose convivono nello stesso file senza strati né interfacce.

Le influenze dichiarate sono tre, e per ciascuna il libro mostrerà via via che cosa Triad ha ripreso:

influenza	idea ripresa in Triad
Julia	`function … end`, `struct`, tipi con `::`, multiple dispatch, operatori come funzioni, `if` / `for` / `while` / `let`
Prolog	`fact`, `rule`, unificazione con occurs-check, backtracking, `solve` con variabili logiche `?x`
Lisp	codice come dato (`Node`), `:( … )` / `quote` con `$`, macro

A questi si aggiungono, come parte del linguaggio e non di una libreria: **errori come dati** (`Err`), **contratti** (`requires` / `ensures`), **budget** (`with budget`), **tracce** (`with trace`) ed **effetti dichiarati** (`!io`, `!state`).

La distinzione tra «parte del linguaggio» e «libreria» non è nominale, e vale la pena capirla subito. Un errore che è un valore può essere restituito, ispezionato e composto come qualsiasi altro dato, invece di interrompere il

programma; un contratto scritto nella firma è visibile all'implementazione, che può controllarlo; un effetto dichiarato permette al runtime di verificare che una funzione pura non stampi. Sono tutte funzionalità che un linguaggio può offrire solo se le conosce, e per questo Triad le definisce nello standard anziché lasciarle a convenzioni tra programmatori.

Da Julia arriva l'ossatura della sintassi: blocchi `function … end`, tipi con `::`, struct e multiple dispatch. Da Prolog arriva il motore logico: fatti, regole, unificazione con occurs-check, backtracking e `solve` con variabili logiche. Da Lisp arriva l'idea che il codice sia un albero uniforme, quindi un valore come gli altri, e la macchina delle macro che su quell'idea si appoggia. Tenere insieme i tre modelli è la parte propriamente nuova: non esiste un «linguaggio del lunedì» e un «linguaggio del martedì», esiste un solo programma con una sola semantica di espressioni e di blocchi.

1.1 Due profili: Core e Full

Lo standard definisce due profili di conformità (SPEC §16.1):

- **Triad-Core** (`SPEC-CORE.md`): interi, Bool, stringhe, simboli, `nothing`, liste, funzioni, lambda e chiusure, `if`/`elseif`/`else`, `while`, `let`, `begin`, ternario, `return`/`break`/`continue`, più alcuni primitivi di bootstrap. È abbastanza piccolo da essere implementato in codice macchina scritto a mano e abbastanza espressivo da scriverci l'interprete di tutto Triad.
- **Triad-Full** (`SPEC.md`): tutto il linguaggio.

La separazione risponde a un vincolo pratico, non a una scelta di comodo. Il primo interprete di Triad deve esistere prima che esista un compilatore Triad, quindi va scritto a mano in codice macchina: più il linguaggio è piccolo, più quel compito è realistico. Allo stesso tempo il sottoinsieme deve bastare a scrivere l'interprete completo nella seconda fase. Triad-Core è esattamente il punto di equilibrio tra le due esigenze, e per questo è definito da una specifica a sé.

Ogni programma Triad-Core (esclusi i primitivi di bootstrap) è anche un programma Triad-Full valido e deve comportarsi allo stesso modo. Il contrario non vale: un programma che usa `struct`, `for` o `float` è Full, e il runtime Core non lo eseguirà. È la distinzione che ritroverai in tutto il libro: la Parte II insegna il profilo che gira oggi, la Parte III aggiunge il resto.

1.2 La catena di bootstrap

Triad non vuole dipendere da nessun linguaggio ospite, ma ogni linguaggio deve pur partire da qualcosa. La catena di bootstrap risolve il problema in tappe: si comincia con un interprete scritto a mano in codice macchina, lo si usa per eseguire un interprete scritto in Triad, e così via fino a un compilatore che compila se stesso. Il piano (`BOOTSTRAP.md`):

```
S0  seed: interprete Triad-Core in codice macchina x86-64 (dump esadecimale nel repo)
S1  interprete di tutto Triad scritto in Triad-Core      (selfhost/, in corso)
S2  compilatore nativo scritto in Triad                  (non iniziato)
S3  compilatore self-hosted: S2 compilato da S2          (non iniziato)
S4  Triad completo: REPL, CLI, tooling, parità Linux     (non iniziato)
```

Il seed S0 non è distribuito come eseguibile ma come dump esadecimale: è il modo più semplice per versionare un binario in un repository testuale, e si materializza con lo strumento di decodifica del sistema operativo (capitolo 2). Gli stadi S1–S4 sono la direzione del progetto, non ciò che serve per leggere questo libro: per seguire il tutorial basta il seed S0 o l'interprete JS.

Gli strumenti in Node.js (`seed/dev/*.js`, `conformance/run.mjs`, `cli/`) sono **strumenti di sviluppo**: aiutano a generare e verificare, ma non fanno parte della catena S0→S4. La Parte IV del libro li presenta uno per uno, spiegando che cosa fanno e perché sono esterni al bootstrap.

---

2. Installare ed eseguire

«Installare» Triad qui non significa installare un pacchetto: significa avere il repository e uno dei due runtime disponibili. Non esiste ancora un compilatore da aggiungere al sistema, né un gestore di pacchetti del linguaggio. Il capitolo risponde quindi a due domande concrete: come si esegue un file e da quale runtime conviene partire.

La scelta consigliata per chi comincia è l'**interprete JS**: si avvia con Node.js, non richiede di decodificare binari ed è quello con cui vengono verificati i casi di conformità. Il seed Windows è interessante quando vuoi vedere il linguaggio eseguito senza runtime ospiti; i suoi limiti sono elencati nel capitolo 15. Per seguire il libro puoi usare solo l'interprete JS, e passare al seed quando un capitolo lo richiede.

2.1 Requisiti

- **Node.js ≥ 18** (la CI usa Node 20) per l'interprete JS, il runner di conformance e la CLI.
- Su Windows, nient'altro per il seed: `seed.exe` si ottiene decodificando `seed.hex` con `certutil`.
- Git per clonare il repository: `git clone https://github.com/GPuntobit/triad2.0.git`.

I requisiti sono pochi di proposito: Triad non vuole dipendenze da altri linguaggi per eseguire i programmi, e l'unica dipendenza dello strumentario di sviluppo è Node.js. Se hai Node.js, hai già tutto ciò che serve per eseguire gli esempi della Parte II.

2.2 Eseguire un file

Un programma Triad è un file di testo che il runtime legge, analizza ed esegue. La tabella mostra come avviarlo sulle due piattaforme e come passargli lo script da stdin:

piattaforma	comando
Linux / macOS	`node seed/dev/triad_core.cjs file.triad arg1 arg2`
Windows (seed)	`seed\windows-x86_64\seed.exe file.triad` oppure `bin\triad.cmd file.triad`
qualsiasi (stdin, interprete JS)	`echo 'println(7 * 6)' | node seed/dev/triad_core.cjs -`

Esempi pronti che girano oggi: tutti i `conformance/core/cases/*/program.td`, cioè i casi che il libro cita come riferimento normativo in fondo a ogni capitolo.

```
node seed/dev/triad_core.cjs conformance/core/cases/ricorsione/program.td   # 120, 55
```

```
bin\triad.cmd conformance\core\cases\ricorsione\program.td
```

Il commento `# 120, 55` non fa parte del comando: indica l'output atteso, cioè quello che dovresti vedere comparire. Il trattino `-` al posto del nome del file dice all'interprete JS di leggere il programma dallo standard input. Lo stesso file può essere eseguito dai due runtime, ma non sempre con lo stesso risultato: la differenza è l'argomento del capitolo 15.

2.3 Materializzare il seed

Il sorgente del seed è il dump esadecimale; l'eseguibile si ottiene con un decodificatore di sistema. Serve solo se vuoi provare il seed: l'interprete JS non richiede questo passaggio.

```
Remove-Item seed\windows-x86_64\seed.exe -ErrorAction SilentlyContinue
certutil -decodehex seed\windows-x86_64\seed.hex seed\windows-x86_64\seed.exe
```

```
# Linux: seed ELF allo stadio E0 (stampa solo "Triad seed Linux OK")
xxd -r -p seed/linux-x86_64/seed.hex seed/linux-x86_64/seed && chmod +x
seed/linux-x86_64/seed
```

Il comando Windows cancella prima l'eseguibile precedente, se c'è, e poi lo ricrea dal dump; su Linux il risultato è un ELF minimale, lo stadio E0, che per ora si limita a stampare un messaggio di avvio: non è ancora un interprete. Non è un errore di installazione, è lo stato del port descritto in `STATUS.md`.

2.4 Launcher `bin/triad` e `bin/triad.cmd`

Un launcher serve a non dover ricordare ogni volta quale binario usare e in quale cartella si trova. Non è obbligatorio, ma è comodo: quando `bin/` è nel `PATH` basta `triad file.triad`, come con un linguaggio installato nel sistema. La ricerca del runtime segue un ordine preciso, dal runtime migliore a quello disponibile:

1. `bin/triad-native` (eseguibile standalone, **non ancora prodotto**: arriverà con S2–S3);
2. `seed/<piattaforma>/seed` (su Windows `seed\windows-x86_64\seed.exe`).

Su Linux il launcher trova il seed ELF E0, che **non interpreta ancora programmi**: per eseguire codice su Linux usa direttamente l'interprete JS. `triad --build` è un residuo della vecchia implementazione Julia: stampa un messaggio che spiega perché la build non è più supportata ed esce con codice 2. Quando nessun runtime è disponibile, il launcher non fallisce in silenzio: stampa le istruzioni per materializzare il seed.

2.5 Codici di uscita

I codici di uscita sono parte del contratto del linguaggio, non un dettaglio di comodità: la suite di conformità confronta anche il codice, quindi un programma che stampa l'output giusto ma esce con il codice sbagliato non passa.

codice	significato
`0`	successo
`1`	errore di parsing o di esecuzione (messaggio su stderr), file non leggibile
`2`	uso non valido (es. nessun file)

Le opzioni di SPEC §14 (`-e`, `-E`, `-l`, `-i`, `--tree`, `--help`, REPL) **non sono ancora implementate**: l'interprete JS le tratta come un nome di file, e il seed Windows, che documenta solo l'uso `seed.exe FILE`, non le riconosce. Per questo, quando un esempio non funziona, la prima domanda da farsi è quale dei tre codici hai ottenuto: un `2` indica un problema nel comando, un `1` un problema nel programma.

2.6 La CLI di sviluppo in breve

La CLI è facoltativa: raccoglie in un solo comando le operazioni che altrimenti faresti a mano nel repository. Le righe seguenti mostrano le più usate, ciascuna con il commento che dice che cosa fa.

```
bin/triad-cli status                # ambiente e seed trovato (Windows:
bin\triad-cli.cmd)
bin/triad-cli test conformance      # conformance Core
bin/triad-cli ai verify -v          # verifica le soluzioni di ai/tasks/
bin/triad-cli seed generate windows # rigenera seed.hex e stampa lo SHA-256
node conformance/run.mjs            # runner di conformance diretto (Core)
```

I launcher `bin/triad-cli*` eseguono `npm install` in `cli/` al primo avvio. In alternativa si usa l'eseguibile standalone `triad-cli.exe` / `triad-cli` allegato alla release beta. Dettagli nel capitolo 39. Per seguire la Parte II la CLI non serve: gli esempi si eseguono direttamente con `node`.

---

3. Il primo programma

Un programma Triad non richiede cerimonie: non c'è un file di configurazione, non ci sono import da dichiarare, non c'è una funzione `main` da invocare. Il primo programma è un file di testo che contiene le istruzioni nell'ordine in cui devono essere eseguite.

Crea `ciao.triad`:

```
#!/usr/bin/env triad
# il mio primo programma Triad
nome = "Triad"
println("ciao, " * nome)
println(6 * 7)
```

Output:

```
ciao, Triad
42
```

Eseguilo:

```
node seed/dev/triad_core.cjs ciao.triad          # Linux/macOS
```

```
seed\windows-x86_64\seed.exe ciao.triad          # Windows
```

Cosa notare:

- la prima riga `#!` (shebang) è ignorata: inizia con `#`, quindi è un commento;
- `println` stampa i suoi argomenti **senza separatori** e aggiunge un a capo;
- `*` tra stringhe è la **concatenazione** (non `+`);
- non c'è una funzione `main`: le istruzioni al top level si eseguono in ordine.

Puoi seguire l'esecuzione riga per riga. La riga `nome = "Triad"` valuta la stringa e associa quel valore al nome `nome`. Nella riga seguente il runtime legge `nome`, concatena i due valori stringa con `*` e passa il risultato a `println`. Nell'ultima riga, `6 * 7` viene valutato prima della chiamata: a `println` arriva il valore intero `42`, non il testo del calcolo. Ogni istruzione viene eseguita una sola volta e nell'ordine in cui compare; una funzione chiamata può naturalmente eseguire il proprio corpo prima che il programma passi all'istruzione successiva.

La riga `#!` non è una direttiva necessaria per l'interprete: in questi esempi è una forma convenzionale utile quando il sistema operativo può avviare direttamente un file eseguibile. Il runtime Triad la ignora perché la tratta come commento. Se un esempio non funziona, verifica separatamente che il file sia salvato con estensione `.triad` o `.td`, che il comando punti al runtime giusto e che il sorgente usi soltanto costrutti supportati dal profilo dichiarato.

3.1 Argomenti da riga di comando

Gli argomenti dopo il nome del file sono in `ARGS`, una lista di stringhe (senza il nome del programma). Esempio con l'interprete JS:

```
printf 'println(ARGS)\nprintln(length(ARGS))\n' > args.triad
node seed/dev/triad_core.cjs args.triad uno due      # [uno, due]  poi  2
```

> *Il seed Windows documenta solo l'uso `seed.exe FILE`: `ARGS` non è tra le funzionalità elencate in `seed/windows-x86_64/README.md`. Usa l'interprete JS per i programmi con argomenti.*

3.2 Il ciclo di lavoro consigliato

1. scrivi il programma usando solo Triad-Core (Parte II);
2. eseguilo con l'interprete JS (`node seed/dev/triad_core.cjs`);
3. se deve girare anche sul seed Windows, controlla la tabella del capitolo 15;
4. se è un comportamento del linguaggio da fissare, trasformalo in un caso di conformance (capitolo 38).

Lo stesso ciclo vale per gli esercizi del capitolo 17: prima si scrive il programma, poi si confronta l'output con quello atteso, e solo alla fine, se serve, si guarda la soluzione.

Parte II — Triad-Core passo per passo (eseguibile oggi)

Ogni capitolo di questa parte usa solo Triad-Core. Gli esempi con **Output:** sono stati eseguiti con l'interprete JS; il caso di conformance indicato in fondo a ogni capitolo è il riferimento normativo da leggere e far girare.

La struttura dei capitoli è sempre la stessa, così sai sempre dove guardare: prima la spiegazione del costrutto e del problema che risolve, poi la sintassi e le regole, quindi un esempio completo con il suo output, e infine il caso di conformance da cui l'esempio deriva. Quando una regola nasconde una trappola, il testo la segnala esplicitamente.

4. Lessico

Il lessico è l'insieme dei mattoni con cui si scrive un programma: che cosa è un nome, come si scrive un numero o una stringa, che cosa il runtime considera separatore tra un'istruzione e l'altra. Non è un elenco da imparare a memoria: serve a capire perché un certo testo viene letto in un modo e non in un altro, e quindi perché un programma che sembra giusto produce un `parse error`.

La tabella raccoglie gli elementi di Triad-Core, con la forma ammessa e un esempio per ciascuno:

elemento	forma	esempi
identificatore	lettera o `_`, poi lettere/cifre/`_`; un `!` finale è ammesso	`x`, `somma_2`, `push!`
intero	cifre decimali, `_` come separatore; il segno è l'operatore unario	`42`, `1_000_000`, `-5`
booleano	`true`, `false`	
nothing	`nothing`	
stringa	`"..."` con escape `\n \t \" \\`; **nessuna interpolazione**	`"ciao\n"`
simbolo	`:` seguito da un nome o da un operatore	`:alice`, `:+`
commento	`#` fino a fine riga; `#= ... =#` su più righe	

elemento	forma	esempi
separatore	a capo oppure `;`	`println(1);` `println(2)`

Operatori di Core: `+ - * % ^ == != < <= > >= && || ! = += -= *= -> ?:`.

Parole riservate usate da Core: `function end if elseif else while let begin return break continue true false nothing`.

Non fanno parte di Core: letterali `Float` (`1.5`), variabili logiche `?x`, macro `@nome`, interpolazione `$x`, l'operatore `/`.

L'operatore `->` costruisce una lambda, mentre `?` e `:` formano il condizionale: sono token del lessico, ma il loro significato è argomento del capitolo 7 e del capitolo 10. Le parole riservate non possono essere usate come nomi di variabile: il parser le riconosce come costruzioni, e un assegnamento come `let = 3` produce un errore di sintassi. L'elenco delle parole riservate è breve perché Core è un linguaggio piccolo: tutto ciò che non figura qui appartiene a Triad-Full, e usarlo in un programma Core non è «codice eseguibile oggi».

Un'altra distinzione da tenere a mente è quella tra spazi e separatori. Spazi e tabulazioni separano i token ma non le istruzioni; il punto e virgola e il ritorno a capo sì. Per questo `println(1); println(2)` mette due istruzioni sulla stessa riga, mentre una riga vuota non produce un'istruzione. Il punto e virgola, insomma, ha lo stesso ruolo del ritorno a capo: è un separatore, non una fine di blocco.

```
#= commento
   su più righe =#
x = 1_000   # commento di riga
println(x); println("tab:\t|", " virgolette: \"", " backslash: \\")
println(:alice, " ", :+)
```

Output:

```
1000
tab:    | virgolette: " backslash: \
alice +
```

> ***Attenzione.*** *`"x = $x"` in Core stampa letteralmente `x = $x`: l'interpolazione è solo Triad-Full. In Core si concatena: `"x = " * string(x)`.*
>
> *I commenti `#= … =#` **non si annidano** nell'interprete JS: il commento si chiude al primo `=#` incontrato. Non contare sull'annidamento; se devi commentare codice che contiene già un blocco `#= … =#`, usa commenti di riga.*

I due avvertimenti hanno la stessa radice: in Core il lessico è volutamente minimo, e ciò che in un altro linguaggio è «solo un dettaglio» qui può non esistere. Il testo di un programma Core non va interpretato con le abitudini di un altro linguaggio: se un escape o un simbolo non compare nella tabella, il runtime non lo riconosce.

Riferimento: `conformance/core/cases/aritmetica`, `simboli_base`, `stringhe_base`.

---

5. Valori, tipi e stampa

Triad è un linguaggio a tipi dinamici: ogni valore ha un tipo ben preciso, ma le variabili no. Una stessa variabile può contenere un intero e poi una stringa, e il tipo viene controllato al momento dell'operazione, non della dichiarazione. Da qui deriva una conseguenza pratica: per sapere che cosa contiene un valore devi chiederlo al runtime con `typeof`, e per scrivere codice robusto devi pensare ai tipi che possono arrivare, non solo a quelli che intendevi passare.

La tabella elenca i tipi che un programma Core può incontrare:

tipo	valori	`typeof(v)`
`Int`	interi con segno a 64 bit, aritmetica con wrap (complemento a due)	`Int`
`Bool`	`true`, `false`	`Bool`
`String`	sequenza immutabile di byte UTF-8	`String`
`Symbol`	`:nome`; uguali se hanno lo stesso nome	`Symbol`
`List`	lista eterogenea **mutabile**, indici da 1	`List`
`Function`	funzione nominata, lambda o chiusura	`Function`
`Nothing`	`nothing`	`Nothing`

`typeof` restituisce il **nome del tipo come simbolo**.

Il dettaglio del simbolo non è pedanteria: `typeof(1)` vale `:Int`, non `"Int"`. Confrontare il risultato con una stringa non funziona, e il confronto corretto è `typeof(v) == :Int`. In generale, in Triad un tipo si nomina con un simbolo; la stringa serve a chi stampa un messaggio, non a chi programma un controllo.

I tipi non sono un ornamento: da essi dipende che cosa un'operazione accetta. `Int` è a 64 bit perché deve stare in una parola di macchina, che è l'unità con cui il seed lavora; `String` è immutabile perché il linguaggio non offre modi per modificarla in loco; `Symbol` è pensato per i nomi e gli identificatori interni; `List` è mutabile perché è la struttura dati di lavoro del linguaggio; `Nothing` rappresenta l'assenza di un valore utile, non un errore.

5.1 Regole di stampa di `print` / `println`

- concatenano gli argomenti senza separatori; `println` aggiunge `\n`;
- `Int` in decimale, `Bool` come `true` / `false`, `String` così com'è, `Nothing` come `nothing`;
- un `Symbol` passato direttamente è stampato **senza** `:`; dentro una lista **con** `:`;
- una `List` è stampata come `[a, b]`; le stringhe dentro una lista sono stampate senza virgolette.

```
xs = [1, :a, "s", true, nothing, [2, 3]]
println(xs)
println(:alice)
println(typeof(1), " ", typeof("s"), " ", typeof(:a), " ", typeof(xs))
println(typeof(nothing), " ", typeof(true), " ", typeof(x -> x))
print("senza ", "a capo")
println()
```

Output:

```
[1, :a, s, true, nothing, [2, 3]]
alice
Int String Symbol List
Nothing Bool Function
senza a capo
```

5.2 Uguaglianza

`==` e `!=` confrontano numeri, stringhe e simboli **per valore**, le liste **elemento per elemento** (anche annidate), le funzioni **per identità**.

```
println([1, [2, [3]]] == [1, [2, [3]]])
println("ab" == "a" * "b", " ", :a == :a, " ", 1 == 2)
```

Output:

```
true
true true false
```

Il valore stampato non è sempre una rappresentazione da riutilizzare come sorgente. Per esempio, un simbolo passato direttamente a `println` appare senza i due punti (`alice`), mentre dentro una lista viene mostrato come `:alice`; le stringhe dentro una lista sono mostrate senza virgolette. `typeof` è quindi il modo corretto per verificare il tipo del valore, mentre la forma visiva di `println` serve soprattutto alla lettura umana. Analogamente, `==` confronta il contenuto per numeri, stringhe, simboli e liste, ma le funzioni per identità: due funzioni scritte con lo stesso testo non sono per questo lo stesso oggetto funzione.

La rappresentazione UTF-8 è misurata in byte, non necessariamente in lettere visibili. Una lettera accentata può occupare più byte; perciò `length` su una stringa non è sempre il conteggio dei caratteri percepiti da una persona. In Core non ci sono operazioni di indicizzazione dei caratteri: i primitivi di

bootstrap lavorano esplicitamente sui byte e vanno usati solo per il compito di lettura e analisi del sorgente.

Riferimento: `conformance/core/cases/valori`, `nothing_base`, `confronti`.

---

6. Variabili e assegnamento

Una variabile, in Triad, è un nome legato a un valore. Non la dichiari e non ne annunci il tipo: il primo assegnamento crea il legame, i successivi lo aggiornano. Questo rende il codice breve, ma sposta l'attenzione su due domande che in un linguaggio con le dichiarazioni sarebbero automatiche: quale valore c'è dentro un nome in questo punto del programma, e in quale ambiente quel nome è visibile.

Le regole di base sono queste:

- `nome = espr` lega o aggiorna un nome; l'assegnamento è un'espressione e vale il valore assegnato.
- `+=`, `-=`, `*=` aggiornano una variabile esistente (il lato sinistro è valutato una volta).
- Usare un nome mai assegnato è un errore di esecuzione (`undefined variable`).
- Non ci sono dichiarazioni né tipi: le variabili sono dinamiche.

Che l'assegnamento sia un'espressione, e non un'istruzione, è la regola meno ovvia. Significa che `x = 42` non «fa qualcosa e basta»: produce il valore `42`, e quel valore può essere usato dove compare l'assegnamento. Nella pratica serve quando un assegnamento compare dentro un blocco che restituisce un valore, ma è anche il motivo per cui non trovi una distinzione rigida tra «istruzione» e «espressione» nel resto del libro. L'assegnamento composto (`+=`, `-=`, `*=`) è zucchero per una lettura e una scrittura della stessa variabile, con la garanzia che il lato sinistro venga valutato una sola volta: se contiene un indice, quell'indice non viene calcolato due volte.

Un nome usato prima di essere assegnato non vale `nothing` e non vale zero: è un errore che interrompe il programma. È una scelta voluta, perché un valore «vuoto» implicito nasconderebbe un errore di logica; il capitolo 14 spiega come si riconosce questo caso e che cosa si vede sul terminale.

```
x = 42
y = x + 8
x = x + 1
z = (x + y) * 2
println(x, " ", y, " ", z)
n = 10
n += 5
n -= 3
n *= 2
println(n)
```

Output:

```
43 50 186
24
```

Le regole di **visibilità** (dentro funzioni, `let`, lambda) sono nel capitolo 10.

Un modo utile di seguire il programma è tenere una piccola tabella dei legami correnti:

istruzione conclusa	`x`	`y`	`z`
`x = 42`	42	—	—
`y = x + 8`	42	50	—

istruzione conclusa	`x`	`y`	`z`
`x = x + 1`	43	50	—
`z = (x + y) * 2`	43	50	186

Questa tabella mostra perché l'assegnamento non è una formula reattiva: `y` conserva il risultato calcolato quando la sua riga viene eseguita, anche dopo che `x` cambia. Per contro, una lista è mutabile e può essere raggiunta da più nomi; modificare un suo elemento agisce sull'oggetto condiviso, non solo su una singola associazione nome-valore. Il capitolo 11 mostra come riconoscere questa differenza nel codice.

Riferimento: `conformance/core/cases/variabili`.

---

7. Operatori ed espressioni

Un'espressione è un modo di combinare valori per produrne un altro. Triad non ha una distinzione rigida tra espressioni e istruzioni: quasi tutto è un'espressione, e anche i costrutti di controllo hanno un valore. Per leggere un'espressione senza ambiguità servono due informazioni: quanto «lega forte» ogni operatore e in che direzione si raggruppano gli operatori dello stesso livello.

7.1 Precedenza (dalla più bassa alla più alta)

livello	operatori	associatività
1	`=` `+=` `-=` `*=`	destra
2	`c ? a : b`	destra
3	`||`	sinistra, cortocircuito
4	`&&`	sinistra, cortocircuito
5	`==` `!=` `<` `<=` `>` `>=`	sinistra
6	`+` `-`	sinistra
7	`*` `%`	sinistra
8	`^`	**destra**
9	unari `-` `!`	prefissi
10	`f(args)` `xs[i]`	postfissi

La tabella va letta come una classifica: il livello 1 lega meno, il livello 10 di più. In `2 + 3 * 4` convivono il livello 6 (`+`) e il livello 7 (`*`); vince il più alto, quindi si calcola prima `3 * 4`. Quando due operatori stanno sullo stesso livello decide l'associatività: `-` è a sinistra, quindi `8 - 3 - 2` è `(8 - 3) - 2` e non `8 - (3 - 2)`. L'eccezione più importante è `^`, che è a destra: `2 ^ 3 ^ 2` significa `2 ^ (3 ^ 2)` e vale 512, non 64.

Questa tabella descrive Triad-Core. Triad-Full aggiunge altri operatori e altri livelli — il range `a:b`, `in`/`isa`/`is`, l'asserzione di tipo `e::T`, la lambda `x -> e`

— descritti nella specifica (SPEC §2). Chi scrive solo Core può ragionare con questa tabella; chi passa a Full deve consultare quella completa.

7.2 Aritmetica intera

- `+ - * ^` su `Int` con **wrap a 64 bit**;
- `%` è il **resto con il segno del dividendo** (come `rem` di Julia): `-7 % 3 == -1`;
- `^` con esponente negativo è un errore; `%` per zero è un errore;
- **non esiste** `/` in Core (la divisione è di Triad-Full).

```
println(2 + 3 * 4, " ", (2 + 3) * 4, " ", 2 ^ 3 ^ 2)
println(7 % 3, " ", -7 % 3, " ", 10 % 4)
println(9223372036854775807 + 1)
```

Output:

```
14 20 512
1 -1 2
-9223372036854775808
```

Il risultato `-9223372036854775808` è la faccia visibile del wrap: il massimo intero più uno non produce un errore, produce il minimo. Triad-Core non ha un tipo «intero grande» né controlli di overflow, perché deve restare implementabile con poche istruzioni di macchina; se un calcolo deve restare nel dominio, il controllo va scritto nel programma.

Il resto negativo segue la stessa logica: `-7 % 3` vale `-1` perché il segno lo dà il dividendo, non il divisore. È il comportamento di `rem` in Julia e va tenuto presente quando si usa `%` per calcolare posizioni cicliche: con valori negativi serve una correzione esplicita.

7.3 Logica e condizioni

- Le condizioni (`if`, `while`, ternario, `&&`, `||`, `!`) accettano **solo** `Bool`: non c'è "truthiness" (`if 1` è un errore).
- `&&` e `||` cortocircuitano.
- I confronti non si concatenano: `1 < 2 < 3` è un errore (si confronta `Bool < Int`); scrivi `1 < 2 && 2 < 3`.

```
println(!false, " ", true && false, " ", false || true)
println(1 < 2 && 2 < 3)
x = 3
println(x > 2 ? "grande" : "piccolo")
```

Output:

```
true false true
true
grande
```

Il **cortocircuito** è osservabile: con `a && b`, se `a` è `false`, `b` non viene valutato; con `a || b`, se `a` è `true`, `b` non viene valutato. Non è solo un'ottimizzazione: impedisce anche che un calcolo non necessario produca un errore. Un controllo del tipo `i <= length(xs) && xs[i] > 0` può quindi verificare il limite prima di accedere alla lista, purché `i` sia un intero valido. Entrambi gli operandi valutati devono comunque essere booleani.

La scelta di accettare solo `Bool` evita una classe intera di ambiguità: in molti linguaggi `0`, `""` e `nothing` sono «falsi» per convenzione, e ogni

programmatore arriva con una convenzione diversa. In Triad non ci sono convenzioni da ricordare: una condizione è vera o falsa, e qualsiasi altro tipo è un errore. Per questo `1 < 2 < 3` non funziona: i confronti si associano a sinistra, quindi il parser legge `(1 < 2) < 3`, cioè un `Bool` confrontato con un `Int`.

Una trappola collegata riguarda `=` e `==`: `if x = 0` non confronta `x` con zero, assegna zero e produce `0` come valore della condizione; il runtime rifiuta quel valore perché non è un `Bool`. Il confronto è `==`, e vale la pena prendere l'abitudine di scriverlo sempre.

La tabella di precedenza descrive il raggruppamento: `a + b * c` equivale a `a + (b * c)`. Quando due operatori hanno lo stesso livello, entra in gioco l'associatività. In caso di dubbio, usa parentesi: rendono esplicita la struttura che vuoi e aiutano a distinguere un errore di raggruppamento da un errore nel valore degli operandi.

Riferimento: `conformance/core/cases/aritmetica`, `operatori`, `interi`, `confronti`.

---

8. Controllo di flusso

Il controllo di flusso decide quali istruzioni eseguire e quante volte. Triad ha un vincolo che semplifica il modello mentale: **tutto è un'espressione**, e anche `if`, `while`, `let` e `begin` hanno un valore (quello dell'ultima istruzione eseguita, `nothing` se non ne esegue nessuna). Non esistono quindi due mondi separati — istruzioni che non valgono niente ed espressioni che valgono qualcosa — ma un solo meccanismo, e i blocchi sono espressioni che contengono altre espressioni.

8.1 `if` / `elseif` / `else`

```
function classifica(x)
    if x < 0
        "negativo"
    elseif x == 0
        "zero"
    else
        "positivo"
    end
end
println(classifica(-3), " ", classifica(0), " ", classifica(7))
v = if 2 > 1 :si else :no end
println(v)
```

Output:

```
negativo zero positivo
si
```

L'`if` valuta una condizione alla volta, dalla prima all'ultima, ed esegue solo il ramo corrispondente al primo valore `true`; se nessuna condizione è vera ed esiste un `else`, esegue quello, altrimenti non esegue niente e vale `nothing`. `elseif` non è uno zucchero per un `else` che contiene un altro `if`: è un ramo della stessa catena, e questo evita di accumulare `end` a ogni livello. L'ultima riga dell'esempio mostra che un `if` può stare a destra di un assegnamento: il suo valore è quello del ramo scelto, qui il simbolo `:si`.

Due trappole ricorrenti: dimenticare l'`end` (che chiude `if`, `while`, `function` e `let`) e scrivere `else if` al posto di `elseif`: in Core `elseif` è un unico ramo della catena, mentre `else if` annida un secondo `if` e richiede un `end` in più. Il `? :` del capitolo 7 è la versione compatta dello stesso schema, comoda quando i due rami sono espressioni brevissime.

8.2 `while`, `break`, `continue`

```
i = 0
somma_dispari = 0
while i < 10
    i += 1
    if i % 2 == 0
        continue
    end
    somma_dispari += i
end
println(somma_dispari)
while true
    break
end
println(:fine)
```

Output:

```
25
fine
```

Il `while` è l'unico ciclo di Triad-Core: non c'è `for`, e l'iterazione su una lista si scrive con un contatore e un limite, come farà il capitolo 11. La scelta è coerente con il bootstrap: un solo costrutto di ciclo è più facile da implementare in codice macchina, e `for` può essere aggiunto dopo, in S1, senza cambiare la semantica del `while`.

La trappola classica del `while` è l'ordine delle istruzioni dentro il corpo: se `continue` salta l'aggiornamento del contatore, il ciclo non termina.

Nell'esempio il contatore viene incrementato prima di ogni `continue`, e non è un caso: la condizione di uscita deve essere raggiungibile lungo tutti i percorsi del corpo. `break` e `continue` agiscono sempre sul `while` più interno, quindi in cicli annidati vanno letti con attenzione.

8.3 `let` e `begin`

`let` introduce un nuovo scope con legami locali; `begin … end` raggruppa istruzioni senza creare legami nuovi.

```
x = 1
let x = 10
    let x = 20
        println(x)
    end
    println(x)
end
println(x)
r = begin
    a = 2
    a * 21
end
println(r)
```

Output:

```
20
10
1
42
```

Un blocco `begin … end` raggruppa istruzioni e vale l'ultima: serve a usare più istruzioni dove ne serve una sola, per esempio a destra di un assegnamento. `let` invece apre un ambiente nuovo, e i nomi che lega lì dentro possono nascondere quelli esterni senza toccarli: è il meccanismo con cui Triad-Core

isola un calcolo. L'esempio mostra i tre legami di `x` (`20`, `10`, `1`) e poi il valore del `begin` (`42`), che arriva all'assegnamento come qualsiasi altro valore.

La differenza tra i due costrutti è la stessa che c'è tra «raggruppare» e «isolare»: usa `begin` quando vuoi solo tenere insieme istruzioni, `let` quando un nome locale deve restare locale.

Riferimento: `conformance/core/cases/controllo`, `if_else`, `if_elseif`, `if_nested`, `if_no_else`, `while_loop`, `let_annidato`, `begin_block`.

---

9. Funzioni

Una funzione è un pezzo di programma a cui dai un nome, con dei parametri, che restituisce un valore. È il mattone con cui si costruisce tutto il resto: in Triad-Core non ci sono struct né metodi, quindi la funzione è anche l'unico modo per dare un nome a un'operazione e riusarla.

9.1 Tre modi di definire una funzione

```
# forma breve: il corpo è un'espressione
quadrato(x) = x * x

# forma blocco: il valore è quello dell'ultima espressione
function calcola(n)
    m = n + 1
    m * 2
end

# funzione anonima assegnata a un nome
moltiplica = function (a, b)
    a * b
end

println(quadrato(4), " ", calcola(3), " ", moltiplica(6, 7))
```

Output:

```
16 8 42
```

Quando `calcola(3)` viene chiamata, `n` riceve `3`; il corpo calcola `m = 4`, poi valuta `m * 2` e restituisce `8` perché è l'ultima espressione. Il legame `m` appartiene alla chiamata e non diventa automaticamente una variabile globale. Con la forma breve, l'espressione a destra di `=` è direttamente il corpo. La funzione anonima può essere assegnata a un nome, passata come argomento o restituita da un'altra funzione: il nome è un riferimento utile, non una proprietà necessaria del valore funzione.

9.2 `return`, ricorsione, arità

- `return e` esce subito dalla funzione; `return` da solo restituisce `nothing`.
- Una funzione nominata è visibile nel proprio corpo: la ricorsione diretta funziona.
- Il numero di argomenti deve coincidere con i parametri (altrimenti `arity mismatch`).

```
function taglia(n)
    if n > 100
        return :grande
    end
    :piccola
end
fattoriale(n) = n < 2 ? 1 : n * fattoriale(n - 1)
fib(n) = n < 2 ? n : fib(n - 1) + fib(n - 2)
mcd(a, b) = b == 0 ? a : mcd(b, a % b)
println(taglia(500), " ", taglia(5))
println(fattoriale(10), " ", fib(15), " ", mcd(48, 18))
```

Output:

```
grande piccola
3628800 610 6
```

La ricorsione si comprende seguendo una catena di chiamate, ciascuna con il proprio parametro. In `fattoriale(4)`, per esempio, la chiamata riduce `n` a 3, 2 e 1; arrivata al caso base restituisce 1, poi le chiamate sospese riprendono e moltiplicano i risultati per 2, 3 e 4. Il caso base è quindi una condizione di

terminazione, non un semplice esempio aggiuntivo. Se il parametro non si avvicina a un caso base, la funzione continua a chiamarsi; nel runtime Core il budget di esecuzione non è un costrutto del linguaggio, quindi è responsabilità del programma evitare ricorsioni illimitate.

Senza `return`, il valore della funzione è l'ultima espressione eseguita, anche quando il corpo termina dentro un `if`. Una funzione è un valore legato a un nome quando l'istruzione di definizione viene eseguita: per questo chiamarla prima della sua riga di definizione produce `undefined variable`, non una risoluzione anticipata. Non ci sono argomenti di default né variadici: l'arità deve coincidere esattamente con i parametri.

9.3 Annotazione di effetto `!io`

In Triad-Full una funzione che stampa deve dichiarare `!io` (capitolo 28). Core accetta l'annotazione e la ignora: scriverla rende il codice già pronto per Full.

```
function mostra() !io
    println("hello from block")
    20 + 22
end
println(mostra())
```

Output:

```
hello from block
42
```

L'annotazione fa parte della firma in Triad-Full: lì una funzione che stampa senza `!io` viola il controllo degli effetti e il chiamante ottiene `Err(:effect)` (capitolo 28). In Core il controllo non esiste, quindi l'annotazione viene accettata e ignorata: il programma gira, ma non hai alcuna garanzia in più. Scriverla fin da subito resta comunque utile, perché il codice sarà valido quando passerà a Full e perché documenta l'intenzione di chi l'ha scritto.

Riferimento: `conformance/core/cases/funzioni`, `funzioni_semplici`, `funzioni_blocco`, `funzioni_io`, `ricorsione`, `gcd`, `formula`.

---

10. Lambda, chiusure e scope

Una lambda è una funzione che non ha bisogno di un nome: la scrivi dove serve, e il suo valore è la funzione stessa. Serve ogni volta che vuoi passare un comportamento a un'altra funzione — «applica questa trasformazione a ogni elemento», «chiama questo controllo su ogni valore» — senza inventare un nome che userai una volta sola. Una chiusura è una lambda che ricorda l'ambiente in cui è nata; è il meccanismo che rende possibile costruire funzioni parametriche senza ricorrere a variabili globali.

10.1 Lambda

```
doppio = x -> x * 2
somma = (a, b) -> a + b
applica = f -> f(3)
println(doppio(5), " ", somma(1, 2), " ", applica(x -> x * x))
```

Output:

```
10 3 9
```

Una lambda con corpo a più istruzioni si scrive con `begin … end` o con `function (…) … end`. Le lambda senza parametri (`() -> …`) **non** sono supportate: usa `function () … end`.

10.2 Chiusure

Una lambda cattura l'ambiente in cui è creata. Secondo SPEC-CORE §5 le variabili catturate sono **condivise**: le mutazioni sono visibili.

```
function add(n)
    x -> x + n
end
f = add(10)
g = add(100)
println(f(5), " ", g(5))

function contatore()
    v = 0
    x -> begin
        v += x
        v
    end
end
acc = contatore()
acc(5)
println(acc(10))
```

Output:

```
15 105
15
```

> *Il seed Windows cattura **per valore** e copre solo l'ultimo legame del frame: un accumulatore come `contatore` può non funzionare sul seed*

> *(capitolo 15).*

La differenza tra una funzione ordinaria e una chiusura è il contesto esterno che resta accessibile. `add(10)` termina, ma la lambda restituita conserva il legame `n = 10`; un'altra chiamata ad `add` crea un ambiente distinto. Nel contatore, invece, la chiusura non si limita a leggere `v`: aggiorna il legame condiviso. L'interprete JS segue il contratto Core in cui questa mutazione è visibile alle chiamate successive. Il seed Windows ha una cattura più limitata, quindi un programma che dipende da questo caso va provato con l'interprete di riferimento e non inferito dal solo fatto che il seed accetti la sintassi.

10.3 Regole di scope

1. `function`, lambda, `let` e `while` introducono ambienti; un nome è visibile nell'ambiente in cui è legato e in quelli interni (scope lessicale).
2. **Dentro una funzione nominata** l'assegnamento lega nel frame locale: una funzione **non modifica i globali**.
3. **Dentro una lambda** l'assegnamento a un nome già legato fuori aggiorna quel legame (è così che funziona l'accumulatore sopra).
4. **Al top level** i cicli aggiornano i globali.
5. `let x = …` crea un nuovo `x`; assegnare `x` dentro il `let` non tocca l'`x` esterno.

```
x = 1
function f()
    x = 2
    x
end
println(f(), " ", x)
c = 0
function incrementa()
    c += 1
end
incrementa()
println(c)
```

Output:

```
2 1
0
```

Il punto delicato è che le regole 2 e 3 sembrano in contraddizione: una funzione nominata non modifica un globale, una lambda sì. La differenza non è capricciosa, dipende da come vengono catturati gli ambienti. La lambda porta con sé il riferimento all'ambiente in cui è stata creata, quindi l'assegnamento trova il legame esterno e lo aggiorna. La funzione nominata è pensata per essere richiamabile da sola, senza effetti nascosti sui nomi dello scope globale: l'assegnamento crea un legame locale, e il globale resta quello che era. L'esempio lo mostra con `x` (che resta `1`) e con `c` (che resta `0` anche dopo `incrementa()`).

Se ti serve comunicare un risultato, ci sono due strade coerenti con questo modello: restituire il valore con `return`, oppure mettere lo stato in una lista, che è mutabile e quindi condivisa anche da una funzione nominata (capitolo 11).

Riferimento: `conformance/core/cases/lambda`, `closure`, `chiusure`, `chiusure_local`, `chiusura_in_if`, `let_chiusura`, `scope`.

---

11. Liste

Una lista è una sequenza ordinata di valori, raggiungibili per posizione. In Triad-Core è la struttura dati di lavoro: non ci sono struct né dizionari, quindi liste, liste di liste e liste taggate sono il modo in cui si rappresentano record, ambienti e alberi. Capire come si comporta una lista — che è mutabile e che gli indici partono da 1 — è il passaggio che apre tutta la programmazione non banale in Core.

Le operazioni di base sono poche, e vale la pena leggerle pensando a che cosa fanno alla lista come oggetto, non solo al valore che restituiscono:

- Letterale: `[1, 2, 3]`, `[]`, eterogenee e annidabili.
- Indici **da 1**: `xs[1]` è il primo elemento; fuori range è un errore.
- Assegnamento per indice: `xs[i] = v`, anche `xs[i] += v`.
- `length(xs)` numero di elementi; `push!(xs, v)` aggiunge in coda, **muta** la lista e la restituisce.
- Le liste sono riferimenti: due nomi possono indicare la stessa lista.

```
xs = [1, 2, 3]
println(xs[1], " ", length(xs))
push!(xs, 4)
xs[1] += 100
println(xs)
alias = xs
push!(alias, 5)
println(xs)
tot = 0
i = 1
```

```
while i <= length(xs)
    tot += xs[i]
    i += 1
end
println(tot)
```

Output:

```
1 3
[101, 2, 3, 4]
[101, 2, 3, 4, 5]
115
```

Core non ha `map`, `filter`, `reduce`, `sum`, `sort`: si scrivono in Core (vedi capitolo 16).

> *Sul seed Windows: liste solo di interi, `push!` sì, `xs[i]` e `length(xs)` **no**; c'è invece l'helper non standard `get(xs, i)` con indici **da 0** (capitolo 15).*

Riferimento: `conformance/core/cases/liste`, `liste_base`, `liste_push`.

Ricorda il modello di alias: dopo `alias = xs`, i nomi non identificano due copie, ma la stessa lista. Cambiare `xs[1]` o aggiungere un elemento attraverso `alias` modifica un oggetto osservato da entrambi. Se hai bisogno di un valore indipendente, costruisci esplicitamente una nuova lista e copi gli elementi con un ciclo. Questo comportamento è diverso da una variabile intera: assegnare `y = x` copia il valore numerico e una successiva riassegnazione di `x` non aggiorna `y`.

Negli algoritmi con indici, la coppia `i = 1` e `i <= length(xs)` esprime l'intervallo completo delle posizioni Core. Prima di leggere `xs[i]`, assicurati che `i` sia dentro quell'intervallo; un indice fuori limite è un errore, non un elemento `nothing`. Il caso di conformance `liste` verifica proprio le operazioni standard, mentre l'helper `get` del seed ha indici 0-based e non deve essere confuso con `xs[i]`.

---

12. Stringhe, simboli e conversioni

Stringhe e simboli sembrano parenti stretti — entrambi contengono testo — ma hanno ruoli diversi. Una stringa è un dato che si costruisce, si concatena e si stampa; un simbolo è un nome che il linguaggio tratta come valore atomico, adatto a etichette, chiavi, tag di record e nomi di tipo. Le conversioni tra i due, e tra testo e numeri, sono il ponte che serve quando un programma legge input, costruisce messaggi o confronta valori che arrivano come testo.

La tabella riassume le operazioni disponibili in Core:

operazione	significato
`a * b`	concatenazione di stringhe
`length(s)`	numero di **byte** UTF-8 (`length("é") == 2`)
`string(x)`	rappresentazione testuale di qualsiasi valore (`string(:a) == "a"`, `string([1, 2]) == "[1, 2]"`)
`Int(s)`	da stringa a intero (`Int("42") == 42`); da Bool: `0`/`1`
`Symbol(s)`	da stringa a simbolo (`Symbol("ciao") == :ciao`)
`typeof(x)`	tipo come simbolo

```
nome = "Triad"
saluto = "ciao " * nome
println(saluto, " (", length(saluto), " byte)")
println("risposta: " * string(40 + 2))
```

```
println(Int("41") + 1, " ", Symbol("bob"), " ", string(true))
println(:ok == Symbol("ok"))
```

Output:

```
 ciao Triad (10 byte)
risposta: 42
42 bob true
true
```

Per accedere ai singoli byte di una stringa servono i primitivi di bootstrap del capitolo successivo.

Tre cose meritano attenzione. Primo, le stringhe sono **immutabili**: `saluto = "ciao " * nome` non modifica `"ciao "`, costruisce una nuova stringa. Questo è coerente con il resto del linguaggio, dove solo le liste sono mutabili, e rende innocuo passare una stringa a una funzione: chi la riceve non può alterarla. Secondo, la concatenazione usa `*` e non `+`: `+` resta l'addizione numerica, e la scelta evita di dover indovinare il tipo degli operandi in un'espressione come `a + b`. Terzo, `length` conta i **byte**, non i caratteri percepiti: una lettera accentata può occupare due byte, quindi in Core «lunghezza di una stringa» significa «dimensione in byte UTF-8», e le operazioni sui singoli caratteri si fanno a livello di byte con i primitivi del capitolo 13.

`string(x)` è una rappresentazione pensata per la lettura, non una promessa che il risultato si possa rileggere come sorgente: `string(:a)` produce `"a"` (senza i due punti), e la stampa di una lista non è necessariamente codice Triad valido. Per i messaggi va benissimo; per ricostruire un valore servono le conversioni esplicite. Anche la conversione non è una validazione:

`Int("42")` presuppone che il testo sia nel formato giusto, e che cosa faccia `Int` su un testo che non è un numero **non è specificato**; il programma deve controllare prima, non sperare in un risultato utile.

L'ultima trappola riguarda i simboli. `println(:ok)` stampa `ok`, ma il valore è ancora un simbolo: `:ok == "ok"` è `false`, e il confronto corretto è con `:ok` oppure con `Symbol("ok")`. L'aspetto stampato di un valore non è il valore.

Riferimento: `conformance/core/cases/stringhe`, `stringhe_concat`, `stringhe_multi`, `simboli_base`.

---

13. Primitivi di bootstrap

Servono a S1 per leggere e analizzare sorgente Triad. **Non** fanno parte di Triad-Full (SPEC-CORE §8): usali solo nel codice di bootstrap.

Il nome «bootstrap» spiega la loro esistenza. Per scrivere in Triad un interprete di Triad serve la capacità di leggere un file, di guardare i singoli byte di quel file e di estrarne pezzi: sono operazioni che il linguaggio non offre con i tipi normali, perché le stringhe sono immutabili e non indicizzabili. I primitivi colmano questa lacuna al livello più basso possibile, quello dei byte, e restano fuori dal linguaggio normale perché una libreria di stringhe completa arriverà con S1, scritta in Triad.

La tabella elenca i cinque primitivi e la loro semantica:

primitiva	semantica
`readfile(path)`	contenuto del file come `String`; errore se non leggibile
`strlen(s)`	numero di byte
`strbyte(s, i)`	byte (`Int` 0–255) alla posizione `i`, da 1
`strsub(s, a, b)`	sottostringa dai byte `a` a `b` inclusi, da 1
`exit(n)`	termina il processo con codice `n`

Tutte le posizioni sono **1-based**, come gli indici delle liste, e `strsub` include entrambi gli estremi: `strsub(s, 1, 3)` restituisce i primi tre byte. `strbyte` restituisce un intero tra 0 e 255, non un carattere: il programma deve sapere quali codici gli interessano. Un indice fuori intervallo è un errore di

esecuzione, non un byte zero; per questo chi analizza un sorgente controlla sempre i limiti prima di leggere.

Esempio: contare le cifre di una stringa e separare la prima parola.

```
s = "abc 123 xyz"
cifre = 0
i = 1
while i <= strlen(s)
    b = strbyte(s, i)
    if b >= 48 && b <= 57
        cifre += 1
    end
    i += 1
end
println(cifre)
fine = 1
while fine <= strlen(s) && strbyte(s, fine) != 32
    fine += 1
end
println(strsub(s, 1, fine - 1))
```

Output:

```
3
abc
```

Il primo ciclo conta le cifre confrontando il byte con i codici ASCII `48`–`57`: `strbyte` non dice «questo è un carattere», dice «questo è il numero 48», e il programma deve sapere che 48 è la cifra `0`. Il secondo ciclo cerca il primo spazio (codice `32`) e poi `strsub` estrae i byte fino a quello precedente; nota

che `fine - 1` può valere `0` se la stringa comincia con uno spazio, ed è un caso che un tokenizer reale deve gestire esplicitamente.

Un programma può leggere se stesso o un altro sorgente con `readfile(ARGS[1])`: è esattamente il primo passo di un interprete self-hosted (capitolo 37). Vale però la pena ribadire il confine: un programma che usa `readfile`, `strlen`, `strbyte`, `strsub` o `exit` **non** è un programma Triad-Full valido. È una delle poche eccezioni alla regola «Core è un sottoinsieme di Full», ed è il motivo per cui questi primitivi compaiono in un capitolo a parte invece che tra le operazioni normali sulle stringhe.

---

14. Errori di esecuzione

Un errore di esecuzione è un punto in cui il programma non può più andare avanti: un nome che non esiste, un tipo sbagliato, un indice fuori intervallo. In Triad-Full gli errori sono valori e si possono gestire; in Triad-Core no, e la differenza è importante: qui un errore **termina il processo con exit code 1** e stampa un messaggio diagnostico su stderr. Quello che era già stato stampato su stdout resta.

Poiché non c'è modo di catturare l'errore, la strategia di un programma Core non è «gestirlo» ma «non provocarlo»: controllare i limiti prima di un accesso, convertire i valori prima di usarli, verificare le ipotesi all'inizio di una funzione. Il messaggio del runtime serve a capire dove e perché il programma si è fermato.

```
println("prima")
println(non_esiste)
println("dopo")
```

```
stdout:  prima
stderr:  error: undefined variable non_esiste
exit:    1
```

Messaggi dell'interprete JS (stderr) e cause tipiche:

messaggio	causa
`parse error: unexpected token …` / `parse error: expected …`	sintassi non Core (es. `1.5`, `/`, `() -> …`, `end` mancante)

messaggio	causa
`error: undefined variable NOME`	nome mai assegnato, o funzione chiamata prima di essere definita
`error: not a function`	si chiama un valore che non è una funzione (`x = 5; x(1)`)
`error: arity mismatch`	numero di argomenti diverso dai parametri
`error: index out of range`	`xs[i]` con `i < 1` o `i > length(xs)`
`error: + type error` (e simili per `- * % < …`)	operandi di tipo sbagliato (`1 + "a"`)
`error: if cond` / `while cond` / `ternary cond` / `&& expects bool`	condizione non `Bool`
`error: div by zero` / `negative exponent`	`x % 0`, `2 ^ -1`
`error: unknown function NOME`	builtin riservato usato in modo non supportato
`cannot read file: …`	file inesistente o non leggibile
`usage: triad FILE [ARG...]` (exit 2)	nessun file passato

Il seed Windows non stampa messaggi dettagliati: esce con codice `1` (sintassi, variabile o funzione non definita, arità, lettura) o `2` (nessun argomento).

Per diagnosticare un errore, separa il **livello** dal **punto del programma**. Un `parse error` indica che il testo non è stato riconosciuto come grammatica Core: controlla prima il profilo, parentesi e `end`, poi gli operatori non Core. Un `undefined variable` o un errore di tipo arriva invece durante la valutazione: controlla l'ordine delle definizioni, l'ambiente in cui nasce il nome e il tipo effettivo degli argomenti. Se il programma stampa una prima riga e poi fallisce, quell'output precedente non viene annullato; riproduci il caso minimo e confronta stdout, stderr e codice di uscita separatamente.

Non copiare automaticamente il testo dell'errore prodotto da un runtime in un test. La suite Core confronta stdout e codice di uscita, non stderr: il contratto importante è la classe di comportamento dichiarata dallo standard. I dettagli diagnostici possono cambiare tra runtime, mentre un test del valore restituito o dell'uscita deve restare ripetibile.

14.1 Errori come valori, in Core

Per propagare errori senza terminare il processo (come farà S1 per i programmi Full) si usa una convenzione con liste: `[:Err, kind, msg]`.

```
err(kind, msg) = [:Err, kind, msg]
is_err(v) = typeof(v) == :List && length(v) == 3 && v[1] == :Err
function div_intera(a, b)
    if b == 0
        return err(:domain, "divisione per zero")
    end
    q = 0
    while a >= b
        a -= b
        q += 1
    end
    q
end
r = div_intera(7, 0)
if is_err(r)
    println("errore ", r[2], ": ", r[3])
end
println(div_intera(17, 5))
```

Output:

```
errore domain: divisione per zero
3
```

La convenzione `[:Err, kind, msg]` non è un tipo del linguaggio: è un protocollo tra funzioni, scritto con i mezzi che Core offre (liste e simboli).

Chi rileva un errore lo costruisce con `err`; chi lo riceve deve controllare il tag prima di leggere i campi, ed è per questo che `is_err` esiste. Il vantaggio è che l'errore viaggia come qualsiasi altro valore: può essere restituito, salvato, confrontato, stampato. Il limite è che nulla obbliga una funzione a rispettare il protocollo: se restituisce una lista normale, il controllo del tag fallisce e il programma prosegue con il valore sbagliato.

Quando S1 eseguirà Triad-Full, il linguaggio avrà i valori `Err` veri (capitolo 24) e questa convenzione diventerà superflua; nel frattempo è il modo di far convivere «errore come dato» con un runtime che non lo prevede.

Nota: `&&` cortocircuita, quindi `v[1]` è valutato solo se `v` è una lista di 3 elementi.

---

15. Interprete JS e seed Windows: differenze

Triad-Core ha due implementazioni, e non sono la stessa cosa. L'interprete JS è lo strumento di sviluppo: gira ovunque ci sia Node.js, è facile da modificare ed è il riferimento usato per verificare i casi di conformità. Il seed Windows è il primo passo del bootstrap: un interprete in codice macchina, senza runtime ospiti, più limitato perché scritto a mano. Sapere che cosa funziona su entrambi è essenziale se un programma deve girare anche sul seed, come il codice di S1.

Il contratto è `conformance/core/`: entrambi i runtime devono produrre gli stessi golden. Oggi l'interprete JS li passa tutti, il seed Windows i 33 casi elencati nel job `seed-windows` della CI. Fuori da quei casi i due runtime differiscono:

funzionalità	**interprete JS** (`seed/dev/triad_core.cjs`)	**seed Windows** (`seed.exe`)
interi, Bool, operatori, confronti, ternario	✅	✅
`if` / `elseif` / `else`, `while`, `let`, `begin`	✅	✅
funzioni forma breve e blocco, ricorsione	✅	✅ (max 4 parametri)

funzionalità	interprete JS (`seed/dev/triad_core.cjs`)	seed Windows (`seed.exe`)
`return`, `break`, `continue`	✅	non elencati in `seed/windows-x86_64/README.md`
lambda	✅ più parametri	✅ **un solo parametro**
chiusure	condivise (mutazioni visibili, SPEC-CORE §5)	**per valore**, solo l'ultimo legame del frame
stringhe, `*`, `length(s)`	✅	✅
`string(x)`	✅ tutti i tipi	solo `Int` e `String`
simboli, `nothing`	✅	✅
liste	eterogenee, `xs[i]`, `xs[i] = v`, `length`, `push!`	solo interi; costruzione, stampa e `push!` (max 64 elementi)
`get(xs, i)`	non esiste	helper **non standard**, indici **da 0**
`-=`, `*=`, `Int`, `Symbol`, `typeof`	✅	non elencati nel README del seed

funzionalità	interprete JS (`seed/dev/triad_core.cjs`)	seed Windows (`seed.exe`)
`ARGS`, primitivi di bootstrap	✅	non elencati nel README del seed
stampa di una funzione	`#lambda(x)`	`fun`
`triad -` (stdin)	✅	no
messaggi d'errore	su stderr	solo exit code

La tabella va letta come una mappa di portabilità: la colonna di sinistra è il comportamento che un programma Core corretto può usare, la destra dice che cosa il seed sa fare oggi. «Non elencato nel README» non significa automaticamente «non funziona», ma significa che non è un comportamento su cui puoi contare senza provarlo. Le differenze più insidiose sono quelle semantiche, non quelle sintattiche: il seed può accettare una lambda e catturarla in modo diverso, quindi un programma che viene eseguito non è detto che dia lo stesso risultato.

Regola pratica: **scrivi codice che rispetta SPEC-CORE e verificalo con l'interprete JS**; se deve girare sul seed (come `selfhost/`), restringiti alla colonna di destra e verificalo su Windows.

---

16. Pattern idiomatici Triad-Core

I capitoli precedenti hanno spiegato i singoli costrutti; questo li mette insieme in alcuni schemi ricorrenti. Non sono l'unico modo di scrivere Triad-Core, ma sono i modi in cui il linguaggio viene effettivamente usato nel repository, compreso il codice di S1 in `selfhost/`. Riconoscerli aiuta a leggere il codice altrui e a scegliere una struttura quando non ne hai già una in mente.

16.1 Funzioni di ordine superiore scritte in Core

Core non ha `map` / `filter` / `reduce`: si scrivono una volta con `while` e lambda.

```
function mappa(f, xs)
    out = []
    i = 1
    while i <= length(xs)
        push!(out, f(xs[i]))
        i += 1
    end
    out
end
function filtra(p, xs)
    out = []
    i = 1
    while i <= length(xs)
        if p(xs[i])
            push!(out, xs[i])
        end
        i += 1
    end
    out
end
function riduci(f, acc, xs)
    i = 1
    while i <= length(xs)
        acc = f(acc, xs[i])
        i += 1
    end
    acc
end
```

```
xs = [1, 2, 3, 4, 5, 6]
println(mappa(x -> x * x, xs))
println(filtra(x -> x % 2 == 0, xs))
println(riduci((a, b) -> a + b, 0, xs))
```

Output:

```
 [1, 4, 9, 16, 25, 36]
[2, 4, 6]
21
```

Questi tre schemi mostrano che le funzioni di ordine superiore non richiedono supporto sintattico: una funzione che riceve un'altra funzione è già possibile, e `while` basta per scorrere la lista. `mappa` costruisce una lista nuova, `filtra` decide elemento per elemento, `riduci` porta avanti un accumulatore. Sono anche il punto di partenza quando il tuo programma deve trasformare dati senza le comodità di Triad-Full.

16.2 Record come liste taggate

Core non ha `struct`: un record è una lista con un simbolo come tag nella prima posizione.

```
punto(x, y) = [:Punto, x, y]
px(p) = p[2]
py(p) = p[3]
somma_punti(a, b) = punto(px(a) + px(b), py(a) + py(b))
p = somma_punti(punto(1, 2), punto(10, 20))
println(p, " ", p[1] == :Punto)
```

Output:

```
[:Punto, 11, 22] true
```

Il tag non è decorazione: è il discriminante che permette di riconoscere il tipo di record a runtime e di scrivere funzioni come `px` e `py` che assumono una forma precisa. In Full lo stesso ruolo sarà dello `struct` (capitolo 20), con il vantaggio di un controllo dei campi; in Core la lista taggata dà la stessa struttura senza quel controllo, e sta al programmatore rispettare la convenzione.

16.3 Ambiente come lista di associazioni

È la struttura usata dagli interpreti in `selfhost/`: una lista piatta `[nome1, valore1, nome2, valore2, …]` (in Core puoi anche usare coppie annidate).

```
function env_get(env, nome)
    i = 1
    while i < length(env)
        if env[i] == nome
            return env[i + 1]
        end
        i += 2
    end
    nothing
end
function env_set(env, nome, valore)
    i = 1
    while i < length(env)
        if env[i] == nome
            env[i + 1] = valore
            return env
        end
        i += 2
    end
    push!(env, nome)
    push!(env, valore)
end
env = []
env_set(env, :x, 10)
env_set(env, :y, 20)
```

```
env_set(env, :x, 11)
println(env, " ", env_get(env, :x), " ", env_get(env, :z))
```

Output:

```
[:x, 11, :y, 20] 11 nothing
```

La lista piatta alterna nome e valore: `env_get` avanza di due posizioni, `env_set` aggiorna se trova il nome e altrimenti aggiunge in coda. Il `while i < length(env)` (non `<=`) è deliberato: l'ultima posizione utile è `length(env) - 1`, perché `env[i + 1]` deve restare dentro la lista. È la stessa struttura che rende possibile un interprete scritto in Core: gli ambienti sono dati come gli altri, non un meccanismo nascosto del linguaggio.

16.4 Stato mutabile condiviso

Poiché una funzione nominata non può riassegnare un globale, lo stato condiviso si tiene in una lista (mutabile) o in una chiusura.

```
stato = [0]
function tick()
    stato[1] += 1
end
tick()
tick()
println(stato[1])
```

Output:

```
2
```

La lista fa da cella condivisa: la funzione nominata non può riassegnare `stato`, ma può modificare l'oggetto a cui `stato` punta, e la mutazione resta visibile fuori. È il modo più semplice di ottenere stato mutabile in Core senza chiusure; la chiusura (capitolo 10) è l'alternativa quando lo stato deve restare privato.

16.5 Costruire stringhe

```
function ripeti(s, n)
    out = ""
    while n > 0
        out = out * s
        n -= 1
    end
    out
end
function unisci(xs, sep)
    out = ""
    i = 1
    while i <= length(xs)
        if i > 1
            out = out * sep
        end
        out = out * string(xs[i])
        i += 1
    end
    out
end
println(ripeti("ab", 3))
println(unisci([1, 2, 3], ", "))
```

Output:

```
ababab
1, 2, 3
```

Costruire una stringa per concatenazioni successive è corretto ma costa: ogni `*` crea una nuova stringa, quindi il ciclo costruisce tante stringhe intermedie quante sono le iterazioni. In Core non ci sono alternative migliori — non c'è una struttura di appoggio — e per i programmi di studio va bene; in un interprete reale conviene accumulare i pezzi in una lista e unirli una volta sola alla fine.

16.6 Stile

Le regole seguenti non sono estetiche: nascono dal fatto che lo stesso codice potrebbe dover passare a Triad-Full e che il repository è scritto in italiano.

- Funzioni piccole con nomi chiari (in italiano, come nel resto del repository).
- Caso base esplicito in ogni ricorsione; per profondità grandi preferisci `while`.
- Cicli con contatori espliciti (`i = 1 … while i <= length(xs)`).
- Non usare `fact`, `rule`, `solve`, `struct` e le altre parole chiave di Full come nomi: il codice deve restare valido anche in Triad-Full.
- Scrivi `!io` sulle funzioni che stampano: Core lo ignora, Full lo richiede.

Sono convenzioni, non regole del linguaggio: il runtime non le controlla. Ma un programma che le rispetta resta valido sia in Core sia in Full, e questa è la qualità che rende il codice del progetto riutilizzabile negli stadi successivi del bootstrap.

---

17. Esercizi guidati

Gli esercizi mettono in pratica i capitoli 4–16 e, letti di seguito, raccontano un percorso: si parte da calcolo e ricorsione, si passa a liste e stringhe, e si finisce con un piccolo tokenizer e un valutatore di espressioni, cioè con i primi due pezzi di un interprete. Ogni esercizio ha una soluzione verificata, nascosta in un riquadro: aprila solo dopo averci provato, perché il punto non è leggere la soluzione ma scriverne una che produca lo stesso output.

Per controllare la tua soluzione, salvala in un file ed eseguila con `node seed/dev/triad_core.cjs`; confronta poi l'output con quello mostrato nella soluzione. Se l'output coincide ma il codice è diverso, va bene: gli esercizi descrivono un comportamento, non un'unica scrittura possibile.

Esercizio 1 — Potenza e somme

Senza usare `^`, scrivi `potenza(b, e)` ricorsiva e `somma_fino(n)` (somma `1 + … + n`) con `while`. Stampa `potenza(3, 13)`, `potenza(2, 10)` e `somma_fino(100)`.

▶ Soluzione

Esercizio 2 — Numeri primi

Stampa la lista dei numeri primi minori di 30.

▶ Soluzione

Esercizio 3 — Inverti una lista

Scrivi `inverti(xs)` che restituisce una nuova lista con gli elementi in ordine inverso.

▶ Soluzione

Esercizio 4 — Ordinamento per inserzione

Ordina in loco `[5, 2, 9, 1, 7]`.

▶ Soluzione

Esercizio 5 — Contatore di parole

Conta le parole (separate da spazi) di `"il linguaggio Triad "` usando `strbyte`.

▶ Soluzione

Esercizio 6 — Un mini-tokenizer

Trasforma `"12+x*3"` in una lista di token `[tipo, testo]`, con tipi `:num`, `:id`, `:op`. È il primo passo di un interprete (vedi `selfhost/tokenizer.td`).

▶ Soluzione

Esercizio 7 — Valutare un'espressione

Usando i token dell'esercizio 6, scrivi un valutatore a discesa ricorsiva per `+` e `*` con precedenza, dove le variabili vengono da una lista di associazioni. Suggerimento: tieni la posizione corrente in una lista `pos = [1]` per poterla aggiornare dalle funzioni.

▶ Soluzione

Parte III — Triad-Full (lo standard completo)

Questa parte descrive il linguaggio definito da `SPEC.md`. **Non è ancora eseguibile** dal runtime attuale: serve per conoscere lo standard, scrivere codice pronto per S1 e capire i golden di `conformance/cases/`. Ogni capitolo indica la sezione di SPEC da cui deriva. Gli output marcati **Output (golden Full):** provengono dai golden congelati.

18. Lessico e tipi Full (SPEC §1, §4)

Full estende il lessico di Core con:

elemento	forma	esempi
identificatore	come Core, anche lettere non ASCII; `!` finale se non seguito da `=`	`età`, `push!`
variabile logica	`?` **attaccato** a un nome	`?x`, `?who`
variabile anonima	`?_` (ogni occorrenza è nuova)	`parent(?x, ?_)`
`Float`	`3.14`, `1e-9`	
interpolazione	`"x = $x"`, `"$(a + b)"`; escape aggiuntivo `\$`	
commenti annidati	`#= a #= b =# c =#`	
macro	`@nome(...)`	`@unless(c, e)`

Parole chiave: `function struct macro end if elseif else for in while let begin return break continue fact rule solve quote try catch with requires ensures true false nothing isa is`. Contestuali dopo `solve`: `limit depth steps trace`.

Regole dei due punti `:`

- `1:n`, `xs[2:end]`: `:` attaccato a un valore, senza spazi → range;

- `f(:a)`, `x = :a`: `:` non preceduto da un valore e seguito da un nome → simbolo;
- `:( … )` → codice quotato;
- `c ? a : b`: `:` circondato da spazi → ternario.

Regola del `?`: `?x` attaccato = variabile logica; `?` con spazio = ternario.

Gli a capo chiudono un'istruzione, tranne dentro `( )`, `[ ]` e `:( )`, e subito dopo un operatore binario o una virgola di `rule`.

Tipi Full (oltre a quelli di Core):

tipo	valori
`Int`, `Float` (sottotipi di `Number`)	`42`, `3.5`
`Expr`	codice quotato (`Node`)
`Term`, `Cons`	termini e pattern lista logici
`Solution`	`{who = :dave}`
`Err`	errore come dato
`Type`	`typeof(3) == Int`
struct utente	`Point(3.0, 4.0)`
`Any`	supertipo di tutto, usato nelle firme

Variabili tipate:

```
x::Int = 5
x = 6          # bene
```

```
x = 5.5        # Err(:type): il nome mantiene il tipo dichiarato
y::Float = 1.0
y = 2          # 2.0: conversione Int → Float
```

Il vincolo nasce con la dichiarazione e vive nell'ambiente in cui il nome è stato creato.

---

19. Funzioni tipate e multiple dispatch (SPEC §5)

```
sq(x) = x * x
area(n::Number) = :number
area(n::Int) = :int

function dist(a::Point, b::Point)::Float
    requires a.x != b.x || a.y != b.y
    sqrt((a.x - b.x)^2 + (a.y - b.y)^2)
end

function +(a::Point, b::Point)      # gli operatori sono funzioni ridefinibili
    Point(a.x + b.x, a.y + b.y)
end
```

Regole:

- vince il metodo **più specifico** tra gli applicabili (`area(1) == :int`, `area(1.5) == :number`);
- nessun metodo applicabile → `Err(:method)` (o fallback builtin per gli operatori numerici);
- più metodi ugualmente specifici → `Err(:ambiguous)`;
- il tipo di ritorno `::R` è controllato all'uscita (con conversione Int→Float);
- funzioni e operatori sono valori: `reduce(+, xs)`, `map(sq, xs)`;
- lambda tipate: `function (x::Int) x end` → `f(2.0)` è `Err(:type)`.

Per prevedere quale implementazione verrà chiamata, controlla prima il numero di argomenti e poi il tipo di ciascuno. Con i metodi `area(n::Number)` e `area(n::Int)`, l'intero `1` è compatibile con entrambi, ma `Int` è più specifico di `Number`; il `Float` `1.5` è compatibile solo con il metodo su `Number`. Nei metodi con più parametri il confronto è per tupla: una firma può essere più specifica sul primo parametro ma più generale sul secondo. Se nessuna firma domina tutte le altre, non si indovina l'intenzione: il risultato è l'errore `:ambiguous`, che richiede una firma più specifica o una revisione delle firme esistenti.

Il controllo del tipo di ritorno avviene dopo il corpo. Per questo `::Float` non converte gli argomenti in anticipo e non garantisce che ogni operazione intermedia sia valida; richiede che il valore prodotto alla fine sia convertibile o compatibile con il tipo dichiarato. Le annotazioni di parametro e di ritorno sono quindi parti distinte del contratto del metodo.

---

20. Struct (SPEC §4)

```
struct Point
    x::Float
    y::Float
end

p = Point(3, 4)      # costruttore posizionale, Int → Float
p.x                  # 3.0
typeof(p) == Point   # true
p isa Point          # true
```

- Immutabili; campi tipizzati.
- Campo inesistente → `Err(:field)`; tipo sbagliato → `Err(:type)`.
- In Core lo stesso concetto si rappresenta con una lista taggata (§16.2).

Una struct fornisce un record **nominato**: il nome del tipo distingue `Point` da un altro record, mentre i nomi dei campi (`x`, `y`) evitano di ricordare posizioni numeriche. La costruzione controlla i valori rispetto ai tipi dei campi; l'accesso `p.x` risolve il campo per nome. La struttura è immutabile nello standard, quindi i campi descrivono lo stato dell'istanza creata e non si aggiornano con un'assegnazione successiva. La lista taggata Core è più flessibile, ma sposta queste garanzie sul codice che costruisce, legge e modifica il record.

---

21. Controllo di flusso Full: `for`, range, comprehension, `try` (SPEC §2)

```
s = 0
for i in 1:10
    s += i
end                                  # s == 55

xs = [1, 2, 3]
xs[end]                              # 3
xs[2:end]                            # [2, 3]
[x^2 for x in 1:4 if x > 2]          # [9, 16]
x = 3; x > 2 ? :big : :small         # :big

e = try div(1, 0) end                # l'errore diventa il valore
r = try div(1, 0) catch err; 0 end   # con catch: err è l'Err, il valore è quello
del ramo
```

Altri valori attesi (dai vecchi test dell'implementazione Julia):

```
x = 2; y = x * 21; y                            # 42
7 / 2                                           # 3.5
"ab" * "cd"                                     # "abcd"
let a = 1, b = a + 1; a + b; end                # 3
if 1 > 2; :a; elseif 2 > 1; :b; else; :c; end   # :b
sq(x) = x * x; map(sq, [1, 2, 3])               # [1, 4, 9]
reduce(+, [1, 2, 3, 4])                         # 10
filter(x -> x % 2 == 0, 1:6)                    # [2, 4, 6]
function fact_(n::Int)::Int
    if n <= 1; return 1; end
```

```
    n * fact_(n - 1)
end; fact_(10)                                  # 3628800
```

`requires c` al top level è un'asserzione; `ensures c` è ammesso solo dentro `function`.

Nel ciclo, il range `1:10` rappresenta i valori interi inclusivi da 1 a 10: non è una lista letterale che devi scrivere a mano. `for` associa a `i` ciascun valore successivo, esegue il corpo per quell'associazione e prosegue fino alla fine dell'iterabile. Una comprehension combina lo stesso schema con la costruzione del risultato: valuta il filtro per ogni elemento e aggiunge all'output solo le espressioni corrispondenti. L'ordine di lettura è quindi "iteratore, filtro, espressione", non una valutazione simultanea.

`try` delimita invece il punto in cui un errore può essere trasformato in risultato. Senza `catch`, l'espressione restituisce direttamente l'`Err`; con `catch err`, il nome locale `err` identifica l'errore e il valore finale è quello del blocco di recupero. Questo rende possibile gestire un caso senza confondere un errore con un valore di successo dello stesso tipo.

---

22. Logica: fatti, regole, `solve` (SPEC §6)

```
fact parent(:alice, :bob)
fact parent(:alice, :carol)
fact parent(:bob, :dave)

rule grandparent(?x, ?z) :- parent(?x, ?y), parent(?y, ?z)
rule ancestor(?a, ?d) :- parent(?a, ?d)
rule ancestor(?a, ?d) :- parent(?a, ?x), ancestor(?x, ?d)
```

`solve` restituisce una **lista di** `Solution` (è un valore, non stampa nulla):

```
sols = solve grandparent(:alice, ?who) limit 5
[s.who for s in sols]                       # [:dave]
[s.d for s in solve ancestor(:alice, ?d)]
```

- Opzioni: `limit n`, `depth d`, `steps s` (default 1 000 000), `trace`.
- Un predicato usato come funzione è un `Bool`: `grandparent(:alice, :dave)` → `true`; si può usare in `if`.
- Goal: `p(t…)`, `g1 && g2`, `(g1, g2)`, `g1 || g2`, `!g`/`not(g)`, `a = b` (unificazione), `?y is espr`, confronti, funzione Triad che restituisce `Bool`, `true`/`false`.
- Occurs-check attivo: `solve ?x = f(?x)` non ha soluzioni (non va in ciclo).
- Liste stile Prolog nei pattern: `[?h | ?t]`.
- `vars(s)` restituisce i nomi legati in una `Solution`; si accede con `s.nome` o `s[:nome]`.

Per leggere una regola, separa la **testa** dal **corpo**. In `grandparent(?x, ?z) :- parent(?x, ?y), parent(?y, ?z)`, la testa descrive quali valori la regola può dimostrare; il corpo richiede due prove in sequenza. Il motore cerca un `parent` per il primo goal, conserva il legame trovato per `?y`, poi usa quel legame nel secondo goal. Se il secondo goal fallisce, torna alla scelta precedente e prova il prossimo fatto compatibile: è il backtracking. Se entrambi riescono, `?z` viene restituito nella soluzione.

Le variabili con lo stesso nome in clausole diverse non condividono accidentalmente il loro stato: a ogni uso della regola vengono rinominate. L'occurs-check impedisce legami ricorsivi impossibili come `?x = f(?x)`. Il limite `depth` taglia i rami troppo profondi; `steps` limita il lavoro totale; `limit` limita quante soluzioni vengono raccolte. Sono controlli differenti, e una lista vuota ottenuta entro i limiti non è equivalente a una ricerca interrotta da `Err(:budget)`.

Esempi:

```
# percorso con costo
fact edge(:a, :b, 2)
fact edge(:b, :c, 3)
rule path(?x, ?y, ?c) :- edge(?x, ?y, ?c)
rule path(?x, ?y, ?c) :- edge(?x, ?z, ?c1), path(?z, ?y, ?c2), ?c is ?c1 + ?c2
(solve path(:a, :c, ?costo))[1].costo      # 5

# append alla Prolog
rule app([], ?l, ?l)
rule app([?h | ?t], ?l, [?h | ?r]) :- app(?t, ?l, ?r)
(solve app([1, 2], [3], ?r))[1].r          # [1, 2, 3]
length(solve app(?a, ?b, [1, 2, 3]))       # 4
```

```
# negazione per fallimento
fact parent(:tom, :bob)
persone = [:tom, :bob]
foglie = filter(p -> !parent(p, ?_), persone)   # [:bob]
```

---

23. Destrutturazione (SPEC §2)

Pattern di lista con **arità esatta**; le parentesi attorno al pattern sono obbligatorie.

```
(a, b) = [1, 2]                 # a == 1, b == 2
e = try (a, b) = [1] end        # e.kind == :bounds
e = try (a, b) = [1, 2, 3] end  # e.kind == :bounds


xs = []
for (k, v) in [[1, 2], [3, 4]]
    push!(xs, k + v)
end                             # xs == [3, 7]
```

- Un elemento non lista o di arità diversa nel `for` → `Err(:bounds)` o `Err(:type)`.
- `function f((a, b))` definisce **due parametri** `a` e `b` (i parametri sono appiattiti): `f(10, 20) == 30`. Per destrutturare un singolo argomento lista usa `(a, b) = pair` nel corpo.
- I pattern annidati ricorsivi non sono ancora previsti.

---

24. Errori come dati (SPEC §8)

Un errore è un valore `Err(kind, msg, data)`; non diventa un'eccezione dell'host salvo che non sia gestito al top level (allora il processo esce con codice 1 e stdout vuoto per l'istruzione fallita: è il caso di conformance `errore`).

```
e = try div(1, 0) end     # Err(:domain, ...)
e.kind                    # :domain
e.msg                     # messaggio
iserr(e)                  # true
7.0 / 0                   # Inf: la divisione Float non è un errore
throw(err(:mio, "ops", 42))
error("boom")             # Err(:error, ...)

describe(e::Err) = string("errore ", e.kind, ": ", e.msg)
describe(x::Any) = string("valore ", x)
for r in [try div(10, d) end for d in [2, 0, 5]]
    println(describe(r))
end
```

Kinds standard: `:syntax`, `:undef`, `:type`, `:method`, `:ambiguous`, `:field`, `:bounds`, `:domain`, `:instantiation`, `:contract`, `:budget`, `:effect`, `:host`.

`Err(:syntax)` porta in `data` la posizione (`e.data.line`, `e.data.col`).

La gestione degli errori separa l'operazione fallita dal punto in cui il programma decide come reagire. `try` non ripara automaticamente lo stato: converte l'esito in un valore `Err` oppure passa quel valore al blocco `catch`. Il chiamante può quindi scegliere tra mostrare un messaggio, provare un'alternativa o propagare l'errore restituendolo. Se nessun `try` lo intercetta,

il runtime lo segnala al livello superiore e il processo termina con codice non riuscito. In questo modo la gestione è esplicita, ma un errore non diventa un successo solo perché è rappresentato da un valore.

---

25. Contratti (SPEC §9)

```
function safe_div(a::Number, b::Number)::Float
    requires b != 0
    ensures abs(result * b - a) < 1e-9
    a / b
end

safe_div(6, 3)            # 2.0
e = try safe_div(1, 0) end
e.kind                    # :contract
e.data.clause             # :requires
e.data.fn                 # :safe_div
```

- `requires c` è controllata dove compare (inizio del corpo, oppure al top level come asserzione).
- `ensures c` è controllata all'uscita, con `result` legato al valore restituito, dopo il controllo del tipo di ritorno.
- Violazione → `Err(:contract, …)` con `data.clause`, `data.fn`, `data.cond`, `data.result`.
- I contratti sono **puri**: un effetto dentro un contratto → `Err(:effect)`.
- `ensures` fuori da `function` → `Err(:syntax)`.

Pensa a `requires` come alla condizione d'ingresso dell'operazione e a `ensures` come alla condizione d'uscita. Una precondizione specifica ciò che il chiamante deve garantire; una postcondizione specifica ciò che la funzione promette quando termina correttamente. Le clausole non sostituiscono il controllo dei tipi: una funzione può ricevere argomenti del tipo giusto e

violare comunque una precondizione di dominio, oppure restituire un valore tipato che viola la postcondizione. Poiché i contratti sono puri, la loro valutazione non può stampare o modificare la base di conoscenza: il controllo non deve cambiare il programma che sta verificando.

---

26. Budget (SPEC §10)

```
r = with budget(steps = 10_000)
    while true end
end
r.kind                    # :budget — il ciclo infinito diventa un valore
```

- Un passo = una chiamata, un'iterazione di ciclo, un passo di risoluzione.
- I budget si annidano: vale il più stretto; solo il budget del `with` corrente diventa valore, quello esterno si propaga.
- `solve … steps N` è il budget di una singola query.
- La ricorsione infinita non fa stack overflow: finisce in `Err(:budget)` (stack esplicito).

Il budget misura unità di lavoro definite dallo standard, non secondi di CPU e non memoria allocata. È quindi un limite riproducibile sul numero di passi del modello di esecuzione, non un timeout generale del sistema operativo. Usalo per rendere limitate le computazioni potenzialmente lunghe, in particolare la risoluzione logica o un ciclo che potrebbe non terminare. Il risultato `Err(:budget)` è distinto dal risultato valido del corpo e può essere gestito con `try` o con il controllo del valore restituito dal `with`.

---

27. Tracce (SPEC §11)

La traccia è un **valore**, non output.

```
fib(n::Int) = n < 2 ? n : fib(n - 1) + fib(n - 2)
t = with trace
    fib(3)
end
t.result      # 2
t.trace       # [Event(port=:call, fn=:fib, args=[3], depth=0), …]

t = solve path(:a, :c) trace
t.trace       # porte Byrd: :call, :exit, :redo, :fail, :depth
```

Gli eventi sono record con `port`, `goal`/`fn`, `depth` (e `value`/`args` per le funzioni): si filtrano come qualsiasi valore.

La traccia registra la storia del calcolo, non solo il suo risultato. In una traccia di funzione `call` indica l'ingresso e `exit` l'uscita; in una query logica compaiono anche `redo` e `fail`, perché la ricerca può provare più alternative. Il valore `result` resta disponibile accanto agli eventi: puoi confrontare il risultato finale con il percorso seguito. La traccia non è un log stampato automaticamente; decide il programma quali eventi filtrare, contare o presentare.

---

28. Effetti dichiarati (SPEC §12)

Le funzioni sono pure per default. Effetti: `io` (`println`, `print`) e `state` (`fact`/`rule` dentro una funzione).

```
function saluta(nome::String) !io
    println("ciao, ", nome)
end

function impara(x) !state
    fact visto($x)
end
```

- Controllo dinamico: una funzione pura che chiama una `!io` o `println` → `Err(:effect)`.
- Il top level permette tutti gli effetti; le lambda ereditano quelli del contesto.
- Macro e contratti sono puri; leggere la KB (`solve`, predicati) è puro.
- Mutare liste o variabili locali non è un effetto.

La dichiarazione di effetto è una verifica dinamica dei permessi di esecuzione, non un'annotazione puramente descrittiva. Il runtime controlla la catena delle chiamate quando una funzione tenta un'operazione con effetto: una funzione `!io` può stampare solo se il contesto chiamante lo consente. Il top level è il contesto più permissivo; una funzione pura chiamata da lì non diventa per questo autorizzata a stampare. Elencare `!io` nella firma documenta il requisito per i chiamanti e permette al runtime di rifiutare l'effetto nel punto in cui violerebbe la dichiarazione.

---

29. Codice come dato, albero uniforme e macro (SPEC §3, §7)

29.1 L'albero uniforme

Il parser produce un solo tipo di nodo, `Node(head, args)`: è lo stesso albero che il kernel esegue, che `:( … )` restituisce e che `--tree` stampa come S-expression.

sintassi	nodo
`f(a, b)`, `a + b`	`(call f a b)`, `(call + a b)`
`a && b`, `a || b`	`(and a b)`, `(or a b)`
`x = e`	`(assign x e)`
`:alice`	`(sym alice)`
`a.x`, `xs[i]`	`(dot a x)`, `(index xs i)`
`[a, b]`, `[h | t]`	`(list a b)`, `(cons (list h) t)`
`[e for x in xs if c]`	`(comprehension e x xs c)`
`x -> e`	`(lambda (tuple x) e)`
`x::T`	`(typed x T)`
`"a $x"`	`(string "a " x)`
`if c a else b end`, `c ? a : b`	`(if c (block a) (block b))`
`while`, `for`, `let`, `begin`	`(while c body)`, `(for x iter body)`, `(let (tuple …) body)`, `(block …)`

sintassi	nodo
`function f(a::T)::R !io … end`	`(function f (tuple (typed a T)) R (tuple io) body)`
`struct P; x::Float; end`	`(struct P (tuple (typed x Float)))`
`macro m(a) … end`, `@m(e)`	`(macro m (tuple a) body)`, `(macrocall m e)`
`:(e)`, `quote … end`, `$e`	`(quasi e)`, `(quasi (block …))`, `(unquote e)`
`fact t`, `rule h :- g1, g2`	`(fact t)`, `(rule h g1 g2)`
`solve g limit 5 trace`	`(solve g (tuple (assign limit 5) (assign trace true)))`
`try b catch e h end`	`(try b e h)`
`with budget(steps = N), trace … end`	`(with (tuple (call budget (assign steps N)) trace) body)`

Leggi una riga della tabella come una mappa tra **sintassi visibile** e **rappresentazione interna**. Per esempio `a + b` non richiede un tipo di nodo speciale per la somma: è una chiamata al nome `+` con due argomenti. Al contrario `&&` diventa un nodo dedicato perché il valutatore deve poter evitare di valutare il secondo operando. La forma dell'albero quindi non è una decorazione: codifica le decisioni che il runtime deve prendere per rispettare la semantica.

Questo modello è utile anche quando si legge un errore: se `sexpr` mostra un albero che non corrisponde al raggruppamento atteso, il problema nasce nel parsing; se l'albero è corretto ma il risultato è sbagliato, il problema è più

probabilmente nella valutazione. La distinzione tra forma del programma e valore del programma è il fondamento della metaprogrammazione.

29.2 Quote, unquote, eval

```
c = :(a.x + f(b, 2))
c.head        # :call
sexpr(c)      # "(call + (dot a x) (call f b 2))"


n = 7
quadrato = :($n * $n)
eval(quadrato)                  # 49
parse("if x > 0; :pos; else; :neg; end")   # albero
unparse(parse_program("1:n"))   # "1:n"
Expr(:call, :f, 1)              # costruisce un Node
```

29.3 Macro

Le macro ricevono codice e restituiscono codice; l'espansione è pura e **non igienica**: usa `gensym()` per nomi freschi.

```
 macro unless(c, body)
    :(if !$c; $body; end)
end


macro swap(a, b)
    tmp = gensym(:tmp)
    quote
        $tmp = $a
        $a = $b
        $b = $tmp
    end
end


macro time_it(label, e)
    :(begin
        r = $e
        println($label, " -> ", r)
        r
    end)
end


@swap(x, y)
@unless(x < y, println("x non è minore di y"))
@time_it("somma", reduce(+, 1:100))     # stampa "somma -> 5050"
```

Per valutare una macro, confronta l'argomento sorgente con il codice che essa costruisce. In `@unless(c, body)`, `c` e `body` arrivano come nodi; la macro li inserisce in un `if` con la condizione negata. Il corpo verrà eseguito solo se la condizione originale è falsa, proprio come se il programmatore avesse scritto quel codice manualmente. Se la macro usasse una funzione ordinaria, gli argomenti verrebbero calcolati prima della chiamata e non sarebbe possibile controllare se `body` viene valutato. Questa capacità è potente, ma rende più difficile leggere l'esecuzione: l'espansione esplicita e i nomi temporanei generati sono strumenti essenziali per evitare sorprese.

29.4 Codice che genera codice

```
function poly(coeffs)
    acc = :(0)
    i = 0
    for c in coeffs
        acc = :($acc + $c * x ^ $i)
        i += 1
    end
    acc
end
p = poly([1, 2, 3])
x = 2
eval(p)        # 17
```

30. Builtin Full (SPEC §13)

ambito	funzioni
aritmetica e confronto	`+ - * / ^ % == != < <= > >= !`, `abs sqrt floor ceil round exp log sin cos div mod min max`; `*` concatena stringhe
liste	`length isempty first last rest reverse sort sum push! pop! append! vcat collect`, `a:b`, `x in xs`
ordine superiore	`map filter reduce any all count`
stringhe e output	`string repr println print`
tipi	`typeof`, `isa`, `Int Float String Symbol`
errori	`err iserr throw error`
codice	`eval parse unparse sexpr Expr gensym`
logica	`vars(sol)`

Di questi, Core fornisce solo `+ - * % ^`, confronti, `&& || !`, `length`, `push!`, indicizzazione, `*` su stringhe, `Int`, `string`, `Symbol`, `print`, `println`, `typeof` (SPEC-CORE §7).

---

31. Riga di comando e REPL (SPEC §14)

Interfaccia normativa di `triad` (oggi il runtime implementa solo `triad FILE [ARG...]` e, nell'interprete JS, `triad -`):

```
triad                           # REPL
triad FILE [ARG...]             # esegue FILE; ARGS == [ARG...]
triad -l FILE                   # carica FILE e apre il REPL
triad -i FILE                   # esegue FILE e poi apre il REPL
triad -e CODE                   # esegue CODE
triad -E CODE                   # esegue CODE e stampa il valore
triad --tree FILE               # stampa l'albero uniforme come S-expression
triad -q                        # REPL senza banner
triad -h, --help
triad -v, --version
triad -                         # legge lo script da stdin
```

- L'estensione `.triad` / `.td` si può omettere; lo shebang `#!` è ignorato.
- Exit: `0` successo, `1` errore non gestito (stampato su stderr), `2` uso non valido.

REPL (stile Prolog):

```
triad> 1 + 2
3
triad> ?- ancestor(:tom, ?d)
?d = :bob ;
?d = :liz.
triad> parent(:jim, ?x)
false.
```

- I blocchi aperti restano in attesa fino a `end`.
- `?- goal` (o un goal su un predicato della KB) esegue una query; dopo ogni soluzione `;` / `n` /spazio = successiva, `a` = tutte, invio/`.` = stop.
- Comandi: `:load FILE…` (`:l`), `:reload` (`:r`, `make`), `:listing [nome|nome/arità]` (`:ls`), `:tree CODICE`, `:reset`, `:help` (`:h`), `:quit` (`:q`, `halt`, Ctrl-D).

---

32. Esempi Full con output golden

I file sono in `examples/` (eseguibili quando S1 sarà pronto); gli output sono i golden congelati in `conformance/cases/`.

file	copre	caso golden
`examples/assaggio.td`	struct, dispatch, logica, macro, contratti	`assaggio`
`examples/famiglia.td`	fatti/regole, ricorsione, liste Prolog, negazione	`famiglia`
`examples/garanzie.td`	errori, contratti, budget, tracce, effetti	`garanzie_core`
`examples/macro.td`	quote, eval, macro, gensym	`macro`
`examples/tour.td`	tutti i paradigmi	`tour`
`examples/hello_shebang.td`	shebang, `ARGS`, effetti	`hello_shebang`
`examples/benvenuto.triad`	estensione canonica, `Float`, `solve`	—

32.1 `assaggio.td`

```
struct Point
    x::Float
    y::Float
end

function dist(a::Point, b::Point)::Float
    requires a.x != b.x || a.y != b.y
    sqrt((a.x - b.x)^2 + (a.y - b.y)^2)
end

fact parent(:alice, :bob)
fact parent(:bob, :dave)
rule grandparent(?x, ?z) :- parent(?x, ?y), parent(?y, ?z)

result = solve grandparent(:alice, ?who) limit 5
println("nipoti: ", result)     # [{who = :dave}]
println("primo: ", result[1].who) # :dave
```

Output (golden Full) del file completo:

```
nipoti di alice: [{who = :dave}]
primo: dave
quanti: 1
dist(p, q) = 5.0
dist(p, p) -> Err(:contract, "requires violata in dist: a.x != b.x || a.y !=
b.y", Info(fn = :dist, clause = :requires, cond = :(a.x != b.x || a.y != b.y)))
tipo: contract
almeno un nipote
alice è nonna di dave? true
```

32.2 `famiglia.td`

Output (golden Full):

```
 discendenti di tom: [:bob, :liz, :ann, :pat, :jim]
fratelli di ann: [:pat]
madri: ["pat -> jim"]
senza figli: [:liz, :ann, :jim]
over 50: [:tom, :bob]
modi di spezzare [1, 2, 3]:
  [] ++ [1, 2, 3]
  [1] ++ [2, 3]
  [1, 2] ++ [3]
  [1, 2, 3] ++ []
figli di bob: [:ann, :pat]
```

32.3 `garanzie.td`

Output (golden Full, traccia abbreviata):

```
 2.5
violazione: contract — requires violata in safe_div: b != 0
clausola: requires, funzione: safe_div
valore 5
errore domain: divisione intera per zero
valore 2
valore 42
errore input: età non valida
catturato mio con dati [1, 2, 3]
ciclo infinito -> budget
solve infinito -> budget
fib(3) = 2 in 10 eventi
chiamate: [[3], [2], [1], [0], [1]]
  call path(:a, :c) @0
  call edge(:a, :c) @1
  fail edge(:a, :c) @1
  …
ciao, Triad
effetto non dichiarato -> println richiede !io ma rumorosa non lo dichiara
imparato: [:gatto]
```

32.4 `macro.td`

Output (golden Full):

```
 codice:  :(a.x + f(b, 2))
head:    call
args:    [:+, :(a.x), :(f(b, 2))]
sexpr:   (call + (dot a x) (call f b 2))
:(7 * 7) = 49
parse:   (if (call > x 0) (block (sym pos)) (block (sym neg)))
unparse: if x > 0; :pos; else; :neg; end
dopo @swap: x = 2, y = 1
x non è minore di y
somma -> 5050
polinomio: :(0 + 1 * x ^ 0 + 2 * x ^ 1 + 3 * x ^ 2)
p(2) = 17
```

32.5 Contratti e budget insieme

```
function safe_div(a::Number, b::Number)::Float
    requires b != 0
    ensures abs(result * b - a) < 1e-9
    a / b
end
e = try safe_div(1, 0) end
e.kind          # :contract
r = with budget(steps = 1000)
    while true end
end
r.kind          # :budget
```

Parte IV — Il codice del progetto

Questa parte spiega come è fatto il repository e come si scrive (o si modifica) ogni tipo di codice che contiene.

33. Mappa del repository

percorso	linguaggio	cosa contiene
`SPEC.md`, `SPEC-CORE.md`, `VERSION`, `CHANGELOG.md`	Markdown	lo standard (Full e Core), versione, storia
`BOOTSTRAP.md`, `STATUS.md`, `TASK.md`, `plan.md`	Markdown	piano S0→S4, stato, backlog e task
`conformance/`	Triad + Node	casi golden (`cases/` Full, `core/cases/` Core) e runner `run.mjs`
`seed/windows-x86_64/`	codice macchina	`seed.hex` (sorgente), `seed.exe` (decodificato), README con layout
`seed/linux-x86_64/`	codice macchina	seed ELF E0 (`seed.hex`, `seed`)
`seed/dev/`	Node.js	generatori del seed (`gen_seed.js`, `gen_seed_linux.js`) e interprete di riferimento `triad_core.cjs`
`selfhost/`	Triad-Core	S1: interprete Triad scritto in Triad-Core (in corso)

percorso	linguaggio	cosa contiene
`examples/`	Triad-Full	programmi di esempio dello standard
`cli/`	Node.js (ESM)	`triad-cli`: CLI di sviluppo (commander, execa, chalk)
`bin/`	sh / cmd	launcher `triad`, `triad.cmd`, `triad-cli`, `triad-cli.cmd`
`ai/`	Triad + Node	benchmark di generazione codice per LLM, prompt, task con golden
`editors/`	JSON / Vim script	grammatica VS Code (TextMate), sintassi Vim, voce Linguist
`docs/`	HTML/CSS/JS	sito GitHub Pages e materiali pubblicati dal repository
`obsidian-triad/`	Markdown	vault Obsidian con indice dei file e grafo del progetto
`.github/workflows/`	YAML	CI (`ci.yml`) e pubblicazione del sito (`pages.yml`, `static.yml`)
`test_*.td`, `min_*.td` (root)	Triad-Core	piccoli probe usati durante lo sviluppo del seed (es. `test_get.td` in CI)

Convenzioni comuni:

- documentazione, commenti e messaggi sono in **italiano**;
- fine riga **LF** ovunque, tranne `*.cmd` (CRLF), imposto da `.gitattributes`;
- `.gitignore` esclude `*.out` con l'eccezione `!conformance/**/expected.out`.

---

34. L'interprete di riferimento JS (`seed/dev/triad_core.cjs`)

È un interprete Triad-Core completo in un solo file CommonJS (~1000 righe, nessuna dipendenza). È lo strumento con cui si verifica la conformance Core su Linux/macOS ed è il modo più rapido per capire la semantica di Core leggendo codice.

34.1 Architettura

```
sorgente ──tokenize()──▶ token ──Parser──▶ AST {kind: …} ──evalStmt/evalExpr──▶ effetti su stdout
```

1. **Valori** (in cima al file): oggetti `{t, v}` costruiti da `I` (Int, `BigInt` con `wrap()` a 64 bit), `B`, `S`, `Y` (Symbol), `L` (List), `F` (funzione: `params`, `body`, `env`, `isLambda`) e la costante `NOTHING`. I builtin sono `{t:'builtin', name}`.
2. **`tokenize(src)`**: produce token `SEP` (a capo o `;`), `NUM`, `STR`, `SYM`, `IDENT`, `OP`, `LPAREN`/`RPAREN`, `LBRACKET`/`RBRACKET`, `COMMA`, `QMARK`, `COLON`, `EOF`. Commenti e spazi sono scartati qui.
3. **`class Parser`**: discesa ricorsiva che segue la precedenza del §7.1: `parseProgram → parseStmt → parseExpr → parseAssign → parseTernary → … → parsePostfix → parsePrimary`, più `parseBlock(terminators)` per i corpi fino a `end`/`else`/`elseif`. Nodi prodotti: `program`, `funcdef`, `return`, `break`, `continue`, `exprstmt`, `num`, `str`, `sym`, `bool`, `nothing`, `var`, `list`, `lambda`, `assign`, `indexassign`, `binary`, `unary`, `ternary`, `call`, `index`, `if`, `while`, `let`, `begin`.
4. **Ambienti**: `makeFrame(parent, owner, isLambda)` crea un frame con una `Map` di legami; `lookup` risale la catena. `writeVar` implementa le regole di scope del §10.3: aggiorna un legame dello stesso "proprietario" (funzione), oppure, dentro una lambda, un legame esterno; altrimenti crea un locale.
5. **Valutazione**: `evalStmt`, `evalExpr` → `evalExprBinary` → `evalExprRest` → `evalExprBlocks`. `return`, `break`, `continue` sono eccezioni JS (`ReturnSignal`, `BreakSignal`, `ContinueSignal`) catturate dalla chiamata o dal `while`.

6. **Builtin**: insieme `BUILTIN_NAMES` + `callBuiltin(name, args, env)` ; stampa con `printValue` (simboli senza `:` solo al top level) e conversione con `stringOf`.
7. **`main()`** : legge il file (o stdin con `-`), lega `ARGS`, esegue; errori di parsing → `parse error: …`, errori di esecuzione → `error: …`, entrambi con exit 1; nessun file → exit 2.

Questo flusso separa responsabilità che conviene diagnosticare una alla volta. Se il token prodotto da `tokenize` è sbagliato, il problema è prima del parser; se il parser genera la forma errata, l'albero AST non corrisponde alla grammatica; se l'albero è giusto ma il risultato no, il difetto è nell'ambiente, nella valutazione o in un builtin. Un caso di conformance piccolo permette di fissare il comportamento osservato mentre si corregge il livello responsabile. Quando si modifica sintassi o semantica, il test, l'interprete e la specifica devono restare coerenti: aggiungere un ramo al parser senza la corrispondente valutazione crea una forma riconosciuta ma non eseguibile.

34.2 Esempio: aggiungere un builtin

Supponiamo di voler sperimentare un builtin `abs` (non è in Core: va prima proposto nello standard, vedi §43). Servono due modifiche:

```
// 1. registrare il nome
const BUILTIN_NAMES = new
Set(['print','println','length','push!','Int','string','Symbol',
  'typeof','readfile','strlen','strbyte','strsub','exit','abs']);

// 2. implementarlo in callBuiltin, prima del throw finale
  if (name === 'abs') {
    if (args.length !== 1 || args[0].t !== 'int') throw new Error('abs');
    return I(args[0].v < 0n ? -args[0].v : args[0].v);
  }
```

Poi: un caso in `conformance/core/cases/` (§38) e `node conformance/run.mjs`. Ricorda che gli `Int` sono `BigInt` (`0n`, `1n`) e che i risultati vanno costruiti con gli helper (`I`, `B`, `S`, `Y`, `L`, `NOTHING`).

34.3 Esempio: aggiungere sintassi

1. se serve un nuovo token, estendi `tokenize` (es. un operatore in `twoOps`);
2. aggiungi il livello di precedenza nel `Parser` producendo un nuovo `kind`;
3. gestisci il `kind` in `evalExpr*`;
4. aggiorna `SPEC-CORE.md` (grammatica §4 e semantica §5) e aggiungi un caso Core.

Regola: l'interprete JS deve restare **fedele a SPEC-CORE**; non aggiungere estensioni che il seed e S1 non dovrebbero avere.

---

35. Il seed S0 Windows e il suo generatore

35.1 Cos'è

`seed/windows-x86_64/seed.exe` è un PE32+ x86-64 di **25.088 byte** che importa solo `KERNEL32.dll` (`GetStdHandle`, `GetCommandLineA`, `CreateFileA`, `ReadFile`, `WriteFile`, `CloseHandle`, `ExitProcess`, `VirtualAlloc`). Non usa libc né runtime. La **fonte** è il dump `seed.hex` (SHA-256 verificato dalla CI: `3A75A21E2F47D8DC4C88D4BC2580A85D5349BCDF7550E5FFA16E2C0AC7CA8C11`).

Il seed **non costruisce un AST**: interpreta direttamente il testo sorgente, con un cursore (`rsi`) che scorre i byte; i corpi di funzioni e lambda sono puntatori nel sorgente che vengono rieseguiti a ogni chiamata.

35.2 Convenzioni interne

registro	ruolo
`r14`	base della memoria `VirtualAlloc` (sorgente a `+0`, ambiente a `+0x10000`, funzioni a `+0x20000`, heap a `+0x30000`)
`r15`	ambiente variabili (entry da 24 byte: nome ptr, len, valore)
`rbx`	contatore variabili
`r13`	tabella funzioni (entry da 128 byte)
`r12`	handle di stdout
`rsi`	cursore nel sorgente

Valori taggati a 2 bit: `Int = n << 2`, `false = 1`, `true = 5`, chiusura `= ptr | 2`, oggetti heap (stringhe `[tipo][len][byte…]`) con tag 3. Il layout completo (header PE, IAT, mappa delle etichette `parse_program`, `st_if`, `parse_primary`, `env_find`, …) è in `seed/windows-x86_64/README.md`.

35.3 Il generatore `seed/dev/gen_seed.js`

Scrivere migliaia di byte a mano è impraticabile: il generatore è un **assembler-lite** in JavaScript che emette i byte e risolve le etichette. Sezioni del file:

sezione	cosa
`emitter`	`B(...)` emette byte, `B32(v)` un dword, `L(nome)` definisce un'etichetta, `reloc` / `je` / `jne` / `jmp` / `callL` emettono salti con patch relative, `callIat(i)` chiama l'import `i`
helper di istruzione	funzioni con il nome dell'istruzione (`incRsi` , `cmpByteRsiImm('(')` , `testRax` , `leaRdxR14(d)` , …) che emettono i byte x86-64 corrispondenti
`PE/data layout`	costanti `IAT = 0x4000` , `SEC_RAW` , `HEADERS`
`main`	prologo, lettura della riga di comando, apertura/lettura del file
`parser`	il cuore dell'interprete: statement, espressioni, funzioni, lambda
`ambiente`	`env_find` , `env_store` , `env_assign`
`exit labels`	`fail` , `read_fail` , `open_fail` , `usage`
`PE builder`	scrive header DOS/PE/COFF, sezione `.text` , tabelle import, e il dump esadecimale

Esempio di stile (dal `main`): saltare il nome del programma nella riga di comando.

```
 cmpByteRsiImm(0x22); jne('skip_plain');   // il path inizia con " ?
L('q_loop');
incRsi();
cmpByteRsiImm(0); je('no_arg');           // fine stringa: nessun argomento →
exit 2
cmpByteRsiImm(0x22); jne('q_loop');
```

35.4 Flusso per modificare il seed

1. modifica `seed/dev/gen_seed.js` (aggiungi un'etichetta, un ramo nel dispatcher, …);
2. rigenera il dump: `node seed/dev/gen_seed.js seed/windows-x86_64/seed.hex` (oppure `bin/triad-cli seed generate windows`, che stampa anche lo SHA-256);
3. decodifica su Windows: `certutil -decodehex -f seed\windows-x86_64\seed.hex seed\windows-x86_64\seed.exe`;
4. esegui i casi Core sul seed e il probe `test_get.td`;
5. aggiorna **insieme**: hash in `.github/workflows/ci.yml` (job `seed-windows`, step `Hash check`), `seed/windows-x86_64/README.md`, `seed/dev/README.md`, `STATUS.md`; se un nuovo caso Core passa sul seed, aggiungilo all'array `$cases` del job.

Trappole note (da `STATUS.md`): il codice cresce e può sovrapporsi ai dati (i nomi import ora partono da RVA `0x5000`); registri usati prima di essere inizializzati (bug `pbs_int`); ordine degli argomenti sullo stack nelle chiamate builtin (`get`). Defender può segnalare il PE come falso positivo (la CI aggiunge un'esclusione).

---

36. Il seed Linux (E0)

`seed/linux-x86_64/seed` è un ELF64 statico di **173 byte** generato da `seed/dev/gen_seed_linux.js`: entry `0x400078`, un solo `PT_LOAD` R|X, syscall dirette `write`/`exit`. Stampa `Triad seed Linux OK` ed esce con 0. È il primo passo della milestone M7; E1/E2/E3 (lettura file, interprete) sono da fare, riusando la logica del seed Windows con un layer `os_*` diverso.

```
node seed/dev/gen_seed_linux.js seed/linux-x86_64/seed.hex   # verifica magic,
entry, round-trip
xxd -r -p seed/linux-x86_64/seed.hex seed/linux-x86_64/seed && chmod +x
seed/linux-x86_64/seed
./seed/linux-x86_64/seed                                     # Triad seed Linux
OK
```

---

37. Self-hosting: S1 in `selfhost/`

S1 è l'interprete di tutto Triad scritto in Triad-Core ed eseguito dal seed. Oggi è allo stadio di prototipi separati, ognuno con un piccolo test in fondo al file:

file	cosa prova
`main.td`	ambiente inline come lista piatta `[chiave, valore]`; smoke test in CI (atteso `Value: 10`)
`value.td`	valori taggati a 2 bit simulati con interi (`make_int`, `make_bool`, `as_int`)
`tokenizer.td`	`next_token(chars, pos)` su un sorgente codificato come lista `[lunghezza, byte…]`; token `[tipo, valore, pos]`
`parser.td`	riconoscimento delle keyword `if`/`end` dai codici ASCII
`eval.td`, `program.td`	valutazione di token numerici, identificatori, assegnamenti e `println`
`env.td`, `heap.td`, `ast.td`, `interpreter.td`	stub dell'ambiente, dell'allocatore bump e dell'AST

37.1 Come si scrive codice S1 oggi

Il codice S1 deve girare sul **seed Windows**, quindi segue le restrizioni della colonna destra del capitolo 15:

- usa `get(xs, i)` con indici **da 0** invece di `xs[i]` (non standard: andrà sostituito con l'indicizzazione Core quando il seed la supporterà);
- liste solo di interi: le stringhe sorgente si rappresentano come liste di codici ASCII, con la lunghezza nella posizione 0;
- `push!` restituisce la lista: si scrive `env = push!(env, v)`;
- costanti per i tipi di token (`0` EOF, `1` NUMBER, `2` IDENTIFIER, …) documentate in un commento in testa al file;
- funzioni con al massimo 4 parametri, lambda a un parametro, niente cattura multi-binding.

```
# stile S1: ambiente piatto e lookup con get (0-based)
env = []
env = push!(env, 1)      # chiave
env = push!(env, 10)     # valore
if get(env, 0) == 1
  println("Value: " * string(get(env, 1)))
end
```

Su Linux questi file **non girano** con l'interprete JS (che non ha `get`): vanno provati su Windows con `seed\windows-x86_64\seed.exe selfhost\main.td`.

37.2 Dove va S1

Le task S1.3–S1.15 sono in `TASK.md`: valori taggati, tokenizer e parser inline, eval di espressioni, `let`, programmi completi, funzioni e chiamate, chiusure, e infine far girare la conformance sopra seed+S1. Gli esercizi 6 e 7 del capitolo 17 sono una versione in Core standard degli stessi passi.

---

38. Conformance: leggere, scrivere e verificare casi

38.1 Formato

```
conformance/core/cases/<nome>/      (oppure conformance/cases/<nome>/ per Full)
  program.td        programma Triad eseguito come script
  args.txt          opzionale: un argomento per riga (ARGS)
  expected.out      stdout atteso, byte per byte, fine riga LF
  expected.golden   copia di expected.out (fallback del runner)
  expected.code     exit code atteso
```

Un caso passa se stdout e codice di uscita coincidono; stderr non è confrontato. Niente stdin né input interattivo.

38.2 Il runner `conformance/run.mjs`

```
node conformance/run.mjs                                # Core (default
conformance/core/cases)
node conformance/run.mjs --cases conformance/cases    # Full (oggi passa solo
errore)
node conformance/run.mjs --skip liste,valori          # salta casi (default:
liste)
node conformance/run.mjs --update                     # RIGENERA i golden dal
runtime corrente
```

Runtime scelto, in ordine: variabile `TRIAD_SEED`; su Linux/macOS l'interprete JS; su Windows `bin/triad-native.exe`, poi `seed/windows-x86_64/seed.exe`. Esempio per provare il seed con un percorso esplicito: `TRIAD_SEED=/percorso/seed node conformance/run.mjs`.

Su Linux/macOS il runner usa l'interprete JS e verifica tutti i 37 casi. Su Windows usa il seed e salta per impostazione predefinita il caso `liste`, perché il seed non implementa l'indicizzazione standard `xs[i]`: il runner stampa quindi 36 casi. Per provare il caso escluso con l'interprete JS:

```
node seed/dev/triad_core.cjs conformance/core/cases/liste/program.td
```

Messaggi:

- `PASS nome` / `FAIL nome (exit X, atteso Y, stdout N byte, attesi M)`;
- `golden mancante`: il checkout non ha `expected.out` (di solito basta `git pull`).

Un golden è un'aspettativa versionata, non la fonte normativa della semantica. Il file `program.td` definisce l'input, `args.txt` gli argomenti, `expected.out` l'output byte per byte e `expected.code` il codice d'uscita. Il runner confronta questi risultati con il runtime scelto; così uno stesso caso può essere eseguito da più interpreti e rendere visibili le divergenze. Se il comportamento del runtime è in dubbio, confronta prima il caso con SPEC-CORE e poi correggi il golden solo quando la specifica conferma il nuovo valore atteso. L'opzione `--update` copia il comportamento corrente nei risultati attesi: usarla senza revisione trasformerebbe un bug in una pretesa di correttezza.

38.3 Aggiungere un caso Core

1. scegli un nome descrittivo in italiano (`liste_annidate`, `stringhe_vuote`, …);
2. scrivi `program.td` usando **solo** SPEC-CORE §2–§7 (niente primitivi di bootstrap: devono girare anche su Full);
3. scrivi a mano `expected.out` (con LF finale) ed `expected.code` (`0`), ragionando sulla specifica — **non** generarli alla cieca con `--update`, altrimenti fissi i bug del runtime;
4. copia `expected.out` in `expected.golden`;
5. `node conformance/run.mjs` deve stampare `PASS <nome>`;
6. se il caso passa anche sul seed Windows, aggiungilo a `$cases` nel job `seed-windows` di `.github/workflows/ci.yml`.

```
mkdir conformance/core/cases/somma_lista
cat > conformance/core/cases/somma_lista/program.td <<'TD'
xs = [3, 4, 5]
tot = 0
i = 1
while i <= length(xs)
    tot += xs[i]
    i += 1
end
println(tot)
TD
printf '12\n' > conformance/core/cases/somma_lista/expected.out
cp conformance/core/cases/somma_lista/expected.out
conformance/core/cases/somma_lista/expected.golden
printf '0' > conformance/core/cases/somma_lista/expected.code
node conformance/run.mjs | grep somma_lista          # PASS  somma_lista
```

I golden Full (`conformance/cases/`) sono **congelati**: non vanno rigenerati finché non esiste un'implementazione Full.

---

39. La CLI di sviluppo `triad-cli`

39.1 Uso

comando	cosa fa
`triad-cli status`	versione Node, piattaforma, seed trovato, root del progetto
`triad-cli test conformance [-c DIR] [-u]`	esegue `conformance/run.mjs` (default Core, `--skip liste`)
`triad-cli test run`	alias della conformance Core (i vecchi test Julia non esistono più)
`triad-cli ai verify [-t DIR] [-v]`	esegue le `solution.triad` di `ai/tasks/` sul seed nativo e le confronta con i golden
`triad-cli seed generate windows|linux [-o FILE]`	esegue il generatore del seed e stampa lo SHA-256 del dump
`triad-cli build standalone`	non disponibile: stampa un messaggio

Avvio: `node cli/triad-cli.mjs …` (dopo `cd cli && npm install`), i launcher `bin/triad-cli` / `bin\triad-cli.cmd` (installano le dipendenze al primo avvio e si fermano se `npm install` fallisce), oppure l'eseguibile standalone della release.

39.2 Struttura del codice

```
cli/
  triad-cli.mjs            definizione dei comandi con commander
  lib/utils.mjs            findProjectRoot, findSeed, runSeed, runNode,
projectPath, …
  lib/commands/test.mjs    handleTestRun, handleTestConformance
  lib/commands/ai.mjs      handleAIVerify
  lib/commands/seed.mjs    handleSeedGenerate (windows/linux)
  lib/commands/build.mjs   handleBuildStandalone
  build-exe.mjs            build dell'eseguibile Node SEA (esbuild + postject)
```

- `findProjectRoot()` cerca la root (una cartella con `conformance/run.mjs` e `seed/`) partendo da `TRIAD_HOME`, dalla cartella del modulo, da quella dell'eseguibile e dalla cartella corrente.
- Ogni handler restituisce `true`/`false`; `triad-cli.mjs` lo traduce in exit code 0/1.

39.3 Aggiungere un comando

1. crea `cli/lib/commands/<nome>.mjs` con una funzione `handle<Nome>(options)` che restituisce un booleano e usa `projectPath`, `runNode`, `runSeed` da `utils.mjs`;
2. registralo in `triad-cli.mjs`:

```
import { handleLint } from './lib/commands/lint.mjs';


program
  .command('lint')
  .description('Controlla i file .triad')
  .option('-v, --verbose', 'Verbose output')
  .action(async (options) => {
    const success = await handleLint(options);
    process.exit(success ? 0 : 1);
  });
```

1. aggiungi la riga in `status` e documentalo in `cli/README.md`.

39.4 Eseguibile standalone

```
cd cli && npm install
npm run build:exe                          # dist/triad-cli per la piattaforma
corrente
node build-exe.mjs path/to/win/node.exe    # dist/triad-cli.exe per Windows, anche
da Linux
```

Il binario Node passato determina la piattaforma; deve avere la stessa versione del Node che esegue la build.

---

40. Benchmark AI (`ai/`)

Misura quanto bene un LLM scrive Triad-Core.

- `ai/SYSTEM_PROMPT.md`: prompt di sistema con il sottoinsieme ammesso (prudente: vieta liste, `string()`, `elseif`, `return` perché il seed non li supportava tutti);
- `ai/tasks/NN_nome/`: `task.md` (richiesta), `expected.out`, `solution.triad` (soluzione di riferimento);
- verifica: `node cli/triad-cli.mjs ai verify` (o `--tasks ai/tasks/01_somma`): `PASS` / `FAIL`, `SKIP` se manca la soluzione;
- `ai/jev_audit.mjs`, `ai/jev_prompt.md`: audit separato (vedi `PROMPT_JEV.md`).

Aggiungere un task: crea `ai/tasks/06_nome/` con `task.md` (output atteso descritto con precisione), `expected.out` e una `solution.triad` di riferimento che passa. Esegui il codice generato da un modello solo in un ambiente isolato.

---

41. Editor, launcher e sito

- **VS Code** (`editors/vscode/`): `package.json` dichiara il linguaggio `triad` (estensioni `.triad`, `.td`); `syntaxes/triad.tmLanguage.json` è la grammatica TextMate (`source.triad`) con le regole `keywords`, stringhe con interpolazione, numeri, `:-`, shebang; `language-configuration.json` gestisce commenti e parentesi. Pacchetto: `cd editors/vscode && npx @vscode/vsce package`.
- **Vim/Neovim** (`editors/vim/`): `ftdetect/triad.vim` e `syntax/triad.vim`.
- **Linguist** (`editors/linguist/`): voce `languages.yml` pronta per una PR a GitHub Linguist.
- Se aggiungi una parola chiave allo standard, aggiornala in **tutte** le grammatiche.
- **Launcher** (`bin/`): `triad` (POSIX, risolve i symlink e calcola `TRIAD_HOME`) e `triad.cmd` (Windows, file CRLF); vedi §2.4.
- **Sito** (`docs/`): HTML statico pubblicato su GitHub Pages; `docs/assets/js/app.js` carica i Markdown del repository con `marked` (es. `docs/pages/tutorial.html` carica questo file). Le ancore dell'indice usano `<a id="…">` espliciti perché funzionino anche lì.

---

42. Integrazione continua

`.github/workflows/ci.yml`, a ogni push e pull request:

job	cosa verifica
`conformance-core` (Ubuntu, Node 20)	`node conformance/run.mjs --cases conformance/core/cases --skip liste` con l'interprete JS
`seed-windows`	decodifica `seed.hex`, esegue 33 casi Core sul PE confrontando stdout, probe `test_get.td` (atteso `20`), smoke test `selfhost/main.td` (atteso `Value: 10`), hash di `seed.hex`
`seed-linux`	decodifica l'ELF E0, lo esegue e ne verifica l'hash

`pages.yml` e `static.yml` pubblicano `docs/` su GitHub Pages a ogni push su `main`.

Il profilo Full **non è in CI** perché non è implementato: non presentarlo come verde.

---

43. Flusso di contributo

Checklist per tipo di modifica:

modifica	da fare
comportamento del linguaggio	aggiorna `SPEC.md` / `SPEC-CORE.md`, `CHANGELOG.md` e, se normativa, `VERSION`; aggiungi o aggiorna casi di conformance
interprete JS	resta fedele a SPEC-CORE; `node conformance/run.mjs` verde
seed Windows	rigenera `seed.hex`, aggiorna hash in CI e nei README, prova su Windows
S1 (`selfhost/`)	gira su `seed.exe`; se cambia l'output di `main.td`, aggiorna lo smoke test in CI
CLI	handler che restituisce un booleano; `cli/README.md`; ricostruisci l'eseguibile se serve
editor	grammatiche VS Code e Vim allineate alle parole chiave di SPEC §1
documentazione	`README.md`, `STATUS.md` e i testi del repository descrivono ciò che gira davvero

Regole generali: non dichiarare funzionante ciò che non è verificato; i golden si scrivono ragionando sulla specifica; i nuovi strumenti di sviluppo non devono entrare nella catena S0→S4.

Parte V — Riferimento

44. Grammatica Triad-Core (EBNF)

Da `SPEC-CORE.md` §4:

```
 program  = { sep } { stmt sep { sep } } ;
stmt     = funcdef | "return" [ expr ] | "break" | "continue" | expr ;
funcdef  = "function" NAME "(" [ NAME { "," NAME } ] ")" block "end"
         | NAME "(" [ NAME { "," NAME } ] ")" "=" expr ;
expr     = assegnamento ;
assegnamento = NAME ( "=" | "+=" | "-=" | "*=" ) expr | ternario ;
ternario = or_expr [ "?" expr ":" expr ] ;
or_expr  = and_expr { "||" and_expr } ;
and_expr = cmp_expr { "&&" cmp_expr } ;
cmp_expr = sum_expr { ( "==" | "!=" | "<" | "<=" | ">" | ">=" ) sum_expr } ;
sum_expr = mul_expr { ( "+" | "-" ) mul_expr } ;
mul_expr = pow_expr { ( "*" | "%" ) pow_expr } ;
pow_expr = unary { "^" unary } ;                    (* destra *)
unary    = ( "-" | "!" ) unary | postfix ;
postfix  = primary { "(" args ")" | "[" expr "]" } ;
primary  = INT | STRING | SYMBOL | "true" | "false" | "nothing" | NAME
         | "(" expr ")"
         | "[" [ expr { "," expr } ] "]"
         | "(" NAME { "," NAME } ")" "->" expr
         | "if" expr block { "elseif" expr block } [ "else" block ] "end"
         | "while" expr block "end"
         | "let" [ NAME "=" expr { "," NAME "=" expr } ] block "end"
         | "begin" block "end"
         | "function" "(" [ NAME { "," NAME } ] ")" block "end" ;
block    = { stmt sep { sep } } ;                   (* fino a end/else/elseif *)
```

La grammatica Full (Pratt, 12 livelli) è in `SPEC.md` §2.

---

45. Schede rapide Core e Full

Triad-Core (eseguibile oggi):

```
assegnamento:  x = e    x += e    x -= e    x *= e    xs[i] = e
funzione:      f(a, b) = e
               function f(a, b) ... end        (anche !io, ignorato)
anonima:       x -> e    (a, b) -> e    function (a) ... end
condizione:    if c ... elseif c ... else ... end    c ? a : b
ciclo:         while c ... end    break    continue
blocchi:       let a = 1, b = 2 ... end    begin ... end
uscita:        return e
liste:         [1, 2]    xs[i] (da 1)    length(xs)    push!(xs, v)
stringhe:      "a" * "b"    length(s)    string(x)
conversioni:   Int("42")    Symbol("s")    typeof(x)
I/O:           print(...)    println(...)    ARGS
bootstrap:     readfile(p)  strlen(s)  strbyte(s, i)  strsub(s, a, b)  exit(n)
```

Triad-Full (specifica):

```
struttura:   struct Nome; x::T; y::T; end
funzione:    function f(a::T)::R !eff ... end
fatto:       fact p(:a, :b)
regola:      rule p(?x, ?y) :- q(?x), r(?y)
query:       solve p(?x) limit N depth D steps S trace
destruttura: (a, b) = [1, 2]        for (k, v) in [[1,2],[3,4]] ... end
ciclo for:   for v in it ... end    1:n    xs[2:end]    [e for x in xs if c]
try/catch:   try b catch e h end / try b end
contratto:   requires c / ensures c
budget:      with budget(steps = N) ... end
traccia:     with trace ... end / solve p trace
```

```
effetto:     function f() !io ... end
macro:       macro m(a) ... end / @m(e)
quote:       :(espr) / quote ... end / $espr
eval:        eval(codice)
```

---

46. Debug e problemi comuni

Core (oggi):

sintomo	causa probabile	soluzione
`parse error: expected RPAREN`	`1.5`, `/`, `() -> …` o altra sintassi Full	resta nel sottoinsieme Core (§4)
`"x = $x"` stampato letteralmente	l'interpolazione è solo Full	`"x = " * string(x)`
`error: if cond`	condizione non `Bool`	usa un confronto (`x != 0`)
`error: < type error` su `a < b < c`	confronti concatenati	`a < b && b < c`
una funzione non aggiorna un globale	le funzioni nominate assegnano in locale	restituisci il valore o usa una lista (§16.4)
`error: undefined variable f`	funzione chiamata prima della definizione	definiscila più in alto
`FAIL … (golden mancante …)`	checkout senza `expected.out`	`git pull`; controlla `.gitignore`
il seed esce con 1 senza messaggi	funzionalità non supportata dal seed	prova con l'interprete JS; vedi §15
il seed Windows viene bloccato	falso positivo dell'antivirus	escludi `seed\windows-x86_64`

Full (specifica):

sintomo	causa probabile	soluzione
`Err(:syntax)` "atteso fine istruzione"	manca `end` o un separatore	aggiungi `end` o `;`
`Err(:undef)`	nome non definito	definiscilo, o usa `:nome` per un simbolo
`Err(:method)` / `Err(:ambiguous)`	nessun metodo / più metodi ugualmente specifici	aggiungi o ridisegna le firme
`Err(:bounds)` in destrutturazione	arità diversa dal pattern	liste di lunghezza esatta
`Err(:effect)`	`!io`/`!state` chiamata da funzione pura	dichiara l'effetto nel chiamante
`Err(:budget)` in `solve`	troppi passi	aumenta `steps` o usa `limit`/`depth`

Strumenti Full previsti: `triad --tree file`, `sexpr(codice)`, `parse`/`unparse`, `with trace … end`, `solve … trace`, `try … end` con `.kind`/`.msg`/`.data`.

---

47. Glossario

- **S0 / seed**: interprete Triad-Core in codice macchina (`seed/`).
- **S1**: interprete di tutto Triad scritto in Triad-Core (`selfhost/`).
- **S2 / S3 / S4**: compilatore nativo, self-hosting, runtime completo (vedi `BOOTSTRAP.md`).
- **E0 / E1 / …**: stadi del port Linux del seed (milestone M7).
- **Triad-Core / Triad-Full**: i due profili di conformità.
- **Interprete JS**: `seed/dev/triad_core.cjs`, riferimento Core per sviluppo e CI.
- **Golden**: output atteso committato (`expected.out`, `expected.code`).
- **Primitivi di bootstrap**: `readfile`, `strlen`, `strbyte`, `strsub`, `exit` (solo Core).
- **Lista taggata**: record Core del tipo `[:Tag, campo1, …]`.
- **Node**: l'unico tipo di albero di Triad Full (`head`, `args`): codice e dati coincidono.
- **Solution**: `{nome = valore, …}` restituita da `solve`; `s.nome` o `s[:nome]`.
- **Err**: errore come dato (`kind`, `msg`, `data`).
- **Traced**: `(result, trace)` restituito da `with trace` / `solve … trace`.
- **Occurs-check**: l'unificazione rifiuta `?x = f(?x)`.
- **Multiple dispatch**: scelta del metodo più specifico tra gli applicabili.
- **Effetti dichiarati**: `!io`, `!state` nella firma; controllo dinamico.
- **Contratti**: `requires` (pre) / `ensures` (post, con `result`).
- **Budget**: `with budget(steps = N)`; l'esaurimento è `Err(:budget)`.
- **Destrutturazione**: `(a, b) = lista`, `for (k, v) in lista_di_liste`.

---

48. Stato, limiti noti e fix storici

Stato (dettagli in `STATUS.md`):

- Core: 37/37 casi con l'interprete JS; 33 casi sul seed Windows in CI.
- Full: 12 casi golden congelati, passa solo `errore`.
- S1: prototipi in `selfhost/`, smoke test in CI; non ancora conformance.
- S2–S4: non iniziati. Seed Linux: E0.
- CLI `triad`: solo esecuzione di file (REPL e opzioni di SPEC §14 da fare).

Limiti dello standard (SPEC §15): argomenti con nome (salvo `budget(steps = …)`), struct mutabili, tipi astratti utente e parametrici, macro senza parentesi, quote annidate a più livelli, cut di Prolog non sono previsti; il controllo degli effetti è dinamico.

Fix storici applicati alla vecchia implementazione Julia; il comportamento "dopo" è quello richiesto da SPEC.md per ogni implementazione:

#	fix	dopo
1	round-trip del range	`unparse(1:n)` produce `"1:n"` (senza spazi)
2	`unparse` di `:destructure`/`:dfor`	hanno rami propri e fanno round-trip
3	`xs[end] = v`	`end` è legato anche in scrittura: `xs[end] = 7` funziona

#	fix	dopo
4	escape sconosciuti	solo `\n \t \" \\ \$`; altri → `Err(:syntax)` con riga e colonna
5	commenti `#=` annidati	la profondità è contata; `#=` non chiuso → errore `incomplete`
6	lambda con parametri tipati	`function (x::Int) x end` controlla i tipi: `f(2.0)` → `Err(:type)`

Quarta di copertina

Triad 2.0 è un linguaggio di programmazione autonomo, definito da una specifica normativa indipendente da qualsiasi implementazione.

Il progetto è rilasciato sotto licenza MIT. SPEC.md (standard 0.2.0), SPEC-CORE.md, conformance/ con golden verificabili.

L'obiettivo è un linguaggio autonomo e riconosciuto: standard proprio, conformance eseguibile, estensione .triad, tooling editor, bootstrap senza linguaggi ospiti e self-hosting.